Applied Mathematics

A Hot-Started Nullspace Active-Set Method for Regularized Column Generation

with Equality and Inequality Linking Constraints

Author: *Kyrian Bäumer*

October 17, 2026

Abstract

Column generation solves linear programs with a very large number of variables by working on a restricted master problem that is enlarged one column at a time. Its classical form is known for unstable dual iterates and slow convergence. A quadratic proximal term in the dual space removes most of this instability, but it turns every master problem into a convex quadratic program, which is considerably more expensive to solve than a linear program.

This work develops a **hot-started nullspace active-set method** for these quadratic master problems, for multi-block problems whose linking constraints may be equalities, "$\geq$" inequalities or "$\leq$" inequalities. Surplus variables bring all linking rows into one equality form. The convexity constraints of the blocks then admit a reordering of the variables into five sets, which yields an explicit and sparse nullspace basis. The reduced KKT system decouples: the surplus part of the search direction is given by an explicit formula, and only an *effective reduced Hessian* on the column weights has to be factorized. This matrix is the reduced Hessian of the equality rows together with the inequality rows whose surplus is fixed at zero. Every change of the working set becomes a low-rank modification of an $LDL^\top$ factorization of this matrix, which is updated in place and carried over from one column generation iteration to the next. The Lagrange multipliers of the method coincide with the reduced costs of column generation and with the dual sign conditions of the inequality rows, and singular reduced Hessians are resolved by an exact zero-curvature step.

Two factorizations are developed. The first is a sparse factorization of fixed dimension, updated with the sparse direct solver CHOLMOD, in which periodic refactorization controls the structural zeros left by the updates. The second exploits that the effective reduced Hessian is small and dense: it is factorized densely in the order in which its indices became active, so that a new index is appended and a removed index is deleted by a rank-one update. This factor never contains structural zeros, needs no periodic refactorization, and its pivots detect singular reduced Hessians exactly.

On generated airline crew scheduling instances with up to 960 linking rows the method reproduces the optimum of the full linear program, and it solves the regularized master problems about three orders of magnitude faster than a general-purpose quadratic programming solver. The dense factorization is two to four times faster than the sparse one, with a speedup that grows with the number of linking rows, and needs an order of magnitude less memory. The complete column generation run is much faster than classical column generation with a linear master problem, and, on all but the smallest instance, faster than solving the full linear program directly.

Since the stabilization parameter does not enter the factorization, it can be changed in every iteration at no cost. Interpreted as the step size of a proximal point method, it should be small while the restricted master problem stays the same and large when new columns arrive. A geometric rule based on this removes the long tail of iterations that only move the stability center, makes the runs up to 2.6 times faster, and makes the final solution exact up to rounding errors.

Contents

Abstract ii

1 Introduction 1

1.1 The Computational Problem . . . 2
1.2 Contributions . . . 2
1.3 Outline . . . 3

2 Problem Formulation 4

2.1 Multi-Block Linear Programs with Mixed Linking Constraints . . . 5
2.2 Dantzig–Wolfe Reformulation . . . 6
2.3 Column Generation . . . 7
2.4 Surplus Variables and the Unified Formulation . . . 9
2.5 Summary of Notation . . . 11

3 Regularization of the Master Problem 12

3.1 Proximal Stabilization of the Dual . . . 13
3.2 The Explicit Regularized Master Problem . . . 13
3.3 The Eliminated Regularized Master Problem . . . 14
3.3.1 The dual vector as a function of the primal iterate . . . 15
3.4 Regularized Column Generation . . . 16
3.4.1 Center updates . . . 16
3.4.2 Exact termination . . . 16
3.4.3 The Lagrangian bound with inequality rows . . . 18
3.4.4 The algorithm . . . 18
3.5 An Adaptive Stabilization Parameter . . . 18
3.5.1 The stabilization parameter as a step size . . . 18
3.5.2 The geometric rule . . . 19

4 An Active-Set Method for the Regularized Master Problem 21

4.1 The Master Problem in Simplified Notation . . . 22
4.2 Optimality Conditions . . . 22
4.3 The Primal Active-Set Method . . . 23
4.4 The Constraint Matrix and the Reordering Theorem . . . 25

5 The Nullspace Method **28**

5.1 The Nullspace Basis . . . 29

5.2 The Reduced KKT System . . . 30

5.2.1 The reduced system . . . 30

5.2.2 Block structure . . . 30

5.2.3 Decoupling and the effective reduced Hessian . . . 31

5.2.4 The search direction . . . 33

5.2.5 Implicit evaluation . . . 34

5.3 The Lagrange Multipliers . . . 34

5.4 Summary of One Iteration . . . 35

6 Efficient Updates of the KKT System **37**

6.1 Fixed-Length Representation . . . 38

6.2 Expanding the Working Set . . . 39

6.3 Reducing the Working Set . . . 41

6.4 Updating the $LDL^\top$ Factorization . . . 42

6.4.1 Row additions and deletions . . . 42

6.4.2 Rank-one modifications . . . 43

6.4.3 Sparse solves . . . 43

6.4.4 Assembly of a new row . . . 43

6.5 Structural Zeros and Periodic Refactorization . . . 44

6.6 Hot Start Between Column Generation Iterations . . . 44

6.6.1 Center-only iterations . . . 45

6.7 The Complete Method . . . 46

7 Singular Reduced Hessians **47**

7.1 Where Singularity Can Arise . . . 48

7.2 Detection . . . 48

7.3 Zero-Curvature Directions . . . 49

7.3.1 Released column . . . 49

7.3.2 Released surplus . . . 50

7.4 The Zero-Curvature Step Removes the Singularity . . . 50

8 A Dense Factorization of the Effective Reduced Hessian **53**

8.1 Why a Dense Factor . . . 54

8.2 Compact Representation . . . 54

8.3 Appending an Index . . . 55

8.3.1 What the singularity test measures . . . 56

8.4 Rank-One Modifications . . . 57

8.5 Removing an Index . . . 59

8.6 Structural Zeros and Refactorization . . . 61
8.7 Comparison with the Sparse Factorization . . . 61

9 Computational Results 63

9.1 Test Problem: Airline Crew Scheduling . . . 64
9.1.1 The model . . . 64
9.1.2 Instance generator . . . 64
9.2 Implementation and Setup . . . 65
9.3 Correctness . . . 66
9.4 Convergence and the Two Phases . . . 68
9.5 Computational Time . . . 70
9.6 Inequality Rows . . . 73
9.7 Fill-In and Refactorization of the Sparse Factor . . . 74
9.8 The Sparse and the Dense Factorization . . . 76
9.9 An Adaptive Stabilization Parameter . . . 78
9.10 Discussion . . . 81

10 Conclusion 83

10.1 Summary . . . 83
10.2 Outlook . . . 84

Bibliography 85

Chapter 1

Introduction

Many large-scale planning problems, such as crew scheduling, vehicle routing and cutting stock, lead to linear programs with a block structure: a number of independent subproblems, one per crew member, vehicle or machine, are coupled by a comparatively small set of **linking constraints**. In crew scheduling, for instance, every pilot follows a schedule of their own, while the linking constraints require that each flight is covered by the pilots together.

Column generation is the standard method for such problems. The problem is first rewritten in terms of complete solutions of the subproblems (a schedule, a route, a cutting pattern), so that every variable, or *column*, represents one such solution. There are far too many columns to write down, but only a few of them are needed in an optimal solution. Column generation therefore works with a **restricted master problem** over a small set of columns and asks the subproblems, in the **pricing problem**, for a new column that can improve the current solution.

In its classical form the restricted master problem is a linear program. Its dual solutions, which drive the pricing problem, are known to jump from one extreme point to another, which makes the method unstable and slow in its final phase [20]. A well-established remedy is **dual stabilization** [9, 3], in its proximal form [3, 19]: the dual problem is augmented by a quadratic term that keeps the dual iterate close to a *stability center*. The price of this regularization is that every restricted master problem becomes a **convex quadratic program**.

This work is about solving this sequence of quadratic programs efficiently, for linking constraints that may be equalities as well as inequalities.

1.1 The Computational Problem

Two consecutive restricted master problems in column generation are almost identical. The new problem contains one more column, and with stabilization the stability center may have moved. A general-purpose quadratic programming solver does not exploit this: it rebuilds and refactorizes its internal linear algebra for every master problem, and its cost per master problem grows with the number of generated columns.

Active-set methods are well suited for sequences of closely related quadratic programs. They move between faces of the feasible region by adding or removing one constraint at a time, and the linear system they solve at each step changes only by a low-rank term. In the **nullspace** variant, this system is the reduced Hessian on the nullspace of the active constraints. Its size equals the number of degrees of freedom of the current face, not the number of variables.

The question addressed in this work is how far this structure can be exploited for the regularized master problem of column generation, in particular:

(i) What is the structure of the active constraints of the regularized master problem, and does it allow an explicit nullspace basis?

(ii) How small is the linear system that has to be factorized, how does it change when the working set changes, and which kind of factorization suits it?

(iii) Can the factorization be carried over from one column generation iteration to the next, so that a new column or a new stability center costs (almost) nothing?

(iv) What changes when the linking constraints are inequalities, which add sign conditions on the dual variables and destroy part of the structure?

(v) How does the resulting method compare in practice with general-purpose solvers and with classical column generation?

1.2 Contributions

The main results of this work are the following.

- **A unified formulation for mixed linking constraints.** Equality rows and inequality rows of both directions are written as equalities with surplus variables. The regularized master problem then has a single form for all cases (Chapters 2 and 3).
- **A five-set column ordering and an explicit nullspace basis.** The convexity constraints of the blocks, together with the surplus variables, admit an ordering of the variables into representatives, working-set columns, nullspace columns, active surpluses and free surpluses. The nullspace basis consists of vectors $e_j - e_{\mathrm{rep}(j)}$ and unit vectors, and it is never stored (Chapters 4 and 5).
- **Decoupling and the effective reduced Hessian.** The reduced KKT system splits into an explicit formula for the surplus directions and a system with the *effective reduced Hessian* $H_{\mathrm{eff}} = (A_{\mathrm{eff}} Z_x)^\top (A_{\mathrm{eff}} Z_x)$. Here A_{eff} holds the equality rows together with the inequality rows whose surplus is fixed at zero. Its dimension never exceeds the number of linking constraints (Chapter 5).

- **Multipliers are reduced costs.** The Lagrange multipliers of the active-set method are exactly the reduced costs of column generation and the dual sign conditions of the inequality rows. The optimality test of the master problem and the pricing test are therefore the same test, applied inside and outside the restricted master problem (Chapters 4 and 5).
- **Updates and hot start.** Every change of the working set is a row addition, a row deletion or a rank-one modification of an $LDL^\top$ factorization, realized with a sparse factorization of fixed size. A new column or a new stability center leaves the factorization unchanged (Chapter 6).
- **Exact treatment of singular reduced Hessians.** A dependent nullspace direction is resolved by a zero-curvature step that keeps every working-set variable except the released one exactly at its bound (Chapter 7).
- **A dense factorization of the effective reduced Hessian.** Since H_{eff} is small and dense, it is factorized densely, in the order in which the indices became active. A new index is appended to the factor, a removed index is deleted by a rank-one update of the indices behind it, and the factor never accumulates structural zeros, so periodic refactorization becomes unnecessary. The new pivot of a released column is exactly its Schur complement, which makes the detection of singular reduced Hessians exact (Chapter 8).
- **An adaptive stabilization parameter.** The stabilization parameter is the step size of a proximal point method on the restricted dual. Since it does not enter the factorization, it can be changed in every iteration at no cost. A geometric rule removes the long proximal phase of a fixed parameter and makes the final solution exact up to rounding errors (Chapters 3 and 9).
- **A computational study** on generated crew scheduling instances with equality and inequality coverage rows: correctness against the full linear program, timing, the two phases of convergence, the fill of the sparse factor and its refactorization, a comparison of the sparse and the dense factorization, and the adaptive stabilization parameter (Chapter 9).

1.3 Outline

Chapter 2 introduces the multi-block linear program with mixed linking constraints, its Dantzig–Wolfe reformulation, column generation and the surplus formulation. Chapter 3 adds the proximal regularization, eliminates the auxiliary variable and derives the relation between the gradient of the quadratic master problem and the reduced costs; it ends with a rule that adapts the stabilization parameter during the run. Chapter 4 sets up the active-set method for the regularized master problem and establishes the five-set reordering of the constraint matrix. Chapter 5 constructs the nullspace basis, solves the reduced KKT system with the effective reduced Hessian and recovers the Lagrange multipliers. Chapter 6 describes the updates of the factorization when the working set changes, the fixed-length representation, periodic refactorization and the hot start between column generation iterations. Chapter 7 treats singular reduced Hessians. Chapter 8 replaces the sparse factorization by a dense factorization of the effective reduced Hessian alone and derives its update operations. Chapter 9 presents the computational results, and Chapter 10 concludes.

Throughout, the reader is assumed to be familiar with linear programming duality, the basic idea of column generation and the basic idea of a primal active-set method for quadratic programming, as found in standard texts such as [22, 14, 8]. Everything beyond this is derived in the text.

Chapter 2

Problem Formulation

This chapter introduces the class of problems considered throughout this work: linear programs with a multi-block structure, whose blocks are coupled by linking constraints that may be equalities or inequalities.

We first state the problem in its original (compact) form and then apply the **Dantzig–Wolfe reformulation**, which replaces the variables of each block by convex combinations of the extreme points of the block. This leads to the **master problem**, a linear program with a very large number of columns, which is solved by **column generation**. We define the restricted master problem, the reduced cost, the pricing problem and the Lagrangian bound, and collect them in the classical column generation algorithm.

Finally, we introduce **surplus variables** for the inequality rows. They bring every linking constraint into equality form and lead to a single, *unified* formulation of the master problem in which equality and inequality rows are treated alike. This formulation is the starting point for everything that follows. It also reveals the one structural property that the inequality rows destroy: the surplus variables belong to no block.

2.1 Multi-Block Linear Programs with Mixed Linking Constraints

We consider a linear program whose variables are partitioned into K blocks $\xi_1, \dots, \xi_K$, with $\xi_k \in \mathbb{R}^{n_k}$. Each block has its own constraints, and the blocks interact only through a set of linking constraints.

Definition 2.1.1 (Multi-block linear program)**.** A **multi-block linear program with mixed linking constraints** is a problem of the form

$$\min_{\xi_1,\dots,\xi_K} \quad \sum_{k=1}^{K} c^{(k)\top} \xi_k \tag{2.1a}$$

$$\text{s.t.} \quad \sum_{k=1}^{K} A_{E,k}\, \xi_k = b_E, \tag{2.1b}$$

$$\sum_{k=1}^{K} A_{I,k}\, \xi_k \geq b_I, \tag{2.1c}$$

$$\xi_k \in P_k, \qquad k = 1, \dots, K, \tag{2.1d}$$

where $A_{E,k} \in \mathbb{R}^{m_E \times n_k}$ and $A_{I,k} \in \mathbb{R}^{m_I \times n_k}$ are the coefficients of block k in the m_E **equality linking rows** and the m_I **inequality linking rows**, and

$$P_k = \{\, \xi_k \in \mathbb{R}^{n_k} \;:\; F_k \xi_k = b^{(k)},\ \underline{\xi}_k \leq \xi_k \leq \overline{\xi}_k \,\}$$

is the **block polytope** of block k, with block cost $c^{(k)} \in \mathbb{R}^{n_k}$, block constraint matrix F_k, right-hand side $b^{(k)}$ and bounds $\underline{\xi}_k$, $\overline{\xi}_k$. We write $m = m_E + m_I$ for the total number of linking rows. Constraints (2.1b)–(2.1c) are the **linking constraints**, and constraints (2.1d) are the **block constraints**.

Assumption 2.1.2. Every block polytope P_k is nonempty and bounded, and problem (2.1) is feasible.

Under Assumption 2.1.2 the feasible region of (2.1) is a nonempty polytope, so the problem has an optimal solution. Its optimal value is denoted by $z^\star$.

The constraint matrix of (2.1) has the **block-angular** structure shown in Fig. 2.1. The block constraints form a block-diagonal matrix, and only the linking rows couple different blocks. If the linking rows were absent, the problem would decompose into K independent problems over $P_1, \dots, P_K$. Column generation exploits exactly this observation.

Remark 2.1.3 (Inequalities of the other direction)**.** A linking row $\sum_k (A_{I,k})_{i,:}\, \xi_k \leq (b_I)_i$ is the row $\sum_k (-A_{I,k})_{i,:}\, \xi_k \geq -(b_I)_i$, so the form (2.1c) covers both directions, and the text only treats "$\geq$" rows. Remark 2.4.4 shows how a "$\leq$" row can be kept as it is instead of being negated.

Example 2.1.4 (Crew scheduling)**.** In the crew scheduling problem studied in Chapter 9, block k is the schedule of pilot k: a path from a source to a sink in a time–space network, with F_k the node–edge incidence matrix of the network. The linking rows are the *coverage rows*: row i counts how often flight i is operated, summed over all pilots. An equality row "$= 1$" requires that the flight is operated exactly once, and an inequality row "≥ 1" requires it at least once, which allows a pilot to travel on a flight as a passenger.

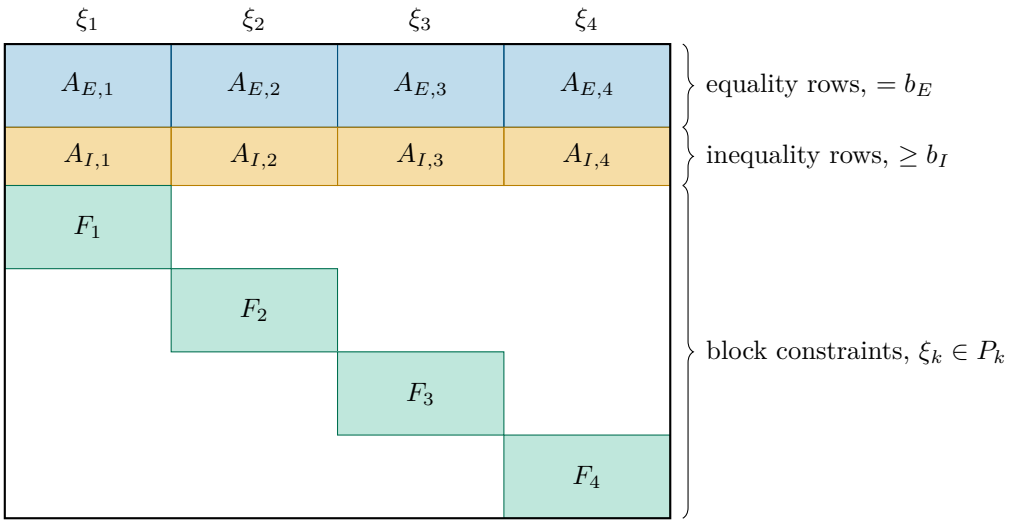


Figure 2.1: Block-angular structure of the multi-block linear program (2.1) for $K = 4$ blocks. The linking rows (top) couple all blocks; the block constraints (bottom) are block-diagonal.

2.2 Dantzig–Wolfe Reformulation

Since P_k is a nonempty polytope, it is the convex hull of its finitely many extreme points (Minkowski's theorem, see [25]). Let $\mathcal{J}_k$ be an index set of the extreme points of P_k, and let $\xi^{(j)}$, $j \in \mathcal{J}_k$, denote these extreme points. Every $\xi_k \in P_k$ can be written as

$$\xi_k = \sum_{j \in \mathcal{J}_k} \lambda_j \, \xi^{(j)}, \qquad \sum_{j \in \mathcal{J}_k} \lambda_j = 1, \qquad \lambda_j \geq 0. \tag{2.2}$$

Substituting (2.2) into (2.1) replaces the variables ξ_k by the weights λ_j. Each extreme point turns into one column of a new problem, which motivates the following definitions.

Definition 2.2.1 (Columns)**.** Let $\mathcal{J} = \mathcal{J}_1 \cup \cdots \cup \mathcal{J}_K$ (disjoint union). For $j \in \mathcal{J}_k$ we define

- the **block** of column j: $\psi(j) = k$;
- the **cost** of column j: $c_j = c^{(k)\top} \xi^{(j)}$;
- the **linking coefficients** of column j: $a_j^E = A_{E,k}\, \xi^{(j)} \in \mathbb{R}^{m_E}$, $a_j^I = A_{I,k}\, \xi^{(j)} \in \mathbb{R}^{m_I}$, and $a_j = [\, a_j^E;\, a_j^I \,] \in \mathbb{R}^m$.

We collect them in the cost vector $c = (c_j)_{j \in \mathcal{J}}$ and the matrices

$$A_E = \big[\, a_j^E \,\big]_{j \in \mathcal{J}} \in \mathbb{R}^{m_E \times |\mathcal{J}|}, \qquad A_I = \big[\, a_j^I \,\big]_{j \in \mathcal{J}} \in \mathbb{R}^{m_I \times |\mathcal{J}|}, \qquad A = \begin{bmatrix} A_E \\ A_I \end{bmatrix} \in \mathbb{R}^{m \times |\mathcal{J}|}.$$

Definition 2.2.2 (Convexity matrix)**.** The **convexity matrix** $C \in \{0,1\}^{K \times |\mathcal{J}|}$ has the entries $C_{kj} = 1$ if $\psi(j) = k$ and $C_{kj} = 0$ otherwise. The constraint $C\lambda = \mathbf{1}$ states that the weights of the columns of every block sum to one.

Definition 2.2.3 (Master problem)**.** The **master problem** is the linear program

$$\min_{\lambda} \quad c^\top \lambda \tag{2.3a}$$

$$\text{s.t.} \quad A_E\, \lambda = b_E, \qquad (\pi_E) \tag{2.3b}$$

$$A_I\, \lambda \geq b_I, \qquad (\pi_I) \tag{2.3c}$$

$$C\lambda = \mathbf{1}, \qquad (\mu) \tag{2.3d}$$

$$\lambda \geq 0, \tag{2.3e}$$

where the symbols in parentheses denote the dual variables of the constraints.

Theorem 2.2.4 (Dantzig–Wolfe [5])**.** Under Assumption 2.1.2, problem (2.3) has the optimal value $z^\star$. If λ is optimal for (2.3), then $\xi_k = \sum_{j \in \mathcal{J}_k} \lambda_j \, \xi^{(j)}$, $k = 1, \ldots, K$, is optimal for (2.1).

Proof. By (2.2), $\xi_k \in P_k$ if and only if ξ_k is a convex combination of the $\xi^{(j)}$, $j \in \mathcal{J}_k$. Under this substitution the objective and the linking rows are linear in λ, with exactly the coefficients of Definition 2.2.1. The feasible points of (2.1) are therefore the images of the feasible points of (2.3), with equal objective values. □

The convexity matrix has a very particular structure, which will be essential for the nullspace method of Chapter 5.

Definition 2.2.5 (Column generation property)**.** A matrix $C \in \mathbb{R}^{K \times n}$ has the **column generation property** if

(i) every entry of C is 0 or 1;

(ii) every column of C contains exactly one entry equal to 1;

(iii) every row of C contains at least one entry equal to 1.

Lemma 2.2.6. The convexity matrix C of Definition 2.2.2 has the column generation property, and so does every submatrix of C formed by a subset of its columns that contains at least one column of every block.

Proof. Property (i) holds by definition. Every column belongs to exactly one block, which gives property (ii) for every subset of columns. Property (iii) holds for a subset of columns if and only if every block is represented in it. □

The consequence used later is simple: if a set of columns contains one column j_k of every block k, the columns $j_1, \ldots, j_K$ of C are the unit vectors $e_1, \ldots, e_K$, i.e. they form an identity matrix. This identity block is the basis of the reordering theorem in Chapter 4.

The dual of the master problem

The dual of (2.3) has one constraint per column and the sign condition of the inequality rows:

$$\max_{\pi_E, \pi_I, \mu} \quad b_E^\top \pi_E + b_I^\top \pi_I + \mathbf{1}^\top \mu \tag{2.4a}$$

$$\text{s.t.} \quad c_j - a_j^\top \pi - \mu_{\psi(j)} \geq 0, \qquad j \in \mathcal{J}, \tag{2.4b}$$

$$\pi_I \geq 0, \tag{2.4c}$$

where $\pi = [\pi_E; \pi_I] \in \mathbb{R}^m$. Compared with a problem that has only equality linking rows, the inequality rows add the **sign condition** (2.4c). This is the only change in the dual, but much of the work in the following chapters is devoted to it.

2.3 Column Generation

The index set $\mathcal{J}$ is typically far too large to enumerate. In crew scheduling, for example, $\mathcal{J}_k$ contains every feasible schedule of pilot k. Column generation solves (2.3) by working with a small subset of the columns at a time.

Definition 2.3.1 (Restricted master problem)**.** For a subset $\mathcal{J}' \subseteq \mathcal{J}$ that contains at least one column of every block, the **restricted master problem** is problem (2.3) with all columns outside $\mathcal{J}'$ removed (equivalently, with $\lambda_j = 0$ fixed for $j \notin \mathcal{J}'$). We write $n = |\mathcal{J}'|$ for the number of columns in the restricted master problem.

For a small $\mathcal{J}'$ the restricted master problem may be infeasible: one column per block can in general not satisfy $A_E\lambda = b_E$. Classical column generation therefore adds artificial variables with a large cost to the linking rows (a *Big-M* start), which keeps every restricted master problem feasible; we assume them included where needed. An optimal solution in which an artificial variable is positive is not feasible for the master problem, so the optimality test below requires all artificial variables to be zero.

Definition 2.3.2 (Reduced cost)**.** For dual values (π, μ) the **reduced cost** of a column $j \in \mathcal{J}$ is

$$\bar{c}_j(\pi, \mu) \;=\; c_j - a_j^\top \pi - \mu_{\psi(j)}.$$

The reduced cost is the slack of the dual constraint (2.4b). A column with a negative reduced cost violates the dual constraints and can improve the restricted master problem. Finding the column of most negative reduced cost in block k is the **pricing problem**

$$\min_{j \in \mathcal{J}_k} \bar{c}_j(\pi, \mu) \;=\; \min_{\xi_k \in P_k} \left(c^{(k)} - A_k^\top \pi\right)^\top \xi_k \;-\; \mu_k, \qquad A_k = \begin{bmatrix} A_{E,k} \\ A_{I,k} \end{bmatrix}, \tag{2.5}$$

where the equality holds because a linear function attains its minimum over a polytope at an extreme point. The pricing problem is a problem over a single block. In crew scheduling it is a shortest path problem with edge weights given by the vector $c^{(k)} - A_k^\top \pi$.

Proposition 2.3.3 (Optimality test)**.** Let λ' be optimal for the restricted master problem, with optimal duals (π, μ), and with all artificial variables (if present) equal to zero. If $\pi_I \geq 0$ and the pricing problem (2.5) has a nonnegative optimal value for every block, then λ', extended by zeros, is optimal for the master problem (2.3).

Proof. The extended λ' is feasible for (2.3). We check the two conditions of the dual (2.4). The sign condition $\pi_I \geq 0$ holds by assumption. The column constraints $c_j - a_j^\top \pi - \mu_{\psi(j)} \geq 0$ hold for all $j \in \mathcal{J}$, because the pricing problem of block k minimizes exactly this expression over all $j \in \mathcal{J}_k$ and has a nonnegative value. So (π, μ) is dual feasible. Complementary slackness ($\lambda'_j > 0 \Rightarrow c_j - a_j^\top \pi - \mu_{\psi(j)} = 0$) holds for the columns in $\mathcal{J}'$ by optimality of the restricted problem, and trivially for the zero weights outside $\mathcal{J}'$. Complementary slackness of the inequality rows, $(\pi_I)_i > 0 \Rightarrow (A_I\lambda')_i = (b_I)_i$, also holds by optimality of the restricted problem, since the zero weights outside $\mathcal{J}'$ do not change $A_I\lambda'$. A primal–dual pair that is feasible and complementary is optimal. □

For a linear restricted master problem the duals satisfy $\pi_I \geq 0$ automatically, so the pricing test alone decides optimality. With the regularized master problems of the next chapter $\pi_I \geq 0$ still holds at an exact solution, but the pricing test no longer decides optimality: their solution satisfies the linking rows only when the duals have converged to the stability center (Proposition 3.3.5 and Theorem 3.4.1). Column generation also provides a lower bound on $z^\star$ at every iteration in which all blocks are priced.

Proposition 2.3.4 (Lagrangian bound)**.** For every $\pi = [\pi_E; \pi_I]$ with $\pi_I \geq 0$, the **Lagrangian bound**

$$\theta(\pi) \;=\; b^\top \pi + \sum_{k=1}^{K} \min_{\xi_k \in P_k} \left(c^{(k)} - A_k^\top \pi\right)^\top \xi_k, \qquad b = \begin{bmatrix} b_E \\ b_I \end{bmatrix},$$

satisfies $\theta(\pi) \leq z^\star$.

Proof. Let $\xi^\star = (\xi_1^\star, \dots, \xi_K^\star)$ be optimal for (2.1). Because $\sum_k A_{E,k}\xi_k^\star = b_E$, $\sum_k A_{I,k}\xi_k^\star \geq b_I$ and $\pi_I \geq 0$, we have $\pi^\top(b - \sum_k A_k\xi_k^\star) \leq 0$. Hence

$$z^\star = \sum_k c^{(k)\top}\xi_k^\star \;\geq\; \sum_k c^{(k)\top}\xi_k^\star + \pi^\top\Big(b - \sum_k A_k\xi_k^\star\Big) = b^\top\pi + \sum_k \big(c^{(k)} - A_k^\top\pi\big)^\top\xi_k^\star \;\geq\; \theta(\pi),$$

since $\xi_k^\star \in P_k$. □

Remark 2.3.5. The sign condition $\pi_I \geq 0$ in Proposition 2.3.4 is essential: for $\pi_i < 0$ in an over-satisfied inequality row, the term $\pi_i\,((b_I)_i - \sum_k (A_{I,k})_{i,:}\,\xi_k^\star)$ in the proof is positive, so its first inequality can fail and $\theta(\pi)$ may exceed $z^\star$. In Section 3.4.3 the bound is therefore evaluated at the projection of the dual vector onto $\{\pi_I \geq 0\}$.

The resulting method is summarized in Algorithm 2.1. The initial columns are usually obtained by solving the pricing problem once with zero duals, which gives one column per block.

Algorithm 2.1: Classical Column Generation

Input: initial columns $\mathcal{J}'$ with at least one column per block
repeat
  Solve the (linear) restricted master problem over $\mathcal{J}'$; obtain λ' and duals (π, μ)
  for each block $k = 1, \dots, K$ **do**
    Solve the pricing problem (2.5); let j be the best column
    if $\bar{c}_j(\pi, \mu) < 0$ **then**
      $\mathcal{J}' \leftarrow \mathcal{J}' \cup \{j\}$
    end if
  end for
until no column with negative reduced cost was found
Output: λ', optimal for the master problem by Proposition 2.3.3

Remark 2.3.6 (Instability)**.** A simplex solver returns basic dual solutions of the linear restricted master problem, i.e. vertices of its dual feasible region. When a column is added, the dual solution can jump to a very different vertex, even if the primal solution hardly changes. As a result, the duals oscillate strongly from one iteration to the next, the pricing problem generates many columns that are never used, and the method converges slowly in its final phase (the *tailing-off effect*). In the dual space, column generation is Kelley's cutting-plane method [18] applied to the Lagrangian dual function, a method known for exactly this behaviour. This is the motivation for the regularization of Chapter 3.

2.4 Surplus Variables and the Unified Formulation

The two kinds of linking rows in (2.3) behave differently: the equality rows fix $A_E\lambda$, while the inequality rows only bound $A_I\lambda$ from below. To treat both in the same way, we introduce for every inequality row i a **surplus variable** $s_i \geq 0$ with

$$(A_I\lambda)_i - s_i = (b_I)_i, \qquad \text{i.e.} \qquad A_I\lambda - s = b_I.$$

The surplus measures how much the row is over-satisfied. For an inequality row i we write $\pi_i := (\pi_I)_i$ for its dual.

Definition 2.4.1 (Unified formulation)**.** Let $s \in \mathbb{R}^{m_I}$ be the vector of surplus variables. The **unified variable** and the **augmented data** are

$$y = \begin{bmatrix}\lambda \\ s\end{bmatrix}, \qquad \bar{A} = \begin{bmatrix}A_E & 0 \\ A_I & -I\end{bmatrix}, \qquad \bar{C} = \begin{bmatrix}C & 0\end{bmatrix}, \qquad \hat{c} = \begin{bmatrix}c \\ 0\end{bmatrix}, \qquad b = \begin{bmatrix}b_E \\ b_I\end{bmatrix}.$$

The **unified master problem** is

$$\min_{y} \ \hat{c}^\top y \qquad \text{s.t.} \qquad \bar{A}y = b, \quad \bar{C}y = \mathbf{1}, \quad y \geq 0. \tag{2.6}$$

Lemma 2.4.2. Problems (2.3) and (2.6) are equivalent: λ is feasible for (2.3) if and only if $y = [\lambda;\ A_I\lambda - b_I]$ is feasible for (2.6), with the same objective value. Moreover, the dual of (2.6) is the dual (2.4) of the master problem.

Proof. The first statement holds because $A_I\lambda \geq b_I$ is equivalent to $s = A_I\lambda - b_I \geq 0$, and the surpluses have zero cost. For the second, the dual of (2.6) has multipliers π for $\bar{A}y = b$ and μ for $\bar{C}y = \mathbf{1}$, and one constraint "cost $-$ (column of $\bar{A}$)$^\top\pi$ $-$ (column of $\bar{C}$)$^\top\mu \geq 0$" per variable, with no sign condition on π, since all rows are equalities.

- For λ_j the column of $\bar{A}$ is a_j and the column of $\bar{C}$ is $e_{\psi(j)}$, so the constraint is $c_j - a_j^\top\pi - \mu_{\psi(j)} \geq 0$, as in (2.4b).
- For s_i the cost is 0, the column of $\bar{A}$ is $-e_i$ in the inequality part, and the column of $\bar{C}$ is 0, so the constraint is $0 + (\pi_I)_i - 0 \geq 0$, i.e. $(\pi_I)_i \geq 0$, as in (2.4c). □

Lemma 2.4.2 shows that the surplus columns *encode the sign conditions as ordinary dual constraints*. Problem (2.6) has the same form as a master problem with equality linking rows only, with (A, C, c) replaced by $(\bar{A}, \bar{C}, \hat{c})$. All the following chapters work with this form.

Remark 2.4.3 (The column generation property is lost)**.** The augmented convexity matrix $\bar{C} = [\,C \quad 0\,]$ has m_I zero columns, one per surplus variable, so it violates property (ii) of Definition 2.2.5. The surpluses do not belong to any block. Chapter 4 shows that this causes no difficulty: zero columns are even simpler to handle than unit columns, but they have to be treated as a separate group of variables.

Remark 2.4.4 ("$\leq$" rows and the slack sign)**.** In this remark only, A_I and b_I collect the inequality rows of both directions. A row $(A_I)_{i,:}\lambda \leq (b_I)_i$ can be written with a *slack* $s_i \geq 0$ as $(A_I)_{i,:}\lambda + s_i = (b_I)_i$. In general, let $\kappa_i \in \{-1, +1\}$ be the coefficient of the slack of inequality row i ($\kappa_i = -1$ for a "$\geq$" row, $\kappa_i = +1$ for a "$\leq$" row) and $D_\kappa = \mathrm{diag}(\kappa)$. The unified formulation then uses

$$\bar{A} = \begin{bmatrix} A_E & 0 \\ A_I & D_\kappa \end{bmatrix}$$

instead of Definition 2.4.1, and the dual constraint of slack i becomes $-\kappa_i(\pi_I)_i \geq 0$, i.e. $(\pi_I)_i \leq 0$ for a "$\leq$" row. Since $\kappa_i^2 = 1$, every result of the following chapters holds for both kinds of rows; only a few formulas carry a factor κ_i, and these are collected in Remark 5.3.3. In the text we write all formulas for "$\geq$" rows ($D_\kappa = -I$).

Remark 2.4.5 (Range rows)**.** A row with two different finite bounds $\underline{b}_i \leq A_{i,:}\lambda \leq \bar{b}_i$ gets a single slack with an upper bound,

$$A_{i,:}\lambda + \kappa_i s_i = b_i, \qquad 0 \leq s_i \leq U_i := \bar{b}_i - \underline{b}_i,$$

starting as a "$\geq$" row ($\kappa_i = -1$, $b_i = \underline{b}_i$). The substitution $\bar{s}_i = U_i - s_i$ rewrites the same row as

$$A_{i,:}\lambda - \kappa_i\bar{s}_i = b_i - \kappa_i U_i, \qquad 0 \leq \bar{s}_i \leq U_i,$$

i.e. as the "$\leq$" row $A_{i,:}\lambda + \bar{s}_i = \bar{b}_i$, and back. Both forms describe the same feasible set and give the same value of $\bar{A}y - b$, hence the same dual estimate and objective. A range row is therefore handled like a one-sided row of Remark 2.4.4 whose form is switched whenever its slack reaches U_i: the slack, now $\bar{s}_i = 0$, enters the working set in the new form, κ_i and b_i change, and nothing

else does. Its multiplier $-\kappa_i(\pi_I)_i$ then automatically has the sign condition of the active bound, $(\pi_I)_i \geq 0$ at $\underline{b}_i$ and $(\pi_I)_i \leq 0$ at $\bar{b}_i$. The only addition to the active-set method is that the ratio test also stops a free slack at U_i. All results of the following chapters hold unchanged, since they depend on κ_i only through $\kappa_i^2 = 1$ and the formulas of Remark 5.3.3, which are evaluated with the current κ_i.

2.5 Summary of Notation

The symbols introduced in this chapter are used throughout the remaining chapters.

symbol	meaning
K, k	number of blocks, block index
m_E, m_I, m	number of equality, inequality and all linking rows
$\mathcal{J}$, $\mathcal{J}_k$, $\mathcal{J}'$	all columns, columns of block k, columns of the restricted problem
n	number of columns in the restricted master problem
$\psi(j)$	block of column j
ξ_k, $c^{(k)}$, P_k	variables, cost and polytope of block k in (2.1)
c_j, $a_j = [a_j^E; a_j^I]$	cost and linking coefficients of column j
A_E, A_I, A	linking coefficient matrices of the columns
C	convexity matrix
λ, s, $y = [\lambda; s]$	column weights, surpluses, unified variable
$\bar{A}$, $\bar{C}$, $\hat{c}$	augmented data of the unified formulation
$\pi = [\pi_E; \pi_I]$, μ	duals of the linking rows and of the convexity rows
$\bar{c}_j(\pi, \mu)$	reduced cost of column j
$\theta(\pi)$, $z^\star$	Lagrangian bound, optimal value of (2.1)

Chapter 3

Regularization of the Master Problem

This chapter replaces the linear restricted master problem by a regularized one. The regularization is formulated in the dual space, where it adds a quadratic penalty on the distance between the dual iterate and a **stability center**. Translated back to the primal space, it yields a convex quadratic program with an auxiliary variable, the **explicit regularized master problem**.

Because all linking rows of the unified formulation are equalities, the auxiliary variable can be eliminated for equality and inequality rows alike. The result, the **eliminated regularized master problem**, is a convex quadratic program whose only constraints are the convexity constraints and the nonnegativity of the variables. This is the problem that the active-set method of the following chapters solves.

Two identities derived in this chapter are used throughout the rest of the work. The first expresses the duals of the linking rows as an explicit function of the primal iterate. The second shows that the gradient of the quadratic objective consists of the reduced costs of the columns and the duals of the inequality rows. The chapter then states the regularized column generation algorithm, its exact stopping criterion and the accuracy that this criterion guarantees. It ends with a rule that adapts the stabilization parameter during the run, based on the interpretation of δ as the step size of a proximal point method.

3.1 Proximal Stabilization of the Dual

Consider the restricted master problem in the unified form (2.6), over the columns $\mathcal{J}'$ and all m_I surpluses. By Lemma 2.4.2, its dual is

$$\max_{\pi,\mu} \; b^\top \pi + \mathbf{1}^\top \mu \qquad \text{s.t.} \qquad c_j - a_j^\top \pi - \mu_{\psi(j)} \geq 0 \;\; (j \in \mathcal{J}'), \qquad \pi_I \geq 0. \tag{3.1}$$

The dual solutions returned by a simplex solver are vertices of this feasible region, which causes the instability described in Remark 2.3.6. Proximal stabilization adds a penalty on the distance to a given point $\hat{\pi} \in \mathbb{R}^m$.

Definition 3.1.1 (Stabilized dual problem)**.** Let $\hat{\pi} \in \mathbb{R}^m$ be the **stability center** and $\delta > 0$ the **stabilization parameter**. The **stabilized dual problem** is

$$\max_{\pi,\mu} \; b^\top \pi + \mathbf{1}^\top \mu - \frac{\delta}{2}\|\pi - \hat{\pi}\|^2 \qquad \text{s.t.} \qquad c_j - a_j^\top \pi - \mu_{\psi(j)} \geq 0 \;\; (j \in \mathcal{J}'), \qquad \pi_I \geq 0. \tag{3.2}$$

The objective of (3.2) is strictly concave in π, so the optimal π is unique. A large δ keeps π close to $\hat{\pi}$ (strong stabilization), and $\delta \to 0$ recovers (3.1). Replacing the center $\hat{\pi}$ by the solution π and solving again is one step of the **proximal point method** [24] applied to the restricted dual problem. If the center is only moved when it improves the Lagrangian bound, we obtain a **proximal bundle method** [19, 3]. Both rules are discussed in Section 3.4.

3.2 The Explicit Regularized Master Problem

Column generation needs a primal master problem: its solution provides the weights λ, and its duals are passed to the pricing problem. The primal counterpart of the stabilized dual (3.2) has an auxiliary vector $w \in \mathbb{R}^m$, one entry per linking row.

Definition 3.2.1 (Explicit regularized master problem)**.** The **explicit regularized master problem** is

$$\min_{y,w} \quad \hat{c}^\top y + \frac{\delta}{2}\|w + \hat{\pi}\|^2 - \frac{\delta}{2}\|\hat{\pi}\|^2 \tag{3.3a}$$

$$\text{s.t.} \quad \bar{A}y + \delta w = b, \qquad (\pi) \tag{3.3b}$$

$$\bar{C}y = \mathbf{1}, \qquad (\mu) \tag{3.3c}$$

$$y \geq 0. \tag{3.3d}$$

Theorem 3.2.2. The dual of (3.3) is the stabilized dual problem (3.2). Strong duality holds, and at a pair of optimal solutions

$$\pi = \hat{\pi} + w. \tag{3.4}$$

Proof. With multipliers π for $\bar{A}y + \delta w = b$, μ for $\bar{C}y = \mathbf{1}$ and $\nu \geq 0$ for $y \geq 0$, the Lagrangian of (3.3) is

$$\mathcal{L}(y, w; \pi, \mu, \nu) = \hat{c}^\top y + \tfrac{\delta}{2}\|w + \hat{\pi}\|^2 - \tfrac{\delta}{2}\|\hat{\pi}\|^2 + \pi^\top(b - \bar{A}y - \delta w) + \mu^\top(\mathbf{1} - \bar{C}y) - \nu^\top y.$$

The dual function is the infimum of $\mathcal{L}$ over y and w, which we compute separately.

Infimum over w. $\mathcal{L}$ is a strictly convex quadratic function of w with gradient $\delta(w + \hat{\pi}) - \delta\pi$, so the minimizer is $w = \pi - \hat{\pi}$. Inserting it gives the w-dependent terms

$$\tfrac{\delta}{2}\|\pi\|^2 - \tfrac{\delta}{2}\|\hat{\pi}\|^2 - \delta\pi^\top(\pi - \hat{\pi}) = -\tfrac{\delta}{2}\|\pi - \hat{\pi}\|^2.$$

Infimum over y. $\mathcal{L}$ is linear in y with coefficient vector $\hat{c} - \bar{A}^\top \pi - \bar{C}^\top \mu - \nu$. The infimum is finite (and then the term vanishes) only if this vector is zero, i.e. only if $\hat{c} - \bar{A}^\top \pi - \bar{C}^\top \mu = \nu \geq 0$. Together, the dual problem is to maximize $b^\top \pi + \mathbf{1}^\top \mu - \frac{\delta}{2}\|\pi - \hat{\pi}\|^2$ subject to $\hat{c} - \bar{A}^\top \pi - \bar{C}^\top \mu \geq 0$. As in the proof of Lemma 2.4.2, the entry of this vector for λ_j is $c_j - a_j^\top \pi - \mu_{\psi(j)}$, and the entry for s_i is $(\pi_I)_i$. This is exactly (3.2).

Problem (3.3) is a convex quadratic program with linear constraints. It is feasible, since $\mathcal{J}'$ contains a column of every block and w is free, and its objective is bounded below, since $\hat{c}^\top y = c^\top \lambda$ with $0 \leq \lambda \leq 1$ and $\frac{\delta}{2}\|w + \hat{\pi}\|^2 \geq 0$. A convex quadratic program that is feasible and bounded below attains its minimum (Frank–Wolfe theorem), and with linear constraints the KKT conditions hold there. This gives strong duality. At an optimal pair, (y, w) minimizes $\mathcal{L}$ for the optimal multipliers, and the minimizer in w is unique, so $w = \pi - \hat{\pi}$, which is (3.4). □

Remark 3.2.3. Theorem 3.2.2 shows that the sign condition $\pi_I \geq 0$ becomes part of the stabilized dual problem automatically, through the surplus columns. The stability center $\hat{\pi}$ itself may violate it; the proximal step then moves π back into the region $\pi_I \geq 0$.

Remark 3.2.4. Without surplus variables, an inequality row $(A_I \lambda)_i \geq (b_I)_i$ would enter the explicit problem as $(A_I \lambda)_i + \delta w_i \geq (b_I)_i$, and w_i would no longer be determined by λ. The surpluses make every row of (3.3b) an equality, which is what allows w to be eliminated for all rows in the next section.

3.3 The Eliminated Regularized Master Problem

The explicit problem (3.3) has m more variables than necessary: every feasible point satisfies $\bar{A}y + \delta w = b$, so $w = (b - \bar{A}y)/\delta$ is determined by y. Substituting this into the objective removes w completely.

Theorem 3.3.1 (Eliminated regularized master problem)**.** Problem (3.3) is equivalent to the convex quadratic program

$$\min_{y} \quad f(y) = \frac{1}{2\delta}\, y^\top \bar{A}^\top \bar{A}\, y + \tilde{c}^\top y \tag{3.5a}$$

$$\text{s.t.} \quad \bar{C}y = \mathbf{1}, \tag{3.5b}$$

$$y \geq 0, \tag{3.5c}$$

with the linear cost

$$\tilde{c} = \hat{c} - \bar{A}^\top \Big(\hat{\pi} + \frac{b}{\delta}\Big). \tag{3.6}$$

More precisely, (y, w) is optimal for (3.3) if and only if y is optimal for (3.5) and $w = (b - \bar{A}y)/\delta$. The objective values are related by

$$\Phi(y) := \hat{c}^\top y + \frac{\delta}{2}\|w + \hat{\pi}\|^2 - \frac{\delta}{2}\|\hat{\pi}\|^2 = f(y) + \frac{1}{2\delta}\|b\|^2 + \hat{\pi}^\top b, \tag{3.7}$$

where $w = (b - \bar{A}y)/\delta$; this defines the master objective Φ for every y.

Proof. Expanding the squares,

$$\tfrac{\delta}{2}\|w + \hat{\pi}\|^2 - \tfrac{\delta}{2}\|\hat{\pi}\|^2 = \tfrac{\delta}{2}\|w\|^2 + \delta\, \hat{\pi}^\top w.$$

Inserting $\delta w = b - \bar{A}y$ in both terms,

$$\tfrac{\delta}{2}\|w\|^2 = \tfrac{1}{2\delta}\|\bar{A}y\|^2 - \tfrac{1}{\delta} b^\top \bar{A}y + \tfrac{1}{2\delta}\|b\|^2, \qquad \delta\, \hat{\pi}^\top w = \hat{\pi}^\top b - \hat{\pi}^\top \bar{A}y.$$

Adding $\hat{c}^\top y$ and collecting the terms that are linear in y gives $\hat{c}^\top y - (\hat{\pi} + b/\delta)^\top \bar{A} y = \tilde{c}^\top y$, so the objective is $f(y) + \frac{1}{2\delta}\|b\|^2 + \hat{\pi}^\top b$. The constraint (3.3b) only defines w, so the remaining constraints are $\bar{C} y = \mathbf{1}$ and $y \geq 0$. Since the objectives differ by a constant, the two problems have the same minimizers. □

The Hessian of f is $\frac{1}{\delta}\bar{A}^\top \bar{A}$, with the block structure

$$\frac{1}{\delta}\bar{A}^\top \bar{A} = \frac{1}{\delta}\begin{bmatrix} A_E^\top A_E + A_I^\top A_I & -A_I^\top \\ -A_I & I \end{bmatrix}. \tag{3.8}$$

It is positive semidefinite but in general singular: it has rank at most m, while the number of variables $n + m_I$ grows with every generated column. This singularity is the main difficulty for an active-set method, and it is addressed in Chapters 5 and 7. Problem (3.5) nevertheless has an optimal solution, because λ ranges over a bounded set and the objective grows quadratically in s.

3.3.1 The dual vector as a function of the primal iterate

Relation (3.4) with $w = (b - \bar{A}y)/\delta$ expresses the optimal dual through the optimal y. The same expression is useful at *every* y, which motivates the following definition.

Definition 3.3.2 (Dual estimate)**.** For $y = [\lambda; s]$ the **dual estimate** is

$$\pi(y) = \hat{\pi} + \frac{1}{\delta}(b - \bar{A}y), \qquad \text{i.e.} \qquad \begin{aligned} \pi_E(y) &= \hat{\pi}_E + \tfrac{1}{\delta}(b_E - A_E\lambda), \\ \pi_I(y) &= \hat{\pi}_I + \tfrac{1}{\delta}(b_I - A_I\lambda + s). \end{aligned} \tag{3.9}$$

A surplus raises the dual of its row: an over-covered inequality row with a positive surplus has a larger π_i than it would have without the surplus. The next lemma is the central identity of this work.

Lemma 3.3.3 (Gradient–dual relation)**.** For every y,

$$\nabla f(y) = \hat{c} - \bar{A}^\top \pi(y), \qquad \text{i.e.} \qquad \frac{\partial f}{\partial \lambda_j} = c_j - a_j^\top \pi(y), \qquad \frac{\partial f}{\partial s_i} = \pi_i(y) \quad (i = 1, \ldots, m_I), \tag{3.10}$$

where π_i denotes the dual of inequality row i.

Proof. Differentiating $f(y) = \frac{1}{2\delta} y^\top \bar{A}^\top \bar{A} y + \tilde{c}^\top y$ and inserting (3.6) for $\tilde{c}$,

$$\nabla f(y) = \tfrac{1}{\delta}\bar{A}^\top \bar{A} y + \hat{c} - \bar{A}^\top \hat{\pi} - \tfrac{1}{\delta}\bar{A}^\top b = \hat{c} - \bar{A}^\top \left(\hat{\pi} + \tfrac{1}{\delta}(b - \bar{A}y)\right) = \hat{c} - \bar{A}^\top \pi(y).$$

The row of $\bar{A}^\top$ belonging to λ_j is $a_j^\top$, and $\hat{c}_j = c_j$. The row belonging to s_i is -1 in the position of inequality row i and zero elsewhere, and its cost is 0, which gives $0 - (-\pi_i) = \pi_i$. □

Lemma 3.3.3 has two consequences that are used throughout this work.

- The gradient component of a column, $c_j - a_j^\top \pi$, is the **reduced cost of column j without the convexity dual**. The gradient component of a surplus is the dual of its row. Chapter 4 shows that this makes the optimality test of the master problem identical to the pricing test of column generation.
- The gradient, an $(n + m_I)$-vector whose length grows with every column, is determined by the m-vector $\pi(y)$. An implementation can store π and evaluate the gradient only where it is needed.

The same idea gives the objective value. In the experiments of Chapter 9 the value $\Phi(y)$ of (3.7) is reported as the *master objective*; the next proposition shows that it can be computed from π without forming $\bar{A}y$ again.

Proposition 3.3.4 (Objective value)**.** For every $y = [\lambda; s]$, with $\pi = \pi(y)$,

$$\Phi(y) = c^\top \lambda + \frac{\delta}{2}\|\pi\|^2 - \frac{\delta}{2}\|\hat{\pi}\|^2.$$

Proof. By definition (3.7), $\Phi(y) = \hat{c}^\top y + \frac{\delta}{2}\|w + \hat{\pi}\|^2 - \frac{\delta}{2}\|\hat{\pi}\|^2$ with $w = (b - \bar{A}y)/\delta$. Hence $w + \hat{\pi} = \pi(y)$ by (3.9), and $\hat{c}^\top y = c^\top \lambda$ because the surpluses have zero cost. □

Finally, the dual estimate measures the infeasibility of y in the unregularized master problem. This is the basis of the stopping criterion in Section 3.4.

Proposition 3.3.5 (Residual and dual distance)**.** For every y,

$$b - \bar{A}y = \delta(\pi(y) - \hat{\pi}).$$

In particular, y satisfies the linking rows $\bar{A}y = b$ of the unified master problem (2.6) if and only if $\pi(y) = \hat{\pi}$.

Proof. Multiply the definition $\pi(y) = \hat{\pi} + \frac{1}{\delta}(b - \bar{A}y)$ by δ and rearrange. □

3.4 Regularized Column Generation

Column generation with the regularized master problem follows the classical scheme of Algorithm 2.1, with two differences: the duals passed to the pricing problem are the stabilized duals $\pi(y)$, and the stability center has to be updated.

3.4.1 Center updates

Two rules for the center update are common.

- **Proximal point rule:** after every master problem, $\hat{\pi} \leftarrow \pi$. Every iteration is a step of the proximal point method on the current restricted dual problem.
- **Bundle rule:** $\hat{\pi} \leftarrow \pi$ only if the Lagrangian bound improves, $\theta(\pi^+) > \theta(\hat{\pi}^+)$ with the projection π^+ of Section 3.4.3 (a *serious step*). Otherwise the center stays (a *null step*). This requires the Lagrangian bound at every iteration, i.e. all blocks must be priced every iteration. In an iteration without a new column, a null step would repeat the same master problem, so there the center is always moved (the center-only iteration of Algorithm 3.1).

3.4.2 Exact termination

In classical column generation, the pricing test alone decides optimality (Proposition 2.3.3). With regularization this is no longer true: when pricing finds no improving column, the regularized master solution may still be infeasible for the unregularized problem, since by Proposition 3.3.5 $\bar{A}y \neq b$ as long as $\pi \neq \hat{\pi}$.

Theorem 3.4.1 (Exact termination)**.** Let $y = [\lambda; s]$ be optimal for the eliminated regularized master problem (3.5) over the columns $\mathcal{J}'$, with dual estimate $\pi = \pi(y)$, and let μ be the Lagrange multipliers of the convexity constraints (3.5b). If

(i) $\pi = \hat{\pi}$, and

(ii) the pricing problem (2.5) has a nonnegative optimal value for every block,

then λ, extended by zeros, is optimal for the master problem (2.3), and (π, μ) is optimal for its dual (2.4).

Proof. Primal feasibility. By Proposition 3.3.5, $\pi = \hat{\pi}$ gives $\bar{A}y = b$, i.e. $A_E\lambda = b_E$ and $A_I\lambda - s = b_I$. Since $s \geq 0$, $A_I\lambda \geq b_I$, so λ is feasible for (2.3).
Dual feasibility. The optimality conditions of (3.5), derived in Theorem 4.2.1 of Chapter 4 (whose proof does not use this theorem), state that

- $c_j - a_j^\top \pi - \mu_{\psi(j)} \geq 0$ for every $j \in \mathcal{J}'$, with equality if $\lambda_j > 0$;
- $\pi_i \geq 0$ for every inequality row i, with equality if $s_i > 0$.

Condition (ii) extends the first inequality to all $j \in \mathcal{J}$, because the pricing problem of each block minimizes $c_j - a_j^\top \pi - \mu_{\psi(j)}$ over its columns. Hence (π, μ) satisfies both constraints of the dual (2.4).
Complementary slackness. Positive weights have zero reduced cost by the first item. For an inequality row, $\pi_i > 0$ forces $s_i = 0$ by the second item, i.e. the row is satisfied with equality. Feasible and complementary primal and dual solutions are optimal. □

If pricing finds no improving column while $\pi \neq \hat{\pi}$, the restricted master problem contains the right columns for the moment, but the proximal term still keeps π away from the restricted dual optimum. The algorithm then moves the center to π and solves the master problem again *without adding a column*. We call this a **center-only iteration**. It is one proximal point step on the current restricted model. As the duals move, pricing may find improving columns again. A run of regularized column generation therefore consists of two kinds of iterations: iterations that add columns, and center-only iterations that drive π towards $\hat{\pi}$. Proposition 6.6.2 analyses the center-only iterations, and Section 9.4 measures both kinds.

Proposition 3.4.2 (Accuracy of the stopping test)**.** In practice condition (i) of Theorem 3.4.1 is tested as

$$\|\pi - \hat{\pi}\|_\infty \;\leq\; \varepsilon_c \max(1, \|\pi\|_\infty) \tag{3.11}$$

with a tolerance $\varepsilon_c > 0$. When this test holds, the residual of the unified linking rows, and with it the violation of the linking rows by λ, is bounded by

$$\max\Big(\|b_E - A_E\lambda\|_\infty,\; \|\max(b_I - A_I\lambda, 0)\|_\infty\Big) \;\leq\; \|b - \bar{A}y\|_\infty \;\leq\; \delta\,\varepsilon_c \max(1, \|\pi\|_\infty).$$

Proof. By Proposition 3.3.5, $\|b - \bar{A}y\|_\infty = \delta\|\pi - \hat{\pi}\|_\infty$; insert (3.11). For the first inequality, $b - \bar{A}y = [\,b_E - A_E\lambda;\; b_I - A_I\lambda + s\,]$, and since $s \geq 0$, $\max(b_I - A_I\lambda, 0) \leq |b_I - A_I\lambda + s|$ componentwise. □

The accuracy of the final solution is thus relative to the size of the duals, and it scales with δ. Section 9.3 and Fig. 9.3 show that the Lagrangian bound and the master objective Φ are much less affected by this residual than the cost $c^\top\lambda$ of a slightly infeasible λ.

3.4.3 The Lagrangian bound with inequality rows

The Lagrangian bound of Proposition 2.3.4 requires $\pi_I \geq 0$. The duals of the regularized master problem satisfy this condition only up to the tolerance of the master solver, and a tiny negative entry would void the guarantee $\theta(\pi) \leq z^\star$ (Remark 2.3.5). The bound is therefore evaluated at the projection

$$\pi^+ = \big[\, \pi_E;\ \max(\pi_I, 0) \,\big],$$

which always satisfies the sign condition, so that $\theta(\pi^+) \leq z^\star$ holds by Proposition 2.3.4. The same projected vector is passed to the pricing problem. For problems with equality rows only, $\pi^+ = \pi$. At an exact optimum of the master problem $\pi_I \geq 0$ (Theorem 4.2.1 (ii)), so $\pi^+ = \pi$ and the pricing test of Algorithm 3.1 is condition (ii) of Theorem 3.4.1.

3.4.4 The algorithm

Algorithm 3.1 collects the steps of this section. The first line of the loop, the solution of the regularized master problem, is the subject of the remaining chapters.

Algorithm 3.1: Regularized Column Generation

Input: initial columns $\mathcal{J}'$ (one per block), center $\hat{\pi}$, parameter $\delta > 0$, tolerances ε_{c}, $\varepsilon_{\mathrm{rc}}$
repeat
  Solve the eliminated regularized master problem (3.5); obtain y, $\pi = \pi(y)$ and μ
  $\pi^+ \leftarrow [\pi_E; \max(\pi_I, 0)]$
  Price the blocks in random order with (π^+, μ); add the first column with $\bar{c}_j < -\varepsilon_{\mathrm{rc}}$ to $\mathcal{J}'$
  if no column was added (all blocks priced) **then**
    $\theta \leftarrow \theta(\pi^+)$ ▷ valid lower bound on $z^\star$
    if $\|\pi - \hat{\pi}\|_\infty \leq \varepsilon_{\mathrm{c}} \max(1, \|\pi\|_\infty)$ **then**
      stop: λ is optimal up to the accuracy of Proposition 3.4.2
    end if
    $\hat{\pi} \leftarrow \pi$ ▷ center-only iteration
  else
    $\hat{\pi} \leftarrow \pi$ ▷ proximal point rule
  end if
until stopped

The bundle rule would need the Lagrangian bound in every iteration and hence full pricing; the experiments of Chapter 9 use the proximal point rule.

3.5 An Adaptive Stabilization Parameter

Algorithm 3.1 keeps δ fixed. This section shows that δ is the step size of a proximal point method on the restricted dual function, recalls what the theory of that method says about variable step sizes, and derives a simple rule that changes δ during the run. Its effect is measured in Section 9.9.

3.5.1 The stabilization parameter as a step size

For a set $\mathcal{J}'$ of columns, write $\mathcal{J}'_k = \mathcal{J}' \cap \mathcal{J}_k$ and let

$$D(\pi) = b^\top \pi + \sum_{k=1}^{K} \min_{j \in \mathcal{J}'_k} (c_j - a_j^\top \pi) \qquad \text{for } \pi_I \geq 0,$$

and $D(\pi) = -\infty$ otherwise. This is the objective of the restricted dual (3.1) with μ maximized out, since the best μ_k is the smallest $c_j - a_j^\top \pi$ of block k. D is concave and piecewise linear, and, since $\mathcal{J}' \subseteq \mathcal{J}$, it overestimates the Lagrangian bound, $D(\pi) \geq \theta(\pi)$ (Proposition 2.3.4): D is the cutting-plane model of θ given by the current columns. A vector g is a **supergradient** of D at π if $D(\pi') \leq D(\pi) + g^\top(\pi' - \pi)$ for all π'; the set of supergradients is $\partial D(\pi)$.

Proposition 3.5.1 (Proximal step)**.** Let y be optimal for the eliminated regularized master problem (3.5) with center $\hat{\pi}$ and parameter δ, and let $\pi = \pi(y)$. Then

$$\pi = \arg\max_{\pi'} \; D(\pi') - \frac{\delta}{2}\|\pi' - \hat{\pi}\|^2 \qquad \text{and} \qquad \pi = \hat{\pi} + \frac{1}{\delta}\, r, \quad r = b - \bar{A}y \in \partial D(\pi).$$

Proof. By Theorems 3.2.2 and 3.3.1, π is the π-part of the optimal solution of the stabilized dual (3.2). Maximizing over μ first gives the stated problem, whose objective is strictly concave, so π is its unique maximizer. Its optimality condition is $0 \in \partial D(\pi) - \delta(\pi - \hat{\pi})$, i.e. $\delta(\pi - \hat{\pi}) \in \partial D(\pi)$, and $\delta(\pi - \hat{\pi}) = b - \bar{A}y$ by Proposition 3.3.5. □

A master solve is therefore an implicit supergradient step of length $1/\delta$ on the restricted dual, and a sequence of center-only iterations is the proximal point method [24] on the fixed function D, with step parameters $c_t = 1/\delta_t$. Its classical theory makes the role of the step size precise.

Remark 3.5.2 (Proximal point method with variable steps)**.** Let $\Pi^\star$ be the set of maximizers of D, assumed nonempty.

(i) The iterates converge to a point of $\Pi^\star$ for every sequence with $\sum_t c_t = \infty$, in particular for every sequence with $\delta_t \leq \delta_0$ [24, 16].

(ii) The values satisfy $D^\star - D(\pi_t) \leq \operatorname{dist}(\pi_0, \Pi^\star)^2/(2\sum_{s<t} c_s)$ [16]. With a fixed δ the gap decreases like δ/t; with $\delta_t = \delta_0\gamma^t$, $\gamma < 1$, it decreases geometrically.

(iii) Since D is polyhedral, the method reaches $\Pi^\star$ after finitely many steps for every sequence c_t bounded away from zero [24, 10].

(iv) For a linear program there is a $\bar{\delta}(\hat{\pi}) > 0$ such that for every $\delta \leq \bar{\delta}(\hat{\pi})$ the proximal point is the projection of $\hat{\pi}$ onto $\Pi^\star$ [21, 12]: a small enough δ reaches $\Pi^\star$ in one step.

On a fixed restricted model, a small δ is therefore always better. With a fixed δ, the center-only iterations walk towards $\Pi^\star$ in steps of length $\|r\|/\delta$, and once the working set has settled the step does not change from one iteration to the next (Proposition 6.6.2); the number of such iterations grows with δ. This is the long proximal phase observed in Section 9.4. A small δ throughout is no solution either: when new columns enter, the model changes, and a small δ lets the duals jump like the duals of the linear master problem, which is the instability that the regularization is meant to remove (Remark 2.3.6). The parameter should be small while the model is fixed and large when it changes.

3.5.2 The geometric rule

Definition 3.5.3 (Geometric stabilization parameter)**.** Let $\delta_0 > 0$ be the base value and $0 < \gamma < 1 \leq \beta$, $0 < \delta_{\min} \leq \delta_0$. After every iteration of Algorithm 3.1 in which the center moves, the parameter of the next master problem is

$$\delta \;\leftarrow\; \begin{cases} \max(\delta_{\min},\ \gamma\,\delta) & \text{after a center-only iteration,} \\ \min(\delta_0,\ \beta\,\delta) & \text{after an iteration that adds a column.} \end{cases} \tag{3.12}$$

During a run of center-only iterations the parameter follows the geometric sequence $\delta_0\gamma^t$, which gives the rule its name; we use $\gamma = 1/2$ and $\beta = 2$. The choice $\beta = \infty$ resets δ to δ_0 whenever a column is added. In the column phase, where almost every iteration adds a column, δ stays at δ_0, so the new columns enter under full stabilization as in Algorithm 3.1. The rule needs no information beyond whether pricing added a column, so it works with first-improving pricing and does not require the Lagrangian bound.

Proposition 3.5.4 (Properties of the geometric rule)**.** Let δ be changed by (3.12), or by any other rule, between two master problems.

(i) The change leaves the iterate feasible, and for the active-set method of the following chapters it leaves the ordering, the nullspace basis and the factorization unchanged. Only the dual estimate changes, to $\hat{\pi} + (b - \bar{A}y)/\delta$.

(ii) Theorem 3.4.1 holds unchanged, and Proposition 3.4.2 holds with the value of δ in the last master problem.

Proof. (i) The constraints (3.5b) and (3.5c) do not contain δ. For the active-set method, H_{eff} does not depend on δ (Remark 5.2.8); δ enters only the right-hand side of the reduced system and the dual estimate (3.9), exactly like a new center (Theorem 6.6.1 (ii)). (ii) Neither the statement nor the proof of Theorem 3.4.1 uses the value of δ, and the proof of Proposition 3.4.2 uses only Proposition 3.3.5 for the last master problem. □

Part (i) is what makes the rule attractive for this method: a general quadratic programming solver sees a new Hessian $\frac{1}{\delta}\bar{A}^\top\bar{A}$ after every change of δ and has to refactorize it, whereas for the active-set method a change of δ costs as little as a new center. Part (ii) shows a second benefit: if the run ends in the proximal phase, the final δ is much smaller than δ_0, and so is the bound on the residual of the linking rows.

The rule is a simple form of the *proximity control* of bundle methods [19], which adapts $1/\delta$ to the progress of the method. A center-only iteration brings no new information into the model, so a longer step is safe; a new column changes the model, and the step is shortened again. Stabilized column generation uses related schedules for the parameters of penalty, box and smoothing stabilization [9, 1, 23]. Rules that compare the actual increase of θ with the increase predicted by D come with convergence guarantees for the bundle method, but they need the Lagrangian bound in every iteration and hence full pricing.

Chapter 4

An Active-Set Method for the Regularized Master Problem

This chapter sets up the **quadratic programming active-set method (QPAS)** for the eliminated regularized master problem. We first derive its optimality conditions and show that the Lagrange multipliers of the nonnegativity constraints are the reduced costs of the columns and the duals of the inequality rows. The optimality test of the master problem and the pricing test of column generation are therefore the same test.

We then recall the primal active-set method: it fixes a **working set** of variables at zero, minimizes over the remaining free variables, and adds or removes one constraint at a time. Every iteration requires the solution of a KKT system whose constraint matrix consists of the convexity rows and the rows of the working set.

The main result of this chapter is a **reordering theorem** for this constraint matrix. The variables of the regularized master problem are partitioned into five sets: representatives, working-set columns, nullspace columns, active surpluses and free surpluses. In this ordering the constraint matrix has a block structure with identity blocks, which is the basis of the explicit nullspace method of the next chapter.

4.1 The Master Problem in Simplified Notation

From now on we consider a single restricted master problem with n columns. To keep the notation light, we drop the explicit reference to the restricted index set and write

- $x \in \mathbb{R}^n$ for the column weights (the vector λ restricted to $\mathcal{J}'$),
- $A_E \in \mathbb{R}^{m_E \times n}$, $A_I \in \mathbb{R}^{m_I \times n}$, $A = [A_E; A_I]$ and $C \in \{0,1\}^{K \times n}$ for the corresponding columns of the linking and convexity matrices,
- $c \in \mathbb{R}^n$ for the costs of the columns, and $a_j \in \mathbb{R}^m$ for column j of A,
- $y = [x; s] \in \mathbb{R}^{n+m_I}$ for the unified variable.

The vector c always denotes the original column costs; the linear term of the quadratic objective is $\tilde{c}$ from (3.6). With this notation the eliminated regularized master problem (3.5) reads

$$\min_{y \in \mathbb{R}^{n+m_I}} \quad f(y) = \frac{1}{2\delta}\, y^\top \bar{A}^\top \bar{A}\, y + \tilde{c}^\top y \tag{4.1a}$$

$$\text{s.t.} \quad \bar{C} y = \mathbf{1}, \tag{4.1b}$$

$$y \geq 0, \tag{4.1c}$$

with

$$\bar{A} = \begin{bmatrix} A_E & 0 \\ A_I & -I \end{bmatrix} \in \mathbb{R}^{m \times (n+m_I)}, \qquad \bar{C} = \begin{bmatrix} C & 0 \end{bmatrix} \in \mathbb{R}^{K \times (n+m_I)}.$$

We write $g = \nabla f(y)$ for the gradient at the current iterate. By the gradient–dual relation (Lemma 3.3.3),

$$g = \begin{bmatrix} g_x \\ g_s \end{bmatrix} = \begin{bmatrix} c - A^\top \pi \\ \pi_I \end{bmatrix}, \qquad \pi = \pi(y) = \hat{\pi} + \tfrac{1}{\delta}(b - \bar{A} y), \tag{4.2}$$

i.e. $g_j = c_j - a_j^\top \pi$ for a column j, and the gradient of surplus i is the dual π_i of inequality row i.

4.2 Optimality Conditions

Problem (4.1) is a convex quadratic program with linear constraints, so the KKT conditions are necessary and sufficient for optimality. Written out, they turn out to be the dual feasibility conditions of the master problem.

Theorem 4.2.1 (KKT conditions of the regularized master problem)**.** A feasible point $y = [x; s]$ of (4.1) is optimal if and only if there exist $\mu \in \mathbb{R}^K$ and $\nu \in \mathbb{R}^{n+m_I}$ with

$$g = \bar{C}^\top \mu + \nu, \qquad \nu \geq 0, \qquad \nu_\ell\, y_\ell = 0 \quad \text{for all } \ell.$$

Written per variable, with $\pi = \pi(y)$, these conditions are equivalent to:

(i) **reduced costs:** $\bar{c}_j(\pi, \mu) = c_j - a_j^\top \pi - \mu_{\psi(j)} \geq 0$ for every column j, with equality if $x_j > 0$;

(ii) **sign condition:** $\pi_i \geq 0$ for every inequality row i;

(iii) **slackness:** $\pi_i = 0$ for every inequality row i with $s_i > 0$.

The multipliers are $\nu_j = \bar{c}_j(\pi, \mu)$ for the columns and $\nu_{n+i} = \pi_i$ for the surpluses.

Proof. The first part is the KKT theorem for convex quadratic programs with the constraints $\bar{C}y = \mathbf{1}$ (multiplier μ) and $y \geq 0$ (multiplier $\nu \geq 0$). We read the stationarity condition $g = \bar{C}^\top \mu + \nu$ row by row.

- Row of column j: the column of $\bar{C}$ is $e_{\psi(j)}$, so $g_j = \mu_{\psi(j)} + \nu_j$. With $g_j = c_j - a_j^\top \pi$ from (4.2), $\nu_j = c_j - a_j^\top \pi - \mu_{\psi(j)} = \bar{c}_j(\pi, \mu)$.
- Row of surplus i: the column of $\bar{C}$ is zero, so $\nu_{n+i} = g_{n+i} = \pi_i$.

With these expressions, $\nu \geq 0$ becomes "$\bar{c}_j \geq 0$ for all j" and "$\pi_i \geq 0$ for all i", and $\nu_\ell y_\ell = 0$ becomes "$x_j > 0 \Rightarrow \bar{c}_j = 0$" and "$s_i > 0 \Rightarrow \pi_i = 0$". These are (i)–(iii). □

Remark 4.2.2 (The optimality test is the pricing test)**.** By Theorem 4.2.1, the multipliers of the nonnegativity constraints of the columns are their reduced costs, the same quantity that the pricing problem minimizes over the columns that have not been generated yet. Solving the regularized master problem to optimality establishes "reduced cost ≥ 0" for the columns inside the restricted problem, and pricing establishes it for the columns outside. The multipliers of the surpluses carry the sign conditions $\pi_I \geq 0$ of the inequality rows. Together with $\pi = \hat{\pi}$, this is exactly the optimality certificate of Theorem 3.4.1.

Remark 4.2.3 (Matching tolerances)**.** In practice the active-set method accepts a multiplier as nonnegative if $\nu_\ell \geq -\varepsilon$, and pricing accepts a column only if $\bar{c}_j < -\varepsilon_{\mathrm{rc}}$. Since both test the same quantity, choosing $\varepsilon \ll \varepsilon_{\mathrm{rc}}$ makes it practically impossible that pricing returns a column that is already in the restricted master problem; a strict guarantee would also have to account for the projection π^+ of Section 3.4.3. We use $\varepsilon = 10^{-10} \max(1, \max_j |c_j|)$ and $\varepsilon_{\mathrm{rc}} = 10^{-6}$.

4.3 The Primal Active-Set Method

The primal active-set method finds the optimal set of variables at zero by trial: it guesses a set of variables fixed at zero, minimizes over the others, and corrects the guess one index at a time.

Definition 4.3.1 (Active set and working set)**.** The **active set** of a feasible point y is $\mathcal{A}(y) = \{\ell : y_\ell = 0\}$. A **working set** is a set $\mathcal{W} \subseteq \{1, \ldots, n + m_I\}$ of indices that are treated as active, i.e. $y_\ell = 0$ for all $\ell \in \mathcal{W}$. We split it into the **column working set** $\mathcal{W}_x$ and the **surplus working set** $\mathcal{W}_s$:

$$\mathcal{W} = \mathcal{W}_x \cup \{\, n + i : i \in \mathcal{W}_s \,\}, \qquad \mathcal{W}_x \subseteq \{1, \ldots, n\}, \quad \mathcal{W}_s \subseteq \{1, \ldots, m_I\}.$$

A surplus $i \in \mathcal{W}_s$ is called **active**; a surplus $i \notin \mathcal{W}_s$ is called **free**.

Given a working set and a feasible iterate y with $y_\ell = 0$ on $\mathcal{W}$, the method computes a search direction p from the **equality-constrained subproblem**

$$\min_p \ \frac{1}{2\delta} p^\top \bar{A}^\top \bar{A} p + g^\top p \qquad \text{s.t.} \qquad \bar{C}p = 0, \qquad p_\ell = 0 \ \ (\ell \in \mathcal{W}). \tag{4.3}$$

Its objective is $f(y + p) - f(y)$, and its constraints keep $\bar{C}(y + p) = \mathbf{1}$ and the working-set variables at zero, so (4.3) minimizes f over the face of the feasible region defined by the working set (ignoring the bounds of the other variables). To write its constraints compactly, let $W_x \in \mathbb{R}^{|\mathcal{W}_x| \times n}$ and $W_s \in \mathbb{R}^{|\mathcal{W}_s| \times m_I}$ be the selection matrices whose rows are the unit vectors $e_j^\top$, $j \in \mathcal{W}_x$, and $e_i^\top$, $i \in \mathcal{W}_s$.

Definition 4.3.2 (Constraint matrix of a working set)**.** The **constraint matrix associated with** $\mathcal{W}$ is

$$D_{\mathcal{W}} = \begin{bmatrix} C & 0 \\ W_x & 0 \\ 0 & W_s \end{bmatrix} \in \mathbb{R}^{(K+|\mathcal{W}_x|+|\mathcal{W}_s|)\times(n+m_I)}. \tag{4.4}$$

The constraints of (4.3) are $D_{\mathcal{W}}\, p = 0$.

The KKT conditions of (4.3) form the linear **KKT system**

$$\begin{bmatrix} \frac{1}{\delta}\bar{A}^\top \bar{A} & D_{\mathcal{W}}^\top \\ D_{\mathcal{W}} & 0 \end{bmatrix} \begin{bmatrix} p \\ -\mu \\ -\nu_x \\ -\nu_s \end{bmatrix} = \begin{bmatrix} -g \\ 0 \\ 0 \\ 0 \end{bmatrix}, \tag{4.5}$$

where $\mu \in \mathbb{R}^K$, $\nu_x \in \mathbb{R}^{|\mathcal{W}_x|}$ and $\nu_s \in \mathbb{R}^{|\mathcal{W}_s|}$ are the Lagrange multipliers of the convexity constraints and of the working-set constraints of the columns and of the surpluses. The first block row reads $D_{\mathcal{W}}^\top[\mu;\nu_x;\nu_s] = g + \frac{1}{\delta}\bar{A}^\top\bar{A}\,p = \nabla f(y+p)$, so these are the multipliers of the gradient at $y+p$. At a subspace minimizer ($p=0$, see below) they are the entries ν_ℓ, $\ell \in \mathcal{W}$, of the multiplier ν of Theorem 4.2.1, whose other entries are zero. If $p = 0$ solves (4.5), the iterate already minimizes f over its face, and we call it a **subspace minimizer**. At a subspace minimizer, the multipliers decide what to do next: if all of them are nonnegative, the KKT conditions of Theorem 4.2.1 hold and the iterate is optimal; otherwise the index with the most negative multiplier is released from the working set. When $p \neq 0$, the method moves along p as far as the bounds of the free variables allow. Algorithm 4.1 summarizes these steps.

Algorithm 4.1: Primal Active-Set Method for the Regularized Master Problem

Input: feasible y, working set $\mathcal{W}$ with $y_\ell = 0$ on $\mathcal{W}$
repeat
  Solve the KKT system (4.5) for p, μ, $\nu = (\nu_x, \nu_s)$
  if $p = 0$ **then**
    if $\nu_\ell \geq -\varepsilon$ for all $\ell \in \mathcal{W}$ **then**
      stop: y is optimal
    else
      $\rho \leftarrow \arg\min_{\ell\in\mathcal{W}} \nu_\ell$; $\mathcal{W} \leftarrow \mathcal{W} \setminus \{\rho\}$ ▷ reduce the working set
    end if
  else
    $\alpha \leftarrow \min\Big\{1, \min_{\ell\notin\mathcal{W},\, p_\ell<0} -\frac{y_\ell}{p_\ell}\Big\}$ ▷ ratio test
    $y \leftarrow y + \alpha p$
    if $\alpha < 1$ **then**
      $q \leftarrow$ the index attaining the minimum; $\mathcal{W} \leftarrow \mathcal{W} \cup \{q\}$ ▷ expand the working set
    end if
  end if
until stopped

Under the usual nondegeneracy assumptions, every iteration either decreases f strictly or changes the working set in a way that cannot cycle, so the method terminates after finitely many iterations [22, 11, 14]. Its cost is dominated by the solution of the KKT system (4.5), which has dimension $n + m_I + K + |\mathcal{W}|$ and grows with every generated column. The rest of this work shows how to avoid ever forming it.

The **nullspace method** of Chapter 5 solves (4.5) through a basis Z of $\mathrm{Null}(D_{\mathcal{W}})$. It relies on two properties of the current working set:

Assumption 4.3.3. The working set $\mathcal{W}$ satisfies the following two conditions.

(i) $D_{\mathcal{W}}$ has full row rank.

(ii) The reduced Hessian $Z^\top \bar{A}^\top \bar{A} Z$ is positive definite, where the columns of Z form a basis of $\mathrm{Null}(D_{\mathcal{W}})$.

Condition (i) guarantees unique multipliers, and condition (ii) a unique search direction. Theorem 4.4.6 below shows that (i) holds for every working set that keeps one column per block free. Condition (ii) can fail, and Chapter 7 shows how the method guarantees it.

4.4 The Constraint Matrix and the Reordering Theorem

The structure of $D_{\mathcal{W}}$ is determined by the convexity matrix C, which has the column generation property, and by the zero columns of the surpluses, which do not. We first arrange the column weights x, and then add the surpluses.

Definition 4.4.1 (Free columns)**.** The **free columns** are $\mathcal{F}_x = \{1, \dots, n\} \setminus \mathcal{W}_x$.

Lemma 4.4.2. If y is feasible and $x_j = 0$ for all $j \in \mathcal{W}_x$, then every block has at least one free column, and the submatrix $C_{\mathcal{F}_x}$ of C has the column generation property.

Proof. For every block k, the convexity constraint gives $\sum_{\psi(j)=k} x_j = 1$. Since $x_j = 0$ for $j \in \mathcal{W}_x$, at least one column of block k is not in $\mathcal{W}_x$, i.e. lies in $\mathcal{F}_x$. So $\mathcal{F}_x$ contains a column of every block, and Lemma 2.2.6 gives the column generation property of $C_{\mathcal{F}_x}$. □

Definition 4.4.3 (Representatives)**.** A **list of representatives** is an ordered list $\mathcal{S} = [S_1, \dots, S_K]$ of free columns with $\psi(S_k) = k$, one per block. The **representative of column** j is

$$\mathrm{rep}(j) = S_{\psi(j)}.$$

The remaining free columns form the **nullspace set** $\mathcal{N}_x = \mathcal{F}_x \setminus \mathcal{S}$.

The representatives exist by Lemma 4.4.2. The column of C belonging to S_k is the unit vector e_k, so the columns of the representatives form the identity, $C_{\mathcal{S}} = I_K$. The name "nullspace set" is explained in Chapter 5: every column in $\mathcal{N}_x$ contributes one basis vector of $\mathrm{Null}(D_{\mathcal{W}})$.

Definition 4.4.4 (Extended ordering)**.** The **extended ordering** of the variables of (4.1) is the partition into the five sets

set	contents	status
$\mathcal{S}$	one representative column per block	free
$\mathcal{W}_x$	columns in the working set	$x_j = 0$ fixed
$\mathcal{N}_x$	the remaining free columns	free
$\mathcal{W}_s$	active surpluses	$s_i = 0$ fixed
$\mathcal{N}_s$	free surpluses	free

We write $[\,\mathcal{S}, \mathcal{W}_x, \mathcal{N}_x \mid \mathcal{W}_s, \mathcal{N}_s\,]$ for this ordering.

Theorem 4.4.5 (Reordering theorem)**.** Let $\mathcal{W} = \mathcal{W}_x \cup \{\, n+i : i \in \mathcal{W}_s \,\}$ be a working set such that every block has a free column, and let $[\,\mathcal{S}, \mathcal{W}_x, \mathcal{N}_x \mid \mathcal{W}_s, \mathcal{N}_s\,]$ be an extended ordering, in which the indices of $\mathcal{W}_x$ and $\mathcal{W}_s$ appear in the same order as the rows of W_x and W_s. Let Q be the permutation matrix that orders the variables in this way. Then (Fig. 4.1)

$$D_{\mathcal{W}}\, Q = \begin{bmatrix} I_K & C_{\mathcal{W}_x} & C_{\mathcal{N}_x} & 0 & 0 \\ 0 & I_{|\mathcal{W}_x|} & 0 & 0 & 0 \\ 0 & 0 & 0 & I_{|\mathcal{W}_s|} & 0 \end{bmatrix}, \tag{4.6}$$

where $C_{\mathcal{W}_x}$ and $C_{\mathcal{N}_x}$ are the columns of C indexed by $\mathcal{W}_x$ and $\mathcal{N}_x$.

Proof. The permutation does not mix column weights and surpluses: $Q = \operatorname{diag}(Q_x, Q_s)$, where Q_x orders the column weights as $[\mathcal{S}, \mathcal{W}_x, \mathcal{N}_x]$ and Q_s orders the surpluses as $[\mathcal{W}_s, \mathcal{N}_s]$. We apply it to the three block rows of $D_{\mathcal{W}} = \left[\begin{smallmatrix} C & 0 \\ W_x & 0 \\ 0 & W_s \end{smallmatrix}\right]$.

- The convexity rows $[\,C \;\; 0\,]$ become $[\,C_{\mathcal{S}} \;\; C_{\mathcal{W}_x} \;\; C_{\mathcal{N}_x} \;\; 0 \;\; 0\,]$, and $C_{\mathcal{S}} = I_K$ as noted after Definition 4.4.3.
- Row ℓ of W_x is the unit vector of the ℓ-th index of $\mathcal{W}_x$. After the permutation this index is in position $K + \ell$, so the rows $[\,W_x \;\; 0\,]$ become $[\,0 \;\; I_{|\mathcal{W}_x|} \;\; 0 \;\; 0 \;\; 0\,]$.
- In the same way, the rows $[\,0 \;\; W_s\,]$ select the surpluses of $\mathcal{W}_s$, which come first among the surpluses, and become $[\,0 \;\; 0 \;\; 0 \;\; I_{|\mathcal{W}_s|} \;\; 0\,]$. □

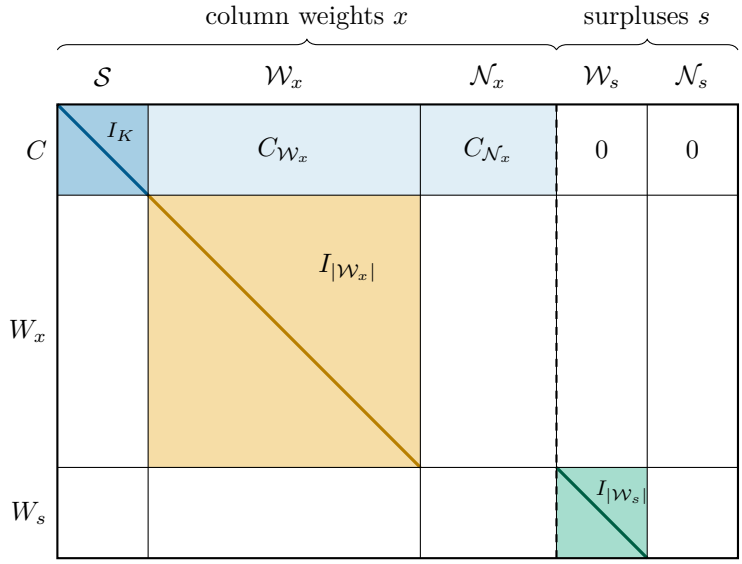


Figure 4.1: Block structure of the reordered constraint matrix $D_{\mathcal{W}}Q$ of Theorem 4.4.5. The diagonal lines are identity blocks; all blocks without a label are zero. The surplus part (right of the dashed line) only contributes the identity block of the active surpluses; the free surpluses $\mathcal{N}_s$ have empty columns.

Theorem 4.4.6 (Full row rank)**.** Under the assumptions of Theorem 4.4.5, $D_{\mathcal{W}}$ has full row rank,

$$\operatorname{rank}(D_{\mathcal{W}}) = K + |\mathcal{W}_x| + |\mathcal{W}_s|,$$

and $\dim \operatorname{Null}(D_{\mathcal{W}}) = |\mathcal{N}_x| + |\mathcal{N}_s|$.

Proof. In the reordered matrix (4.6), the three identity blocks I_K, $I_{|\mathcal{W}_x|}$ and $I_{|\mathcal{W}_s|}$ lie in disjoint rows and disjoint columns, and together they cover all $K + |\mathcal{W}_x| + |\mathcal{W}_s|$ rows. The columns of these blocks therefore form an identity matrix of that size after reordering the rows, so the rows of $D_{\mathcal{W}}Q$ are linearly independent. Permuting columns does not change the rank. By the rank–nullity theorem,

$$\dim \operatorname{Null}(D_{\mathcal{W}}) = (n + m_I) - (K + |\mathcal{W}_x| + |\mathcal{W}_s|) = |\mathcal{N}_x| + |\mathcal{N}_s|,$$

because the column weights split as $n = K + |\mathcal{W}_x| + |\mathcal{N}_x|$ and the surpluses as $m_I = |\mathcal{W}_s| + |\mathcal{N}_s|$. □

The assumption of Theorem 4.4.5, a free column in every block, holds throughout the method: a block's last free column could only reach zero if the weights of its block sum to less than one, which the convexity constraint excludes (see also Lemma 6.2.2). Hence condition (i) of Assumption 4.3.3 holds for every working set that occurs.

Remark 4.4.7 (Comparison with problems without inequality rows)**.** Without inequality rows ($m_I = 0$), the surplus sets are empty, and the extended ordering reduces to the three sets $[\mathcal{S}, \mathcal{W}_x, \mathcal{N}_x]$. The inequality rows thus only add two groups of variables: active surpluses, which behave like working-set columns, and free surpluses, which behave like nullspace columns without a representative. The loss of the column generation property (Remark 2.4.3) does not affect the reordering; it only means that the surpluses form groups of their own.

Figure 4.2 shows the reordering on an actual constraint matrix, taken from the final working set of a column generation run on one of the test instances of Chapter 9. In this example, $K = 7$, $|\mathcal{W}_x| = 523$, $|\mathcal{N}_x| = 186$, $|\mathcal{W}_s| = 34$ and $|\mathcal{N}_s| = 34$.

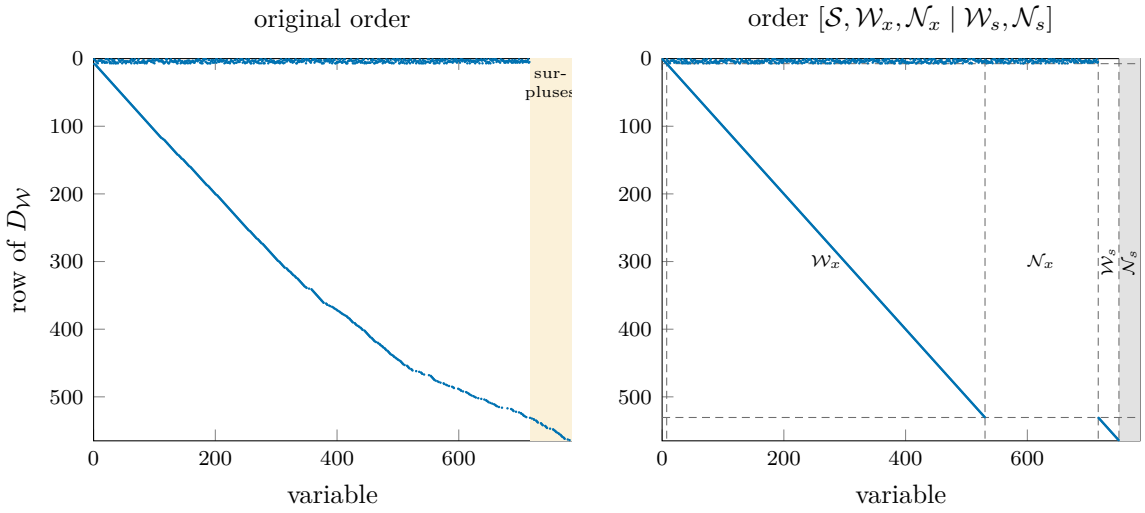


Figure 4.2: Sparsity pattern of the constraint matrix $D_{\mathcal{W}}$ at the end of a column generation run (instance I2 with 30% inequality rows). Left: the original order, with the surpluses at the right (shaded). Right: the order of Theorem 4.4.5, with the groups separated by dashed lines. The convexity rows (top) start with the identity of the representatives; the working-set columns form the long diagonal; the nullspace columns only have entries in the convexity rows; the active surpluses form the identity block in the bottom rows; and the columns of the free surpluses (grey) are empty.

Chapter 5

The Nullspace Method

The nullspace method solves the KKT system (4.5) in two smaller steps. The search direction is written as $p = Z\eta$, where the columns of Z span the nullspace of the constraint matrix. This turns the KKT system into a **reduced system** with the reduced Hessian, whose dimension equals the number of degrees of freedom of the current face. The Lagrange multipliers are then recovered from the remaining equations.

For the regularized master problem both steps can be carried out explicitly. The reordering theorem of the previous chapter gives a nullspace basis that consists of the vectors $e_j - e_{\mathrm{rep}(j)}$ and of unit vectors for the free surpluses. This basis is **never stored**: it is determined by the five sets alone.

The central result of this chapter is a **decoupling** of the reduced system. The surplus part of the direction is given by an explicit formula, and only an $|\mathcal{N}_x| \times |\mathcal{N}_x|$ system has to be solved. Its matrix is the **effective reduced Hessian** $H_{\mathrm{eff}} = (A_{\mathrm{eff}} Z_x)^\top (A_{\mathrm{eff}} Z_x)$, the reduced Hessian of the equality rows together with the inequality rows whose surplus is active. Finally, the multipliers turn out to be the reduced costs of the columns and the duals of the inequality rows, computed from the single vector π without any linear solve.

5.1 The Nullspace Basis

Every search direction must satisfy the constraints of the subproblem (4.3), $D_{\mathcal{W}}\, p = 0$. The set of such directions is $\mathrm{Null}(D_{\mathcal{W}})$, which by Theorem 4.4.6 has dimension $|\mathcal{N}_x| + |\mathcal{N}_s|$. We construct a basis whose vectors are labelled by the indices of $\mathcal{N}_x$ and $\mathcal{N}_s$.

Theorem 5.1.1 (Nullspace basis)**.** Under the assumptions of Theorem 4.4.5, the matrix

$$Z = Q\tilde{Z}, \qquad \tilde{Z} = \begin{bmatrix} -C_{\mathcal{N}_x} & 0 \\ 0 & 0 \\ I_{|\mathcal{N}_x|} & 0 \\ 0 & 0 \\ 0 & I_{|\mathcal{N}_s|} \end{bmatrix} \begin{matrix} \leftarrow \mathcal{S} \\ \leftarrow \mathcal{W}_x \\ \leftarrow \mathcal{N}_x \\ \leftarrow \mathcal{W}_s \\ \leftarrow \mathcal{N}_s \end{matrix} \tag{5.1}$$

is a basis of $\mathrm{Null}(D_{\mathcal{W}})$. In the original order of the variables it is block-diagonal,

$$Z = \begin{bmatrix} Z_x & 0 \\ 0 & Z_s \end{bmatrix}, \qquad Z_x \in \mathbb{R}^{n \times |\mathcal{N}_x|}, \quad Z_s \in \mathbb{R}^{m_I \times |\mathcal{N}_s|}, \tag{5.2}$$

where Z_x acts on the column weights and Z_s is the selection matrix of the free surpluses (its columns are the unit vectors of $\mathbb{R}^{m_I}$ belonging to the rows $i \in \mathcal{N}_s$).

Proof. Since $QQ^\top = I$, a matrix Z satisfies $D_{\mathcal{W}}Z = 0$ if and only if $\tilde{Z} = Q^\top Z$ satisfies $(D_{\mathcal{W}}Q)\tilde{Z} = 0$. We split $\tilde{Z}$ into the five row blocks of the extended ordering,

$$\tilde{Z} = [\, \tilde{Z}_{\mathcal{S}};\ \tilde{Z}_{\mathcal{W}_x};\ \tilde{Z}_{\mathcal{N}_x};\ \tilde{Z}_{\mathcal{W}_s};\ \tilde{Z}_{\mathcal{N}_s} \,].$$

With the reordered matrix of Theorem 4.4.5,

$$(D_{\mathcal{W}}Q)\tilde{Z} = \begin{bmatrix} I_K & C_{\mathcal{W}_x} & C_{\mathcal{N}_x} & 0 & 0 \\ 0 & I & 0 & 0 & 0 \\ 0 & 0 & 0 & I & 0 \end{bmatrix} \begin{bmatrix} \tilde{Z}_{\mathcal{S}} \\ \tilde{Z}_{\mathcal{W}_x} \\ \tilde{Z}_{\mathcal{N}_x} \\ \tilde{Z}_{\mathcal{W}_s} \\ \tilde{Z}_{\mathcal{N}_s} \end{bmatrix} = \begin{bmatrix} \tilde{Z}_{\mathcal{S}} + C_{\mathcal{W}_x}\tilde{Z}_{\mathcal{W}_x} + C_{\mathcal{N}_x}\tilde{Z}_{\mathcal{N}_x} \\ \tilde{Z}_{\mathcal{W}_x} \\ \tilde{Z}_{\mathcal{W}_s} \end{bmatrix}.$$

This vanishes if and only if $\tilde{Z}_{\mathcal{W}_x} = 0$, $\tilde{Z}_{\mathcal{W}_s} = 0$ and $\tilde{Z}_{\mathcal{S}} = -C_{\mathcal{N}_x}\tilde{Z}_{\mathcal{N}_x}$. The blocks $\tilde{Z}_{\mathcal{N}_x}$ and $\tilde{Z}_{\mathcal{N}_s}$ can be chosen freely. With $|\mathcal{N}_x|+|\mathcal{N}_s|$ columns, the choice $\tilde{Z}_{\mathcal{N}_x} = [\, I \ \ 0\,]$, $\tilde{Z}_{\mathcal{N}_s} = [\, 0 \ \ I\,]$ gives (5.1). Its columns are linearly independent, since each has a 1 in a position where all others have a 0, and their number equals $\dim \mathrm{Null}(D_{\mathcal{W}})$, so they form a basis. Finally, $Q = \mathrm{diag}(Q_x, Q_s)$ does not mix column weights and surpluses (proof of Theorem 4.4.5); the first group of columns has nonzeros only in the column weights ($\mathcal{S}$ and $\mathcal{N}_x$), and the second group only in the surpluses ($\mathcal{N}_s$), which gives the block-diagonal form (5.2). □

Corollary 5.1.2 (Explicit nullspace vectors)**.** The columns of Z are

$$z_j = e_j - e_{\mathrm{rep}(j)} \quad (j \in \mathcal{N}_x), \qquad\qquad z_{n+i} = e_{n+i} \quad (i \in \mathcal{N}_s), \tag{5.3}$$

where e_ℓ denotes the ℓ-th unit vector in $\mathbb{R}^{n+m_I}$.

Proof. Column $j \in \mathcal{N}_x$ of $\tilde{Z}$ has a 1 in the position of j (from $\tilde{Z}_{\mathcal{N}_x} = I$) and, in the block of the representatives, minus column j of C. Column j of C is the $\psi(j)$-th unit vector of $\mathbb{R}^K$, so this block has a single -1 in position $\psi(j)$ of the list $\mathcal{S}$, which is the representative $S_{\psi(j)} = \mathrm{rep}(j)$. So $z_j = e_j - e_{\mathrm{rep}(j)}$. The column of $\tilde{Z}$ belonging to $i \in \mathcal{N}_s$ has a single 1 in the position of surplus i, i.e. it is $z_{n+i} = e_{n+i}$. We index it by $n + i$, the position of surplus i in y. □

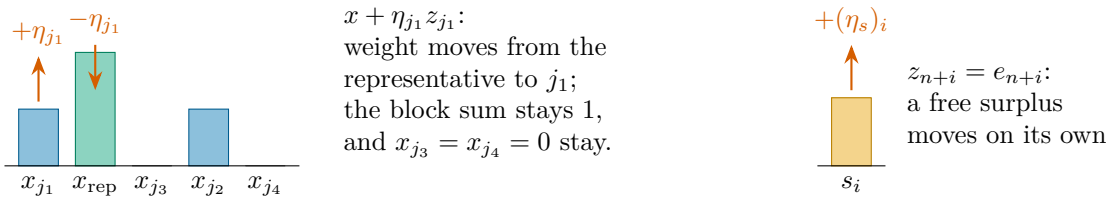


Figure 5.1: Interpretation of the nullspace vectors (5.3) for one block with nullspace columns j_1, j_2, a representative and two working-set columns j_3, j_4. A vector $z_j = e_j - e_{\text{rep}(j)}$ transfers weight between a nullspace column and the representative of its block, which keeps the convexity constraint satisfied and the working-set columns at zero. A free surplus belongs to no block and moves independently.

Note 5.1.3. The column part of z_j is the vector $e_j - e_{\text{rep}(j)} \in \mathbb{R}^n$, which is column j of Z_x. To keep the notation light we use the symbol z_j also for this vector, whenever only the column weights are involved.

Figure 5.1 illustrates these vectors. Corollary 5.1.2 shows that Z is completely determined by the extended ordering: by the sets $\mathcal{N}_x$, $\mathcal{N}_s$ and the representative mapping. It therefore never has to be formed. The same holds for the products of the linking matrix with the nullspace vectors, which appear in the reduced Hessian below. Since column j of $\bar{A}$ is a_j and column $n+i$ is $-e_i^{(I)}$, the unit vector of inequality row i in $\mathbb{R}^m$,

$$\bar{A} z_j = a_j - a_{\text{rep}(j)} =: u_j \quad (j \in \mathcal{N}_x), \qquad \bar{A} z_{n+i} = -e_i^{(I)} \quad (i \in \mathcal{N}_s). \tag{5.4}$$

5.2 The Reduced KKT System

5.2.1 The reduced system

Every direction that satisfies the constraints of (4.3) has the form $p = Z\eta$ with $\eta = [\eta_x; \eta_s] \in \mathbb{R}^{|\mathcal{N}_x|+|\mathcal{N}_s|}$. Inserting $p = Z\eta$ into the objective of (4.3) gives

$$\phi(\eta) = \tfrac{1}{2\delta}\, \eta^\top Z^\top \bar{A}^\top \bar{A} Z\, \eta + (Z^\top g)^\top \eta,$$

an unconstrained quadratic in η. Its stationary points are the solutions of the **reduced KKT system**

$$\frac{1}{\delta}\, Z^\top \bar{A}^\top \bar{A} Z\, \eta = -Z^\top g; \tag{5.5}$$

under Assumption 4.3.3 (ii) the solution is unique and is the minimizer of ϕ. (Equivalently, it is the first block row of the KKT system (4.5) multiplied by $Z^\top$, which removes the multiplier term because $Z^\top D_{\mathcal{W}}^\top = (D_{\mathcal{W}} Z)^\top = 0$.) We call $\bar{H} = Z^\top \bar{A}^\top \bar{A} Z$ the (unscaled) **reduced Hessian**, and $Z^\top g$ the **reduced gradient**. Multiplied by δ, the system reads $\bar{H}\eta = -\delta Z^\top g$, so δ only appears on the right-hand side.

5.2.2 Block structure

To exhibit the structure of $\bar{H}$, we introduce the products of the linking matrices with the column part of the basis,

$$B_E := A_E Z_x \in \mathbb{R}^{m_E \times |\mathcal{N}_x|}, \qquad B_I := A_I Z_x \in \mathbb{R}^{m_I \times |\mathcal{N}_x|},$$

and split the rows of B_I and of π_I according to the status of the surplus of each row:

$$B_{\mathcal{W}_s} := W_s B_I, \quad \pi_{\mathcal{W}_s} := W_s \pi_I \qquad \text{(rows with an active surplus)},$$
$$B_{\mathcal{N}_s} := Z_s^\top B_I, \quad \pi_{\mathcal{N}_s} := Z_s^\top \pi_I \qquad \text{(rows with a free surplus)}.$$

The entries of vectors indexed by the free surpluses, such as $\pi_{\mathcal{N}_s}$ and η_s, are labelled by the rows $i \in \mathcal{N}_s$, and those indexed by the active surpluses by $i \in \mathcal{W}_s$. Since every inequality row belongs to exactly one of the two groups, the two selections together give the identity, $W_s^\top W_s + Z_s Z_s^\top = I$, and therefore

$$B_I^\top B_I = B_I^\top (W_s^\top W_s + Z_s Z_s^\top) B_I = B_{\mathcal{W}_s}^\top B_{\mathcal{W}_s} + B_{\mathcal{N}_s}^\top B_{\mathcal{N}_s}. \tag{5.6}$$

Lemma 5.2.1 (Blocks of the reduced system)**.** With the block-diagonal basis (5.2),

$$\bar{A}Z = \begin{bmatrix} B_E & 0 \\ B_I & -Z_s \end{bmatrix}, \qquad \bar{H} = \begin{bmatrix} H & -B_{\mathcal{N}_s}^\top \\ -B_{\mathcal{N}_s} & I \end{bmatrix}, \qquad Z^\top g = \begin{bmatrix} Z_x^\top g_x \\ \pi_{\mathcal{N}_s} \end{bmatrix},$$

where $H = B_E^\top B_E + B_I^\top B_I = (AZ_x)^\top (AZ_x)$ is the reduced Hessian of the column weights with all linking rows.

Proof. Multiplying the blocks,

$$\bar{A}Z = \begin{bmatrix} A_E & 0 \\ A_I & -I \end{bmatrix} \begin{bmatrix} Z_x & 0 \\ 0 & Z_s \end{bmatrix} = \begin{bmatrix} A_E Z_x & 0 \\ A_I Z_x & -Z_s \end{bmatrix} = \begin{bmatrix} B_E & 0 \\ B_I & -Z_s \end{bmatrix}.$$

Then $\bar{H} = (\bar{A}Z)^\top (\bar{A}Z)$ has the diagonal blocks $B_E^\top B_E + B_I^\top B_I = H$ and $Z_s^\top Z_s = I$, and the off-diagonal block $-B_I^\top Z_s = -(Z_s^\top B_I)^\top = -B_{\mathcal{N}_s}^\top$. Finally, with $g = [g_x; g_s]$ and $g_s = \pi_I$ by (4.2), $Z^\top g = [\, Z_x^\top g_x;\ Z_s^\top \pi_I \,] = [\, Z_x^\top g_x;\ \pi_{\mathcal{N}_s} \,]$. □

5.2.3 Decoupling and the effective reduced Hessian

The identity block in the lower right corner of $\bar{H}$ allows us to eliminate the surplus part of the direction.

Theorem 5.2.2 (Decoupling of the reduced system)**.** The reduced system (5.5) is equivalent to

$$H_{\text{eff}}\, \eta_x = -\delta\, Z_x^\top (c - A_E^\top \pi_E - A_I^\top W_s^\top \pi_{\mathcal{W}_s}), \tag{5.7}$$
$$\eta_s = B_{\mathcal{N}_s}\, \eta_x - \delta\, \pi_{\mathcal{N}_s}, \tag{5.8}$$

with the **effective reduced Hessian**

$$H_{\text{eff}} := H - B_{\mathcal{N}_s}^\top B_{\mathcal{N}_s} = B_E^\top B_E + B_{\mathcal{W}_s}^\top B_{\mathcal{W}_s}. \tag{5.9}$$

Proof. Multiplied by δ, the reduced system is $\bar{H}\eta = -\delta Z^\top g$. With the blocks of Lemma 5.2.1 it consists of the two equations

$$H\eta_x - B_{\mathcal{N}_s}^\top \eta_s = -\delta Z_x^\top g_x, \qquad -B_{\mathcal{N}_s}\eta_x + \eta_s = -\delta \pi_{\mathcal{N}_s}.$$

The second equation, solved for η_s, is (5.8). Substituting it into the first equation gives

$$H\eta_x - B_{\mathcal{N}_s}^\top (B_{\mathcal{N}_s}\eta_x - \delta\pi_{\mathcal{N}_s}) = -\delta Z_x^\top g_x \quad \Longleftrightarrow \quad (H - B_{\mathcal{N}_s}^\top B_{\mathcal{N}_s})\eta_x = -\delta(Z_x^\top g_x + B_{\mathcal{N}_s}^\top \pi_{\mathcal{N}_s}).$$

The matrix on the left is H_{eff}; its second expression in (5.9) follows from $H = B_E^\top B_E + B_I^\top B_I$ and (5.6). For the right-hand side, insert $g_x = c - A_E^\top \pi_E - A_I^\top \pi_I$ from (4.2). Its last term contributes $Z_x^\top A_I^\top \pi_I = B_I^\top \pi_I$, which splits like (5.6) as $B_I^\top \pi_I = B_{\mathcal{W}_s}^\top \pi_{\mathcal{W}_s} + B_{\mathcal{N}_s}^\top \pi_{\mathcal{N}_s}$. Hence

$$Z_x^\top g_x + B_{\mathcal{N}_s}^\top \pi_{\mathcal{N}_s} = Z_x^\top c - B_E^\top \pi_E - B_{\mathcal{W}_s}^\top \pi_{\mathcal{W}_s} - B_{\mathcal{N}_s}^\top \pi_{\mathcal{N}_s} + B_{\mathcal{N}_s}^\top \pi_{\mathcal{N}_s} = Z_x^\top (c - A_E^\top \pi_E - A_I^\top W_s^\top \pi_{\mathcal{W}_s}),$$

using $B_E^\top = Z_x^\top A_E^\top$ and $B_{\mathcal{W}_s}^\top = Z_x^\top A_I^\top W_s^\top$. This is (5.7). The steps can be reversed, so the systems are equivalent. □

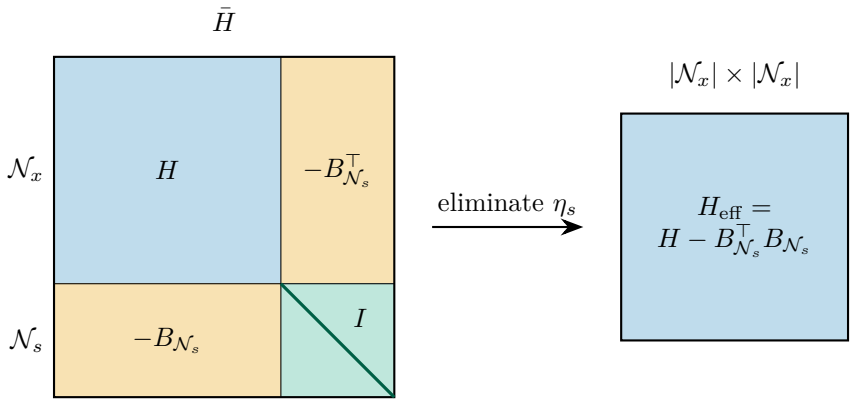


Figure 5.2: The reduced Hessian $\bar{H}$ of Lemma 5.2.1 and its Schur complement. Eliminating the identity block of the free surpluses leaves the effective reduced Hessian H_{eff} on the nullspace columns only.

In linear algebra terms, H_{eff} is the Schur complement of the identity block in $\bar{H}$ (Fig. 5.2). Only an $|\mathcal{N}_x| \times |\mathcal{N}_x|$ system has to be solved; the surplus part (5.8) costs one sparse matrix–vector product. The following corollary gives the interpretation that the rest of this work builds on.

Corollary 5.2.3 (Effective linking matrix)**.** Define the **effective linking matrix** and the corresponding duals

$$A_{\text{eff}} = \begin{bmatrix} A_E \\ W_s A_I \end{bmatrix}, \qquad \pi_{\text{eff}} = \begin{bmatrix} \pi_E \\ \pi_{\mathcal{W}_s} \end{bmatrix},$$

i.e. the equality rows together with the inequality rows whose surplus is active. Then

$$H_{\text{eff}} = (A_{\text{eff}} Z_x)^\top (A_{\text{eff}} Z_x), \qquad H_{\text{eff}}\, \eta_x = -\delta\, Z_x^\top (c - A_{\text{eff}}^\top \pi_{\text{eff}}). \tag{5.10}$$

This is the reduced system of a master problem with equality linking rows only, whose linking rows are the rows of A_{eff}, with the same convexity rows, column working set and representatives.

Proof. Stacking the rows, $A_{\text{eff}} Z_x = [\, A_E Z_x;\ W_s A_I Z_x \,] = [\, B_E;\ B_{\mathcal{W}_s} \,]$, so $(A_{\text{eff}} Z_x)^\top (A_{\text{eff}} Z_x) = B_E^\top B_E + B_{\mathcal{W}_s}^\top B_{\mathcal{W}_s} = H_{\text{eff}}$ by (5.9). Similarly $A_{\text{eff}}^\top \pi_{\text{eff}} = A_E^\top \pi_E + A_I^\top W_s^\top \pi_{\mathcal{W}_s}$, so the right-hand side of (5.10) is that of (5.7). For a problem without inequality rows, Theorem 5.2.2 gives $H_{\text{eff}} = B_E^\top B_E$ and the right-hand side $-\delta Z_x^\top (c - A_E^\top \pi_E)$; replacing A_E by A_{eff} gives (5.10). The duals match as well: since $s_i = 0$ for $i \in \mathcal{W}_s$, (3.9) gives $\pi_{\text{eff}} = \hat{\pi}_{\text{eff}} + \frac{1}{\delta}(b_{\text{eff}} - A_{\text{eff}} x)$ with $b_{\text{eff}} = [b_E; W_s b_I]$ and $\hat{\pi}_{\text{eff}} = [\hat{\pi}_E; W_s \hat{\pi}_I]$, which is the dual estimate of the equality-only problem with data $(A_{\text{eff}}, b_{\text{eff}}, \hat{\pi}_{\text{eff}})$. Its nullspace basis for the same $\mathcal{W}_x$ and $\mathcal{S}$ is the same Z_x. □

Remark 5.2.4 (Interpretation)**.** An inequality row with a *free* surplus does not restrict the column weights: any change of its residual is absorbed by the surplus, so the row contributes neither curvature nor dual pressure to the column part of the problem. An inequality row with an *active* surplus behaves like an equality row. The column part of the method is therefore the method for equality rows, applied to a linking matrix A_{eff} whose rows change with the surplus working set. Without inequality rows, or with all surpluses active, $A_{\text{eff}} = A$ and $H_{\text{eff}} = H$.

Corollary 5.2.5 (Positive definiteness and dimension)**.** $\bar{H}$ is positive definite if and only if H_{eff} is, i.e. if and only if $A_{\text{eff}} Z_x$ has full column rank. In this case

$$|\mathcal{N}_x| \;\le\; \operatorname{rank}(A_{\text{eff}}) \;\le\; m_E + |\mathcal{W}_s|, \qquad \text{and hence} \qquad |\mathcal{N}_x| + |\mathcal{N}_s| \le m.$$

Proof. Completing the square with the blocks of Lemma 5.2.1 gives, for every $\eta = [\eta_x; \eta_s]$,

$$\eta^\top \bar{H} \eta = \eta_x^\top H \eta_x - 2\eta_s^\top B_{\mathcal{N}_s} \eta_x + \eta_s^\top \eta_s = \eta_x^\top H_{\text{eff}} \eta_x + \|\eta_s - B_{\mathcal{N}_s} \eta_x\|^2.$$

If H_{eff} is positive definite, the right-hand side is positive unless $\eta_x = 0$ and then $\eta_s = B_{\mathcal{N}_s}\eta_x = 0$, so $\bar{H}$ is positive definite. Conversely, choosing $\eta_s = B_{\mathcal{N}_s}\eta_x$ shows $\eta_x^\top H_{\text{eff}}\eta_x = \eta^\top \bar{H}\eta$, so H_{eff} is positive definite if $\bar{H}$ is. By Corollary 5.2.3, H_{eff} is a Gram matrix, $\eta_x^\top H_{\text{eff}}\eta_x = \|A_{\text{eff}}Z_x\eta_x\|^2$, which is positive for all $\eta_x \neq 0$ if and only if $A_{\text{eff}}Z_x$ has full column rank. Full column rank requires $|\mathcal{N}_x| \leq \operatorname{rank}(A_{\text{eff}}Z_x) \leq \operatorname{rank}(A_{\text{eff}})$, and A_{eff} has $m_E + |\mathcal{W}_s|$ rows. Adding $|\mathcal{N}_s| = m_I - |\mathcal{W}_s|$ to both sides gives $|\mathcal{N}_x| + |\mathcal{N}_s| \leq m_E + m_I = m$. □

Corollary 5.2.5 is the reason the method scales: as long as the reduced Hessian is positive definite, the matrix H_{eff} that has to be factorized never has more rows than there are linking rows, *however many columns are generated*.

5.2.4 The search direction

With η from Theorem 5.2.2, the direction $p = Z\eta = \sum_{j\in\mathcal{N}_x}\eta_j z_j + \sum_{i\in\mathcal{N}_s}(\eta_s)_i z_{n+i}$ is obtained by inserting the explicit vectors (5.3). Every $z_j = e_j - e_{\text{rep}(j)}$ adds η_j to column j and subtracts it from the representative of its block, and every z_{n+i} moves surplus i. Hence

$$p_j = \eta_j \ (j \in \mathcal{N}_x), \qquad p_{S_k} = -\sum_{j\in\mathcal{N}_x,\,\psi(j)=k} \eta_j \ (k = 1,\dots,K), \qquad (p_s)_i = (A_I p_x)_i - \delta\,\pi_i \ (i \in \mathcal{N}_s), \tag{5.11}$$

and p is zero on $\mathcal{W}_x$ and $\mathcal{W}_s$. For the surplus component we used (5.8) together with $B_{\mathcal{N}_s}\eta_x = Z_s^\top A_I Z_x\eta_x = Z_s^\top A_I p_x$, i.e. $(B_{\mathcal{N}_s}\eta_x)_i = (A_I p_x)_i$. The column part p_x and the vector $A_I p_x$ only involve the columns in $\mathcal{S}\cup\mathcal{N}_x$.

Proposition 5.2.6 (Full step)**.** If the ratio test returns $\alpha = 1$, the new iterate $y + p$ is a subspace minimizer, i.e. the reduced gradient $Z^\top\nabla f(y+p)$ vanishes.

Proof. The gradient of f is affine, $\nabla f(y+p) = g + \frac{1}{\delta}\bar{A}^\top\bar{A}p$. With $p = Z\eta$,

$$Z^\top\nabla f(y+p) = Z^\top g + \tfrac{1}{\delta}Z^\top\bar{A}^\top\bar{A}Z\,\eta = Z^\top g + \tfrac{1}{\delta}\bar{H}\eta = 0$$

by the reduced system (5.5). A full step adds no index to the working set, and since $D_{\mathcal{W}}$ has full row rank, $Z^\top\nabla f(y+p) = 0$ means $\nabla f(y+p) \in \operatorname{range}(D_{\mathcal{W}}^\top)$, i.e. $p = 0$ solves (4.5) at $y + p$. □

After a full step the method can therefore skip the next direction computation and test the multipliers directly. The next proposition shows how the free surpluses act on the duals.

Proposition 5.2.7 (Duals of free-surplus rows)**.** A step $y \leftarrow y + \alpha p$ changes the dual of every row with a free surplus as

$$\pi_i \ \leftarrow \ (1-\alpha)\,\pi_i \qquad (i \in \mathcal{N}_s).$$

In particular $\pi_i = 0$ on these rows after a full step, and at every subspace minimizer.

Proof. By the definition $\pi(y) = \hat{\pi} + \frac{1}{\delta}(b - \bar{A}y)$, the step changes π by $-\frac{\alpha}{\delta}\bar{A}p$. Row i of $\bar{A}p$, for an inequality row, is $(A_I p_x)_i - (p_s)_i$. For $i \in \mathcal{N}_s$, (5.11) gives $(p_s)_i = (A_I p_x)_i - \delta\pi_i$, so $(\bar{A}p)_i = \delta\pi_i$ and the new dual is $\pi_i - \frac{\alpha}{\delta}\delta\pi_i = (1-\alpha)\pi_i$. At a subspace minimizer the reduced gradient vanishes, and its component for $i \in \mathcal{N}_s$ is π_i (Lemma 5.2.1). □

Remark 5.2.8 (Independence of δ and $\hat{\pi}$)**.** $H_{\text{eff}} = (A_{\text{eff}}Z_x)^\top(A_{\text{eff}}Z_x)$ depends only on the columns and the extended ordering (working set and representatives), not on δ, $\hat{\pi}$ or the iterate. A change of the stability center or of δ only changes the right-hand side of (5.7). This is what makes the hot start of Section 6.6 possible.

5.2.5 Implicit evaluation

Neither Z, nor $A_{\text{eff}}Z_x$, nor the gradient has to be stored. Let $M_{\text{eff}} \in \mathbb{R}^{m\times m}$ be the diagonal **row mask** with entry 0 for the rows of free surpluses and 1 otherwise, so that $u_j^\top M_{\text{eff}} u_{j'}$ sums only over the rows of A_{eff}. With $u_j = a_j - a_{\text{rep}(j)}$ from (5.4), the entries of the system (5.10) are, for $j, j' \in \mathcal{N}_x$,

$$(H_{\text{eff}})_{jj'} = u_j^\top M_{\text{eff}}\, u_{j'}, \qquad \left(Z_x^\top (c - A_{\text{eff}}^\top \pi_{\text{eff}})\right)_j = (c_j - a_j^\top M_{\text{eff}}\pi) - (c_{\text{rep}(j)} - a_{\text{rep}(j)}^\top M_{\text{eff}}\pi). \tag{5.12}$$

Both are computed from the sparse columns a_j and the m-vector π. The cost of one entry of the right-hand side is proportional to the number of nonzeros of two columns, independent of the total number of columns n.

5.3 The Lagrange Multipliers

The multipliers follow from the first block row of the KKT system (4.5): $\frac{1}{\delta}\bar{A}^\top \bar{A}p - D_{\mathcal{W}}^\top[\mu;\nu_x;\nu_s] = -g$, i.e.

$$D_{\mathcal{W}}^\top \begin{bmatrix}\mu\\ \nu_x\\ \nu_s\end{bmatrix} = h, \qquad h := \frac{1}{\delta}\bar{A}^\top \bar{A}\, p + g. \tag{5.13}$$

Once p is known, h is known, and (5.13) is an overdetermined but consistent linear system for the multipliers. The reordering makes its solution explicit.

Theorem 5.3.1 (Lagrange multipliers)**.** Let $p = Z\eta$ with η the solution of (5.5). With the extended ordering of Theorem 4.4.5 and $Q^\top h$ split into the blocks $Q^\top h = [h_{\mathcal{S}}; h_{\mathcal{W}_x}; h_{\mathcal{N}_x}; h_{\mathcal{W}_s}; h_{\mathcal{N}_s}]$, the solution of (5.13) is

$$\mu = h_{\mathcal{S}}, \qquad \nu_x = h_{\mathcal{W}_x} - C_{\mathcal{W}_x}^\top \mu, \qquad \nu_s = h_{\mathcal{W}_s}, \tag{5.14}$$

and moreover $h_{\mathcal{N}_x} = C_{\mathcal{N}_x}^\top \mu$ and $h_{\mathcal{N}_s} = 0$.

Proof. Multiplying (5.13) by $Q^\top$ reorders its rows into the five groups; $Q^\top D_{\mathcal{W}}^\top = (D_{\mathcal{W}}Q)^\top$ is the transpose of (4.6):

$$\begin{bmatrix} I_K & 0 & 0\\ C_{\mathcal{W}_x}^\top & I & 0\\ C_{\mathcal{N}_x}^\top & 0 & 0\\ 0 & 0 & I\\ 0 & 0 & 0\end{bmatrix} \begin{bmatrix}\mu\\ \nu_x\\ \nu_s\end{bmatrix} = \begin{bmatrix} h_{\mathcal{S}}\\ h_{\mathcal{W}_x}\\ h_{\mathcal{N}_x}\\ h_{\mathcal{W}_s}\\ h_{\mathcal{N}_s}\end{bmatrix}.$$

Read row group by row group: the first gives $\mu = h_{\mathcal{S}}$, the second $C_{\mathcal{W}_x}^\top \mu + \nu_x = h_{\mathcal{W}_x}$, the fourth $\nu_s = h_{\mathcal{W}_s}$, which is (5.14). The third and fifth groups do not involve ν; they hold because p solves the reduced system. Indeed, by (5.1) the two blocks of $\tilde{Z}^\top Q^\top h = Z^\top h$ are $h_{\mathcal{N}_x} - C_{\mathcal{N}_x}^\top h_{\mathcal{S}}$ and $h_{\mathcal{N}_s}$, and $Z^\top h = \frac{1}{\delta}\bar{H}\eta + Z^\top g = 0$ by (5.5). With $\mu = h_{\mathcal{S}}$ this gives $h_{\mathcal{N}_x} = C_{\mathcal{N}_x}^\top \mu$ and $h_{\mathcal{N}_s} = 0$. □

The multipliers are only needed at a subspace minimizer, where $p = 0$ and therefore $h = g$. In this case Theorem 5.3.1 takes a particularly simple form.

Corollary 5.3.2 (Multipliers at a subspace minimizer)**.** At a subspace minimizer, with $\pi = \pi(y)$,

$$\begin{aligned} \mu_k &= c_{S_k} - a_{S_k}^\top \pi && (k = 1, \dots, K),\\ (\nu_x)_j &= c_j - a_j^\top \pi - \mu_{\psi(j)} = \bar{c}_j(\pi, \mu) && (j \in \mathcal{W}_x),\\ (\nu_s)_i &= \pi_i && (i \in \mathcal{W}_s), \end{aligned}$$

and $\bar{c}_j(\pi,\mu) = 0$ for $j \in \mathcal{S} \cup \mathcal{N}_x$, $\pi_i = 0$ for $i \in \mathcal{N}_s$.

Proof. With $h = g$, the entries of h are $g_j = c_j - a_j^\top \pi$ for column j and π_i for surplus i (by (4.2)). Column j of C is $e_{\psi(j)}$, so $(C_{\mathcal{W}_x}^\top \mu)_j = \mu_{\psi(j)}$. Inserting this into Theorem 5.3.1: $\mu_k = g_{S_k}$, $(\nu_x)_j = g_j - \mu_{\psi(j)}$, $(\nu_s)_i = \pi_i$; the identity $h_{\mathcal{N}_x} = C_{\mathcal{N}_x}^\top \mu$ reads $g_j = \mu_{\psi(j)}$, i.e. $\bar{c}_j = 0$ for $j \in \mathcal{N}_x$; and $h_{\mathcal{N}_s} = 0$ reads $\pi_i = 0$ for $i \in \mathcal{N}_s$. For a representative, $\bar{c}_{S_k} = g_{S_k} - \mu_k = 0$. □

Corollary 5.3.2 gives a concrete form of Remark 4.2.2: the optimality test of the active-set method checks the reduced costs of the working-set columns and the signs of the duals of the inequality rows with active surpluses. All of them are computed from the single vector π. The test touches every column of $\mathcal{W}_x$ once and costs $O(\mathrm{nnz}(A))$; it is the only operation of the method whose cost grows with the number of generated columns.

Remark 5.3.3 ("$\le$" rows)**.** For a "$\le$" row with slack coefficient $\kappa_i = +1$ (Remark 2.4.4), the column of $\bar{A}$ belonging to surplus i is $\kappa_i e_i^{(I)}$ instead of $-e_i^{(I)}$, and the surplus part of the formulas changes as follows:

$$\pi_I = \hat{\pi}_I + \tfrac{1}{\delta}(b_I - A_I x - D_\kappa s), \qquad g_{n+i} = -\kappa_i \pi_i, \qquad \bar{A} z_{n+i} = \kappa_i e_i^{(I)},$$

$$(\nu_s)_i = -\kappa_i \pi_i, \qquad (\eta_s)_i = \kappa_i(\delta \pi_i - (B_{\mathcal{N}_s} \eta_x)_i), \qquad (p_s)_i = \kappa_i(\delta \pi_i - (A_I p_x)_i).$$

In Lemma 5.2.1 the block $-Z_s$ of $\bar{A}Z$ becomes $D_\kappa Z_s$, so the off-diagonal blocks of $\bar{H}$ change sign on these rows. Because $\kappa_i^2 = 1$, the identity block, H_{eff}, A_{eff}, the right-hand side of (5.7) and Proposition 5.2.7 are unchanged. For $\kappa_i = -1$ these are the formulas of the text.

5.4 Summary of One Iteration

Algorithm 5.1 collects the computations of one iteration of the active-set method in nullspace form, and Table 5.1 lists their costs. Here $\bar{\ell}$ denotes the average number of nonzeros of a column a_j.

Algorithm 5.1: One Iteration of the Active-Set Method in Nullspace Form

Given: iterate $y = [x; s]$, the extended ordering, $\pi = \pi(y)$, the factor $H_{\mathrm{eff}}^{\mathrm{emb}} = LDL^\top$ (Section 6.1)
Compute $\mu_k^{\mathrm{eff}} = c_{S_k} - a_{S_k}^\top M_{\mathrm{eff}} \pi$ and the right-hand side $\zeta_j = -\delta\big(c_j - a_j^\top M_{\mathrm{eff}} \pi - \mu_{\psi(j)}^{\mathrm{eff}}\big)$, $j \in \mathcal{N}_x$ ▷ (5.12)
Solve $LDL^\top \eta = \zeta$ (ζ zero outside $\mathcal{N}_x$) and set $\eta_x = \eta_{\mathcal{N}_x}$ ▷ Lemma 6.1.2
Expand $p_x = Z_x \eta_x$ and compute $(p_s)_i = (A_I p_x)_i - \delta \pi_i$, $i \in \mathcal{N}_s$ ▷ (5.11)
if $p \approx 0$ **then**
    $\mu_k = c_{S_k} - a_{S_k}^\top \pi$; $\nu_x =$ reduced costs on $\mathcal{W}_x$; $\nu_s = \pi$ on $\mathcal{W}_s$
    Stop if $\nu_x \ge -\varepsilon$ and $\nu_s \ge -\varepsilon$; otherwise release the index with the most negative of them (Section 6.3)
else
    Ratio test over $\mathcal{S} \cup \mathcal{N}_x \cup \mathcal{N}_s$; $y \leftarrow y + \alpha p$; $\pi \leftarrow \pi - \frac{\alpha}{\delta} \bar{A} p$
    If blocked, expand the working set (Section 6.2)
end if

operation	cost	grows with n?
right-hand side of (5.7)	$O((\lvert\mathcal{N}_x\rvert + K)\,\bar{\ell})$	no
solve restricted to $\mathcal{N}_x$ (Lemma 6.1.2)	at most $O(\lvert\mathcal{N}_x\rvert^2)$	no
surplus direction (5.8)	$O((\lvert\mathcal{N}_x\rvert + K)\,\bar{\ell} + m_I)$	no
ratio test, step and update of π	$O((\lvert\mathcal{N}_x\rvert + K)\,\bar{\ell} + m)$	no
multipliers (Corollary 5.3.2)	$O(\mathrm{nnz}(A) + m_I)$	yes

Table 5.1: Cost of the operations of one iteration, for a factor without structural zeros (Section 6.5). Only the multiplier test touches all columns.

Chapter 6

Efficient Updates of the KKT System

Every change of the working set in the active-set method adds or removes exactly one index: a blocking variable is added, or a variable with a negative multiplier is released. The previous chapter showed that everything the method needs is determined by the extended ordering and by a factorization of the effective reduced Hessian H_{eff}. This chapter shows that a change of the working set changes H_{eff} only by a **low-rank modification**, so that its $LDL^\top$ factorization can be updated in place instead of being recomputed.

We first introduce a **fixed-length representation**, in which the factorization has a fixed dimension and inactive indices are represented by identity rows. We then derive the updates for the three ways of **expanding** the working set and the two ways of **reducing** it, and translate them into row additions, row deletions and rank-one modifications of the factorization, which we realize with a sparse direct solver. Its row deletions leave structural zeros in the factor; we analyse this effect and the **periodic refactorization** that controls it. Chapter 8 develops a dense alternative that avoids it.

Finally, we show that the state of the method can be carried over from one column generation iteration to the next. A new column and a new stability center leave the factorization unchanged, which gives a **hot start** of every master problem. We also analyse the center-only iterations of Section 3.4 in this light.

6.1 Fixed-Length Representation

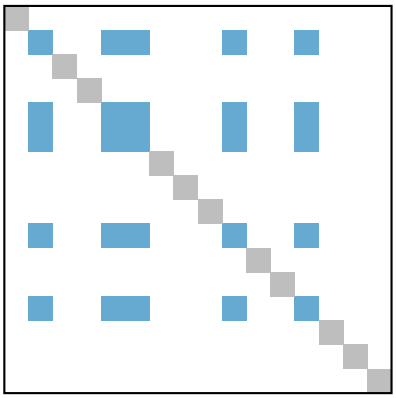
The number of nullspace columns $|\mathcal{N}_x|$ changes in almost every iteration, and the number of columns n grows during column generation. Resizing a sparse factorization at every change would be expensive and complicated. Instead, the factorized matrix is given a fixed dimension.

Definition 6.1.1 (Fixed-length representation)**.** Let $n_{\max} \geq n$ be a fixed **column capacity**, the maximal number of columns that column generation may generate. The **embedded effective reduced Hessian** is the $n_{\max} \times n_{\max}$ matrix

$$(H_{\text{eff}}^{\text{emb}})_{jj'} = \begin{cases} (H_{\text{eff}})_{jj'} = u_j^\top M_{\text{eff}} u_{j'}, & j, j' \in \mathcal{N}_x, \\ 1, & j = j' \notin \mathcal{N}_x, \\ 0, & \text{otherwise}, \end{cases}$$

with $u_j = a_j - a_{\text{rep}(j)}$ and the row mask M_{eff} of (5.12). Index j of the embedded matrix is column j of the master problem. Indices of columns that are not in $\mathcal{N}_x$, including columns that have not been generated yet, carry an identity row and column.

In this representation, a column entering $\mathcal{N}_x$ replaces an identity row by a row of H_{eff}, and a column leaving $\mathcal{N}_x$ is replaced by an identity row (Fig. 6.1). The dimension never changes. The surpluses do not appear at all: by Theorem 5.2.2 they have no index in the factorized matrix, so the inequality rows do not increase the size of the factorization.



$H_{\text{eff}}^{\text{emb}}$, $n_{\max} = 16$

entries of H_{eff} ($\mathcal{N}_x = \{2, 5, 6, 10, 13\}$)
identity on inactive indices

A column entering $\mathcal{N}_x$ fills its row and column (row addition); a column leaving $\mathcal{N}_x$ empties them (row deletion).

Figure 6.1: Fixed-length representation of the effective reduced Hessian: the active indices $\mathcal{N}_x$ are scattered over the fixed index range $1, \ldots, n_{\max}$, and every other index carries an identity row.

The identity rows do not interfere with the computation of the search direction:

Lemma 6.1.2 (Decoupling of inactive indices)**.** Let $H_{\text{eff}}^{\text{emb}} = LDL^\top$ be the $LDL^\top$ factorization without pivoting. Then for every $j \notin \mathcal{N}_x$, row and column j of L are the unit vectors $e_j^\top$ and e_j, and $D_{jj} = 1$. Consequently, for a right-hand side ζ that vanishes outside $\mathcal{N}_x$, the solution of $H_{\text{eff}}^{\text{emb}} \eta = \zeta$ vanishes outside $\mathcal{N}_x$, and its restriction to $\mathcal{N}_x$ solves $H_{\text{eff}} \eta_x = \zeta_{\mathcal{N}_x}$.

Proof. A symmetric permutation P that moves the indices of $\mathcal{N}_x$ to the front, keeping their relative order, turns $H_{\text{eff}}^{\text{emb}}$ into $\operatorname{diag}(H_{\text{eff}}, I)$, because an inactive index has a 1 on the diagonal and zeros elsewhere in its row and column. With $H_{\text{eff}} = L_1 D_1 L_1^\top$, this block-diagonal matrix has the block-diagonal factorization

$$\operatorname{diag}(H_{\text{eff}}, I) = \operatorname{diag}(L_1, I)\ \operatorname{diag}(D_1, I)\ \operatorname{diag}(L_1, I)^\top,$$

which exists because H_{eff} is positive definite. Since P keeps the relative order of the indices of $\mathcal{N}_x$ (and of the inactive indices), the permuted factor $P\,\mathrm{diag}(L_1, I)P^\top$ is again unit lower triangular, and by the uniqueness of the $LDL^\top$ factorization without pivoting it is the factor L of $H_{\text{eff}}^{\text{emb}}$. In it, every inactive index j has row and column e_j, and $D_{jj} = 1$. For the second statement, $\mathrm{diag}(H_{\text{eff}}, I)\,[\eta_x; \eta_0] = [\zeta_{\mathcal{N}_x}; 0]$ gives $H_{\text{eff}}\eta_x = \zeta_{\mathcal{N}_x}$ and $\eta_0 = 0$. □

6.2 Expanding the Working Set

A step is blocked when a free variable reaches zero. The blocking index q is then added to the working set. There are three cases, depending on the set that q belongs to. For a block k we write

$$\mathcal{N}_x^{(k)} = \{\, j \in \mathcal{N}_x : \psi(j) = k \,\}$$

for the nullspace columns of block k; these are the columns whose nullspace vectors $z_j = e_j - e_{S_k}$ refer to the representative S_k.

Lemma 6.2.1 (Ordering after an expansion)**.** Let q be the blocking index. The extended ordering is updated as follows.

(i) $q \in \mathcal{N}_x$: $\mathcal{N}_x \leftarrow \mathcal{N}_x \setminus \{q\}$, $\mathcal{W}_x \leftarrow \mathcal{W}_x \cup \{q\}$.

(ii) $q = S_k \in \mathcal{S}$: choose a **replacement** $S_k' \in \mathcal{N}_x^{(k)}$, the new representative of block k; then $\mathcal{N}_x \leftarrow \mathcal{N}_x \setminus \{S_k'\}$, $\mathcal{W}_x \leftarrow \mathcal{W}_x \cup \{q\}$, and S_k' takes the place of q in the list $\mathcal{S}$.

(iii) q is the free surplus $i \in \mathcal{N}_s$: $\mathcal{N}_s \leftarrow \mathcal{N}_s \setminus \{i\}$, $\mathcal{W}_s \leftarrow \mathcal{W}_s \cup \{i\}$.

In case (ii) the representative itself reaches zero, and another free column of its block has to take its place in the list $\mathcal{S}$. The next lemma shows that one is always available.

Lemma 6.2.2 (Existence of a replacement)**.** In case (ii), the set $\mathcal{N}_x^{(k)}$ is not empty.

Proof. By (5.11), the direction changes the representative by $p_{S_k} = -\sum_{j \in \mathcal{N}_x^{(k)}} \eta_j$. The representative can only reach zero if it decreases, $p_{S_k} < 0$, so the sum is not empty. □

The same argument applies to the zero-curvature directions of Chapter 7, whose representative components are $\mp \sum_{j \in \mathcal{N}_x^{(k)}} \beta_j$ (upper sign for a released surplus, lower sign for a released column), except in the block of a released column r, where d_{S_k} contains the additional term -1; that case is handled by Remark 7.4.4.

Any $S_k' \in \mathcal{N}_x^{(k)}$ is a valid replacement. A good choice is the column with the largest weight, $S_k' = \arg\max_{j \in \mathcal{N}_x^{(k)}} x_j$, since it is the least likely to block in the next step.

Theorem 6.2.3 (Update of the nullspace basis)**.** After an expansion, the nullspace basis changes as follows.

(i) $q \in \mathcal{N}_x$: the vector z_q is removed.

(ii) $q = S_k$ with replacement S_k': the vector $z_{S_k'}$ is removed, and for every $j \in \mathcal{N}_x^{(k)} \setminus \{S_k'\}$ the vector z_j is replaced by

$$z_j' = e_j - e_{S_k'} = z_j - z_{S_k'}.$$

All other vectors are unchanged.

(iii) q the surplus i: the vector e_{n+i} is removed.

Proof. By Corollary 5.1.2 there is one basis vector per index of $\mathcal{N}_x$ and of $\mathcal{N}_s$, given by $z_j = e_j - e_{\mathrm{rep}(j)}$ and $z_{n+i} = e_{n+i}$. In cases (i) and (iii) one index leaves $\mathcal{N}_x$ or $\mathcal{N}_s$, and the representatives do not change, so only its vector disappears. In case (ii), S'_k leaves $\mathcal{N}_x$ and the representative of block k changes from q to S'_k. For $j \in \mathcal{N}_x^{(k)} \setminus \{S'_k\}$ the new vector is $z'_j = e_j - e_{S'_k} = (e_j - e_q) - (e_{S'_k} - e_q) = z_j - z_{S'_k}$. The vectors of other blocks refer to other representatives and do not change. □

Theorem 6.2.4 (Update of the effective reduced Hessian, expansion)**.** After an expansion, H_{eff} changes as follows.

(i) $q \in \mathcal{N}_x$: row and column q are removed.

(ii) $q = S_k$ with replacement S'_k: row and column S'_k are removed, and for $j, j' \in \mathcal{N}_x^{(k)} \setminus \{S'_k\}$ and $\ell \in \mathcal{N}_x \setminus \mathcal{N}_x^{(k)}$

$$(H'_{\mathrm{eff}})_{\ell j} = (H_{\mathrm{eff}})_{\ell j} - (H_{\mathrm{eff}})_{\ell S'_k}, \qquad (H'_{\mathrm{eff}})_{j'j} = (H_{\mathrm{eff}})_{j'j} - (H_{\mathrm{eff}})_{j'S'_k} - (H_{\mathrm{eff}})_{S'_k j} + (H_{\mathrm{eff}})_{S'_k S'_k};$$

all other entries are unchanged.

(iii) q the surplus of inequality row i: row i of A_I joins A_{eff}, and

$$H'_{\mathrm{eff}} = H_{\mathrm{eff}} + v_i v_i^\top, \qquad v_i = Z_x^\top (A_I)_{i,:}^\top, \qquad (v_i)_j = (A_I)_{ij} - (A_I)_{i,\mathrm{rep}(j)} \ \ (j \in \mathcal{N}_x). \tag{6.1}$$

Proof. We use $(H_{\mathrm{eff}})_{\ell j} = (A_{\mathrm{eff}} z_\ell)^\top (A_{\mathrm{eff}} z_j)$ from Corollary 5.2.3.
(i) The vector z_q disappears, so the column $A_{\mathrm{eff}} z_q$ of $A_{\mathrm{eff}} Z_x$ is removed, and with it row and column q of the Gram matrix H_{eff}.
(ii) The vector $z_{S'_k}$ disappears, which removes row and column S'_k. For the remaining columns of block k, $A_{\mathrm{eff}} z'_j = A_{\mathrm{eff}} z_j - A_{\mathrm{eff}} z_{S'_k}$ by Theorem 6.2.3. For $\ell \notin \mathcal{N}_x^{(k)}$ the vector z_ℓ is unchanged, so $(H'_{\mathrm{eff}})_{\ell j} = (A_{\mathrm{eff}} z_\ell)^\top (A_{\mathrm{eff}} z_j - A_{\mathrm{eff}} z_{S'_k}) = (H_{\mathrm{eff}})_{\ell j} - (H_{\mathrm{eff}})_{\ell S'_k}$. For two columns j, j' of block k, expanding the product $(A_{\mathrm{eff}} z_{j'} - A_{\mathrm{eff}} z_{S'_k})^\top (A_{\mathrm{eff}} z_j - A_{\mathrm{eff}} z_{S'_k})$ gives the second formula.
(iii) The basis Z_x does not change, but A_{eff} gains the row $(A_I)_{i,:}$. Hence

$$A'_{\mathrm{eff}} Z_x = \begin{bmatrix} A_{\mathrm{eff}} Z_x \\ (A_I)_{i,:} Z_x \end{bmatrix}, \qquad H'_{\mathrm{eff}} = (A_{\mathrm{eff}} Z_x)^\top (A_{\mathrm{eff}} Z_x) + \big((A_I)_{i,:} Z_x\big)^\top \big((A_I)_{i,:} Z_x\big) = H_{\mathrm{eff}} + v_i v_i^\top.$$

Entry j of $v_i = Z_x^\top (A_I)_{i,:}^\top$ is the product of column j of Z_x, which is $e_j - e_{\mathrm{rep}(j)}$, with row i of A_I, i.e. $(v_i)_j = (A_I)_{ij} - (A_I)_{i,\mathrm{rep}(j)}$. □

The vector v_i is row i of $B_I = A_I Z_x$. Its entry $(v_i)_j = (A_I)_{ij} - (A_I)_{i,\mathrm{rep}(j)}$ is nonzero only if the columns j and $\mathrm{rep}(j)$ have different coefficients in row i; for the 0/1 data of crew scheduling, if exactly one of the two schedules covers flight i.

Proposition 6.2.5. If H_{eff} is positive definite before an expansion, it is positive definite after it.

Proof. By Corollary 5.2.5, H_{eff} is positive definite if and only if the vectors $A_{\mathrm{eff}} z_j$, $j \in \mathcal{N}_x$, are linearly independent. (i) Removing one vector from a linearly independent set leaves it independent. (ii) The new set consists of the unchanged vectors $A_{\mathrm{eff}} z_\ell$, $\ell \notin \mathcal{N}_x^{(k)}$, and the vectors $A_{\mathrm{eff}} z_j - A_{\mathrm{eff}} z_{S'_k}$ for $j \in \mathcal{N}_x^{(k)} \setminus \{S'_k\}$. Together with $A_{\mathrm{eff}} z_{S'_k}$ these $|\mathcal{N}_x|$ vectors span the same space as the $|\mathcal{N}_x|$ old, independent vectors, so they are independent as well, and so is the new set without $A_{\mathrm{eff}} z_{S'_k}$. (iii) For every $\eta_x \neq 0$, $\eta_x^\top (H_{\mathrm{eff}} + v_i v_i^\top) \eta_x = \eta_x^\top H_{\mathrm{eff}} \eta_x + (v_i^\top \eta_x)^2 > 0$. □

6.3 Reducing the Working Set

At a subspace minimizer with a negative multiplier, the index with the most negative multiplier is released from the working set. By Corollary 5.3.2, the multipliers of columns (reduced costs) and of surpluses (duals) are both measured in units of cost, so they can be compared directly. Releasing an index adds a basis vector to Z:

Theorem 6.3.1 (Update of the effective reduced Hessian, reduction)**.** The two kinds of reduction change H_{eff} as follows.

(iv) Column $r \in \mathcal{W}_x$ is released ($\mathcal{W}_x \leftarrow \mathcal{W}_x \setminus \{r\}$, $\mathcal{N}_x \leftarrow \mathcal{N}_x \cup \{r\}$). The basis gains $z_r = e_r - e_{\text{rep}(r)}$, and

$$H'_{\text{eff}} = \begin{bmatrix} H_{\text{eff}} & \gamma_r \\ \gamma_r^\top & \gamma_{rr} \end{bmatrix}, \qquad \gamma_r = (A_{\text{eff}} Z_x)^\top A_{\text{eff}} z_r, \qquad \gamma_{rr} = \|A_{\text{eff}} z_r\|^2. \tag{6.2}$$

(v) The surplus of inequality row i is released ($\mathcal{W}_s \leftarrow \mathcal{W}_s \setminus \{i\}$, $\mathcal{N}_s \leftarrow \mathcal{N}_s \cup \{i\}$). The basis gains e_{n+i}, row i leaves A_{eff}, and

$$H'_{\text{eff}} = H_{\text{eff}} - v_i v_i^\top \tag{6.3}$$

with $v_i = Z_x^\top (A_I)_{i,:}^\top$ as in (6.1).

Proof. (iv) The new column part of the basis is $Z'_x = [\, Z_x \;\; z_r \,]$, so $A_{\text{eff}} Z'_x = [\, A_{\text{eff}} Z_x \;\; A_{\text{eff}} z_r \,]$ and

$$H'_{\text{eff}} = \begin{bmatrix} (A_{\text{eff}} Z_x)^\top \\ (A_{\text{eff}} z_r)^\top \end{bmatrix} \begin{bmatrix} A_{\text{eff}} Z_x & A_{\text{eff}} z_r \end{bmatrix} = \begin{bmatrix} H_{\text{eff}} & (A_{\text{eff}} Z_x)^\top A_{\text{eff}} z_r \\ (A_{\text{eff}} z_r)^\top A_{\text{eff}} Z_x & \|A_{\text{eff}} z_r\|^2 \end{bmatrix}.$$

(v) Z_x does not change, and A_{eff} loses the row $(A_I)_{i,:}$. This is the reverse of case (iii) of Theorem 6.2.4, so $H_{\text{eff}} = H'_{\text{eff}} + v_i v_i^\top$. □

Lemma 6.3.2 (Descent after a reduction)**.** Suppose the reduced Hessian is positive definite after the reduction. Then the next search direction moves the released variable away from its bound: $p_r > 0$ in case (iv), and $(p_s)_i > 0$ in case (v).

Proof. Before the reduction the iterate is a subspace minimizer, so every entry of the reduced gradient $Z^\top g$ is zero. The reduction adds one basis vector z (z_r or e_{n+i}) and leaves the others unchanged, so the new reduced gradient has the single nonzero entry $z^\top g$. For $z = z_r$ it is $g_r - g_{\text{rep}(r)}$, and for $z = e_{n+i}$ it is π_i; by Corollary 5.3.2 both equal the multiplier $\nu < 0$ of the released index. Let ℓ be the position of the new basis vector, so that $Z'^\top g = \nu\, e_\ell$. The new reduced system $\bar{H}'\eta = -\delta Z'^\top g = -\delta\nu\, e_\ell$ gives $\eta = -\delta\nu\, \bar{H}'^{-1} e_\ell$, whose own component is

$$\eta_\ell = -\delta\nu\, (\bar{H}'^{-1})_{\ell\ell} > 0,$$

since $\bar{H}'^{-1}$ is positive definite and $\nu < 0$. By (5.11) this component is the change of the released variable ($p_r = \eta_r$, $(p_s)_i = (\eta_s)_i$). □

Unlike an expansion, a reduction can make the reduced Hessian singular: the new vector $A_{\text{eff}} z_r$ in case (iv) can lie in the span of the old ones, and the downdate in case (v) can remove the last direction of curvature. This is the subject of Chapter 7.

6.4 Updating the $LDL^\top$ Factorization

The updates of Sections 6.2 and 6.3 are applied to the factorization $H_{\text{eff}}^{\text{emb}} = LDL^\top$, with L unit lower triangular and D diagonal, in the natural order of the indices. In this chapter we use the sparse direct solver CHOLMOD [4], which provides the three required modifications [6, 7]; Chapter 8 provides the same operations for a dense factor of H_{eff} alone:

change of the working set	change of $H_{\text{eff}}^{\text{emb}}$	CHOLMOD routine
(i) column leaves $\mathcal{N}_x$	row/column q becomes e_q	`rowdel`
(ii) representative swap	rows/columns of $\mathcal{N}_x^{(k)}$ rewritten	`rowdel` + `rowadd`
(iii) surplus becomes active	$+\,v_i v_i^\top$	`updown` (update)
(iv) column enters $\mathcal{N}_x$	row/column r set to $[\gamma_r; \gamma_{rr}]$	`rowadd`
(v) surplus becomes free	$-\,v_i v_i^\top$	`updown` (downdate)

6.4.1 Row additions and deletions

The following proposition shows what a row addition computes. It explains the cost of the operation and the meaning of the new pivot, which is used to detect singularity in Chapter 7.

Proposition 6.4.1 (Row addition)**.** Let row and column j of $H_{\text{eff}}^{\text{emb}}$ currently be e_j, and partition the indices into the groups $1 = \{1, \dots, j-1\}$, $2 = \{j\}$ and $3 = \{j+1, \dots, n_{\max}\}$:

$$L = \begin{bmatrix} L_{11} & & \\ 0 & 1 & \\ L_{31} & 0 & L_{33} \end{bmatrix}, \qquad D = \operatorname{diag}(D_1, 1, D_3).$$

If row and column j are set to a vector $\gamma = [\gamma_1; \gamma_2; \gamma_3]$, split into the same groups (γ_2 is a scalar), the new factor has

$$\begin{aligned} L_{21}^\top &= D_1^{-1} L_{11}^{-1} \gamma_1 && \text{(triangular solve with the leading block)}, \\ D'_{22} &= \gamma_2 - L_{21} D_1 L_{21}^\top && \text{(new pivot)}, \\ L_{32} &= (\gamma_3 - L_{31} D_1 L_{21}^\top)/D'_{22} && \text{(new column below } j\text{)}, \\ L'_{33} D'_3 L'^\top_{33} &= L_{33} D_3 L_{33}^\top - D'_{22}\, L_{32} L_{32}^\top && \text{(rank-one downdate)}, \end{aligned}$$

while L_{11}, D_1 and L_{31} are unchanged.

Proof. The new factor has the form $L' = \begin{bmatrix} L_{11} & & \\ L_{21} & 1 & \\ L_{31} & L_{32} & L'_{33} \end{bmatrix}$, $D' = \operatorname{diag}(D_1, D'_{22}, D'_3)$. Multiplying out $L'D'L'^\top$ block by block and comparing with the new matrix:

- block $(2,1)$: $L_{21} D_1 L_{11}^\top = \gamma_1^\top$, which gives L_{21};
- entry $(2,2)$: $L_{21} D_1 L_{21}^\top + D'_{22} = \gamma_2$, which gives D'_{22};
- block $(3,2)$: $L_{31} D_1 L_{21}^\top + L_{32} D'_{22} = \gamma_3$, which gives L_{32};
- block $(3,3)$: $L_{31} D_1 L_{31}^\top + D'_{22} L_{32} L_{32}^\top + L'_{33} D'_3 L'^\top_{33}$ must equal the unchanged block $L_{31} D_1 L_{31}^\top + L_{33} D_3 L_{33}^\top$, which gives the downdate.

The blocks $(1,1)$ and $(3,1)$ do not involve row j and stay as they are. □

A row deletion is the reverse operation: row and column j are reset to e_j, which requires the rank-one *update* $L_{33}D_3L_{33}^\top + D_{22}L_{32}L_{32}^\top$ of the trailing block, where D_{22} and L_{32} are the current pivot and column of index j. Two consequences of Proposition 6.4.1 are important.

- If j is larger than every active index, the trailing block contains no active index, the downdate is trivial, and a row addition costs a single triangular solve with the active part, $O(|\mathcal{N}_x|^2)$ for a dense factor. This is used by the refactorization in Section 6.5.
- The pivots D_1 are unchanged, the new pivot is D'_{22}, and the downdate changes the active pivots in D_3. Since the determinant of an $LDL^\top$ factorization is the product of its pivots, all these changes together multiply the determinant of the active part by the Schur complement $\sigma_r = \gamma_{rr} - \gamma_r^\top H_{\text{eff}}^{-1}\gamma_r$ of (6.2): $\det H'_{\text{eff}} = \det H_{\text{eff}} \cdot \sigma_r$. If $j = r$ is larger than every active index, $D'_{22} = \sigma_r$; in general $D'_{22} \geq \sigma_r$, and a singular H'_{eff} may show up in the pivot of a later index rather than in D'_{22} (Chapter 7).

6.4.2 Rank-one modifications

The surplus cases (iii) and (v) modify the whole matrix by $\pm v_i v_i^\top$. The factorization $LDL^\top \pm v_i v_i^\top = L'D'L'^\top$ is computed by the classical rank-one update and downdate algorithms [13, 6], which process the columns of L along the path in the elimination tree that starts at the first nonzero of v_i. Their cost is proportional to the number of entries of L in these columns. An update keeps the matrix positive definite (Proposition 6.2.5). A downdate can make it singular; by the matrix determinant lemma (write $H_{\text{eff}} - v_i v_i^\top = H_{\text{eff}}(I - H_{\text{eff}}^{-1} v_i v_i^\top)$; the matrix $I - ab^\top$ has the eigenvalue $1 - b^\top a$ for the eigenvector a and the eigenvalue 1 on the orthogonal complement of b),

$$\det(H_{\text{eff}} - v_i v_i^\top) = \det H_{\text{eff}} \cdot (1 - v_i^\top H_{\text{eff}}^{-1} v_i),$$

so this happens exactly when $\omega_i := 1 - v_i^\top H_{\text{eff}}^{-1} v_i$ vanishes. Chapter 7 shows that $\omega_i \geq 0$ always holds for the downdates of the active-set method, and treats the case $\omega_i = 0$.

6.4.3 Sparse solves

The right-hand side of (5.7) vanishes outside $\mathcal{N}_x$. By Lemma 6.1.2, the solve only has to touch the rows of L that can be reached from $\mathcal{N}_x$ in the graph of L, the set $\text{reach}_L(\mathcal{N}_x)$. CHOLMOD supports such solves directly. Their cost is independent of $n_{\max}$, which is what makes a large capacity $n_{\max}$ affordable.

6.4.4 Assembly of a new row

A row addition in case (iv) of Theorem 6.3.1 needs the vector γ_r of (6.2). By (5.12), $(\gamma_r)_j = u_j^\top M_{\text{eff}} u_r$ with $u_r = a_r - a_{\text{rep}(r)}$. In an implementation, $M_{\text{eff}} u_r$ is scattered into a dense m-vector once, with the entries of the rows of free surpluses set to zero. Then $(\gamma_r)_j = a_j^\top (M_{\text{eff}} u_r) - a_{\text{rep}(j)}^\top (M_{\text{eff}} u_r)$ is computed for every $j \in \mathcal{N}_x$, where the products with the K representatives are computed once and reused. The cost is $O((|\mathcal{N}_x| + K)\,\bar{\ell})$, independent of n. A representative swap (case (ii) of Lemma 6.2.1) first deletes all rows of $\mathcal{N}_x^{(k)}$, including the row of S'_k, and then adds the rows of $\mathcal{N}_x^{(k)} \setminus \{S'_k\}$ again, assembled with the new vectors z'_j. Every intermediate matrix is then the Gram matrix of a subset of the old vectors, or of such a subset together with some of the new vectors $A_{\text{eff}} z'_j$. Together with $A_{\text{eff}} z_{S'_k}$, the old and the new vectors each form a basis of the same space (Proposition 6.2.5), so every such set is linearly independent, and the matrix stays positive definite. This costs the same as applying the formulas of Theorem 6.2.4, and it needs no stored copy of H_{eff}.

6.5 Structural Zeros and Periodic Refactorization

A row deletion sets row and column j of L to e_j *numerically*, but a sparse factorization keeps the positions of the removed entries in its sparsity pattern: the pattern of a column of a simplicial factor can only grow. The same holds for rank-one modifications. Over a long sequence of updates, the stored pattern of L therefore becomes the union of the patterns of all factors that ever existed. To quantify this effect we distinguish the following quantities, which are measured in Section 9.7.

Definition 6.5.1 (Stored and effective entries)**.** The **stored entries** of L are the off-diagonal positions in its sparsity pattern. The **effective entries** are the stored entries whose value is nonzero. The difference are the **structural zeros**. The **dense equivalent** is $|\mathcal{N}_x|(|\mathcal{N}_x|-1)/2$, the number of off-diagonal entries of a dense factor of H_{eff}.

Every solve and every update iterates over the stored entries of the columns it touches, including the structural zeros. Moreover, structural zeros can connect active indices to indices that are no longer active, which enlarges $\text{reach}_L(\mathcal{N}_x)$. The stored pattern only returns to the effective one when the factorization is rebuilt.

Definition 6.5.2 (Refactorization)**.** A **refactorization** resets the factor to the identity and re-activates all indices of $\mathcal{N}_x$ by row additions *in increasing index order*, with the rows assembled as in Section 6.4.

Proposition 6.5.3 (Cost of a refactorization)**.** A refactorization costs $O(|\mathcal{N}_x|^3)$ arithmetic operations for the factor (about $|\mathcal{N}_x|^3/3$ for a dense H_{eff}) and $O\big(|\mathcal{N}_x|(|\mathcal{N}_x|+K)\,\bar{\ell}\big)$ for the assembly of the rows. Its stored pattern contains no entries left over from deleted rows.

Proof. When the indices are added in increasing order, every new index is the largest active index, so by Proposition 6.4.1 each row addition is a single triangular solve with the current active part, which costs $O(\ell^2)$ for the ℓ-th index. Summing over $\ell = 1, \dots, |\mathcal{N}_x|$ gives $O(|\mathcal{N}_x|^3)$, about $|\mathcal{N}_x|^3/3$ as for a dense Cholesky factorization. Each row costs $O\big((|\mathcal{N}_x|+K)\,\bar{\ell}\big)$ to assemble (Section 6.4). Since no row is ever deleted during a refactorization, every stored entry is computed from the current matrix. □

The method refactorizes after every τ updates of the factorization (row additions, row deletions and rank-one modifications), and whenever a pivot has become too small (Chapter 7). The **refactorization interval** τ balances the cost of a rebuild against the growth of the structural zeros, which is roughly proportional to the number of updates. Its effect is studied in Section 9.7.

Remark 6.5.4 (Natural ordering)**.** Sparse direct solvers usually reorder a matrix to reduce fill-in. For H_{eff} this would gain nothing: the effective factor is almost exactly as dense as a dense factor of H_{eff} (Section 9.7), because columns of the same block share their representative and are therefore strongly coupled. The natural ordering has two advantages: index j of the factor is column j of the master problem, and the refactorization of Definition 6.5.2 only appends rows. The density of the factor is also the starting point of Chapter 8.

6.6 Hot Start Between Column Generation Iterations

Between two master problems, regularized column generation adds columns and moves the stability center (Algorithm 3.1). Both changes can be absorbed without touching the factorization.

Theorem 6.6.1 (Hot start)**.** Let $y = [x; s]$ be the final iterate of a master problem, with its extended ordering and the factorization of H_{eff}.

(i) **New column.** If a column $n+1 \le n_{\max}$ is added with weight $x_{n+1} = 0$ and placed in the working set $\mathcal{W}_x$, then y remains feasible, $\pi(y)$ is unchanged, and the nullspace basis, A_{eff}, H_{eff} and its factorization are unchanged. The multiplier of the new column is its reduced cost $c_{n+1} - a_{n+1}^\top \pi - \mu_{\psi(n+1)}$.

(ii) **New center.** If the center changes from $\hat{\pi}$ to $\hat{\pi} + \Delta$, then y remains feasible, the ordering and the factorization are unchanged, and the dual estimate becomes $\pi(y) + \Delta$.

Proof. (i) The new column has weight zero, so the convexity constraint of its block and the product $\bar{A}y$ are unchanged; hence y stays feasible and $\pi(y) = \hat{\pi} + \frac{1}{\delta}(b - \bar{A}y)$ is unchanged. The new index is placed in $\mathcal{W}_x$, so the sets $\mathcal{S}$, $\mathcal{N}_x$, $\mathcal{N}_s$, and with them Z, A_{eff} and H_{eff}, are unchanged. In the fixed-length representation, index $n+1$ already carries an identity row, so the factorization is unchanged too. By Corollary 5.3.2, the multiplier of a working-set column is its reduced cost $c_{n+1} - a_{n+1}^\top \pi - \mu_{\psi(n+1)}$. (ii) The constraints $\bar{C}y = \mathbf{1}$, $y \ge 0$ do not involve $\hat{\pi}$, and $H_{\text{eff}} = (A_{\text{eff}} Z_x)^\top (A_{\text{eff}} Z_x)$ does not depend on it either. Finally $\pi(y) = (\hat{\pi} + \Delta) + \frac{1}{\delta}(b - \bar{A}y)$ is the old dual estimate plus Δ. □

If only a column is added, the previous optimum remains a subspace minimizer, and its only negative multiplier is that of the new column: pricing found its reduced cost below $-\varepsilon_{\text{rc}}$ (with $\pi^+ = \pi$ up to the master tolerance). The next master problem then starts by releasing the new column (case (iv) of Theorem 6.3.1). Under the proximal point rule the center moves at the same time; the gradient changes, the iterate is in general no longer a subspace minimizer, and the next master problem starts with a direction computation on the inherited factorization. The surpluses are never added or removed, and their status is carried over like that of the columns.

6.6.1 Center-only iterations

The hot start also explains the behaviour of the center-only iterations of Section 3.4, in which the center moves to the current dual and no column is added. Let $\Pi = \bar{A}Z\,\bar{H}^{-1}(\bar{A}Z)^\top$ denote the orthogonal projector onto the range of $\bar{A}Z$.

Proposition 6.6.2 (Center-only iterations)**.** Let y be optimal for the current master problem, with scaled residual $w = (b - \bar{A}y)/\delta$ (the auxiliary variable of (3.3)) and dual $\pi = \hat{\pi} + w$, and let the center move to $\hat{\pi}' = \pi$. Suppose that the next master problem takes one full step and keeps the working set. Then

(i) the new scaled residual is $w' = (I - \Pi)\,w$ and the new dual is $\pi' = \pi + (I - \Pi)\,w$;

(ii) if the working set stays the same in the following center-only iterations, the primal iterate no longer changes, and every further iteration moves the dual by the fixed vector w':

$$\pi^{(t+1)} = \pi^{(t)} + w', \qquad (\nu_x)_j^{(t+1)} = (\nu_x)_j^{(t)} - u_j^\top w', \qquad (\nu_s)_i^{(t+1)} = (\nu_s)_i^{(t)} + (w'_I)_i,$$

where t counts the center-only iterations and $u_j = a_j - a_{\text{rep}(j)}$ for every column j.

Proof. (i) The center moves by $\Delta = \hat{\pi}' - \hat{\pi} = \pi - \hat{\pi} = w$. By Theorem 6.6.1, the dual at the old iterate becomes $\pi + \Delta$, and since the gradient is $g = \hat{c} - \bar{A}^\top \pi$ (Lemma 3.3.3), it changes by $-\bar{A}^\top \Delta$. The old iterate was optimal, so its reduced gradient was zero, and the new reduced gradient is

$$Z^\top g' = Z^\top g - Z^\top \bar{A}^\top \Delta = -(\bar{A}Z)^\top \Delta.$$

The reduced system $\bar{H}\eta = -\delta Z^\top g'$ gives $\eta = \delta\,\bar{H}^{-1}(\bar{A}Z)^\top \Delta$, hence $\bar{A}p = \bar{A}Z\eta = \delta\,\Pi\Delta$. After the full step, the dual is

$$\pi' = (\pi + \Delta) - \tfrac{1}{\delta}\bar{A}p = \pi + \Delta - \Pi\Delta = \pi + (I - \Pi)\Delta,$$

and by Proposition 3.3.5 the new scaled residual is $w' = \pi' - \hat{\pi}' = (I - \Pi)\Delta = (I - \Pi)\, w$.
(ii) The vector $w' = (I - \Pi)w$ is orthogonal to the range of $\bar{A}Z$, i.e. $(\bar{A}Z)^\top w' = 0$. In the next center-only iteration the center moves by $\Delta' = w'$, and the same computation as in (i) gives the reduced gradient $-(\bar{A}Z)^\top \Delta' = 0$: the direction is zero and y does not move. The dual becomes $\pi' + \Delta'$; since the new center is π', the scaled residual is $(\pi' + \Delta') - \pi' = w'$ again, and the argument repeats. The multipliers follow from Corollary 5.3.2. A shift of π by Δ' changes g_j by $-a_j^\top \Delta'$, hence $(\nu_x)_j = g_j - g_{\mathrm{rep}(j)}$ by $-u_j^\top \Delta'$; and $(\nu_s)_i = \pi_i$ changes by $(\Delta'_I)_i$. □

Proposition 6.6.2 shows that in a center-only phase the dual walks along a fixed direction with step length $\|w'\|$, while the primal iterate stays put. The walk ends when a multiplier changes sign, so that the working set changes, or when pricing finds an improving column. If the scaled residual w' is small, many such iterations are needed, which explains the long tail of center-only iterations observed in Section 9.4. Each of them is cheap: one direction computation, which returns zero, and one multiplier test. The step length is proportional to $1/\delta$, and a change of δ costs nothing by Remark 5.2.8; the geometric rule of Section 3.5 uses this to remove the tail (Section 9.9).

6.7 The Complete Method

Algorithm 6.1 combines the results of Chapters 4 to 6. It is called in the first line of the loop of regularized column generation (Algorithm 3.1), and its state, consisting of the iterate, the extended ordering and the factorization, is kept from one call to the next.

Algorithm 6.1: Hot-Started Nullspace Active-Set Method

Input: previous state (iterate y, ordering, factor), new columns, center $\hat{\pi}$
Put the new columns into $\mathcal{W}_x$ with weight 0 ▷ Theorem 6.6.1
$\pi \leftarrow \hat{\pi} + (b - \bar{A}y)/\delta$
repeat
  if a full step was taken and the working set has not changed since **then**
    $p \leftarrow 0$ ▷ Proposition 5.2.6
  else
    Compute p by Algorithm 5.1 (lines 2–4)
  end if
  if $p \approx 0$ **then**
    Compute the multipliers (Corollary 5.3.2)
    if $\nu_x \geq -\varepsilon$ and $\nu_s \geq -\varepsilon$ **then return** y, π, μ
    end if
    Release the index ρ with the most negative multiplier by Algorithm 7.1
  else
    Ratio test over $\mathcal{S} \cup \mathcal{N}_x \cup \mathcal{N}_s$; $y \leftarrow y + \alpha p$; $\pi \leftarrow \pi - \frac{\alpha}{\delta}\bar{A}p$
    if blocked by q **then**
      Expand: `rowdel` ($q \in \mathcal{N}_x$), representative swap ($q \in \mathcal{S}$), or update ($q \in \mathcal{N}_s$)
    end if
  end if
  Refactorize if τ updates have passed since the last refactorization ▷ sparse factor only, or if an expansion violated the pivot invariant
until returned

The algorithm accesses the factorization only through the operations of Section 6.4: row additions, row deletions, rank-one modifications and solves. It is therefore independent of how the factor is stored. With the dense factor of Chapter 8, the periodic refactorization in the last line is dropped, and only the pivot test remains (Corollary 8.5.3 and the remark after it).

Chapter 7

Singular Reduced Hessians

The nullspace method requires the reduced Hessian to be positive definite (Assumption 4.3.3 (ii)). The Hessian of the regularized master problem is only positive semidefinite, and its rank is bounded by the number of linking rows, while the number of columns keeps growing. It is therefore unavoidable that a reduction of the working set occasionally produces a nullspace direction along which the objective has no curvature.

This chapter shows where such singularities can arise, how they are detected from the pivots of the factorization, and how they are resolved. The resolution is a **zero-curvature step**: along a direction in the kernel of the reduced Hessian, the objective decreases linearly, and the step is taken until a variable reaches its bound. We prove that adding this variable to the working set removes the singularity, so that the interrupted reduction can be completed. The treatment covers released columns and released surpluses with one argument, and keeps every working-set variable other than the released one exactly at its bound.

7.1 Where Singularity Can Arise

By Corollary 5.2.5, the reduced Hessian is positive definite if and only if $H_{\text{eff}} = (A_{\text{eff}} Z_x)^\top (A_{\text{eff}} Z_x)$ is, i.e. if and only if the vectors $A_{\text{eff}} z_j$, $j \in \mathcal{N}_x$, are linearly independent. Proposition 6.2.5 shows that expansions preserve this property. Singularity can therefore only arise in one of the two reductions of Theorem 6.3.1.

Proposition 7.1.1 (Singular reductions)**.** Let H_{eff} be positive definite before the reduction.

(i) After releasing column r (case (iv) of Theorem 6.3.1), the matrix H'_{eff} of (6.2) is singular if and only if $A_{\text{eff}} z_r \in \operatorname{span}\{A_{\text{eff}} z_j : j \in \mathcal{N}_x\}$, which is the case if and only if the Schur complement $\sigma_r = \gamma_{rr} - \gamma_r^\top H_{\text{eff}}^{-1} \gamma_r$ vanishes.

(ii) After releasing the surplus of row i (case (v) of Theorem 6.3.1), the matrix $H'_{\text{eff}} = H_{\text{eff}} - v_i v_i^\top$ is singular if and only if $\omega_i = 1 - v_i^\top H_{\text{eff}}^{-1} v_i$ vanishes.

Moreover $\sigma_r \geq 0$ and $0 \leq \omega_i \leq 1$.

Proof. (i) H'_{eff} is the Gram matrix of the vectors $A_{\text{eff}} z_j$, $j \in \mathcal{N}_x$, and $A_{\text{eff}} z_r$. It is singular if and only if these vectors are linearly dependent; since the vectors $A_{\text{eff}} z_j$ alone are independent, this means $A_{\text{eff}} z_r \in \operatorname{span}\{A_{\text{eff}} z_j\}$. For the bordered matrix $H'_{\text{eff}} = \left[\begin{smallmatrix} H_{\text{eff}} & \gamma_r \\ \gamma_r^\top & \gamma_{rr} \end{smallmatrix}\right]$ of (6.2), block elimination gives $\det H'_{\text{eff}} = \det H_{\text{eff}} \cdot (\gamma_{rr} - \gamma_r^\top H_{\text{eff}}^{-1} \gamma_r) = \det H_{\text{eff}} \cdot \sigma_r$, so H'_{eff} is singular if and only if $\sigma_r = 0$.
(ii) By the matrix determinant lemma, $\det(H_{\text{eff}} - v_i v_i^\top) = \det H_{\text{eff}} \cdot (1 - v_i^\top H_{\text{eff}}^{-1} v_i) = \det H_{\text{eff}} \cdot \omega_i$, so H'_{eff} is singular if and only if $\omega_i = 0$.
In both cases H'_{eff} is a Gram matrix, of $[\, A_{\text{eff}} Z_x \;\; A_{\text{eff}} z_r \,]$ in (i) and of $A'_{\text{eff}} Z_x$ (A_{eff} without row i) in (ii). It is therefore positive semidefinite, so $\det H'_{\text{eff}} \geq 0$, and with $\det H_{\text{eff}} > 0$ we get $\sigma_r \geq 0$ and $\omega_i \geq 0$. Finally $\omega_i \leq 1$ because H_{eff}^{-1} is positive definite, so $v_i^\top H_{\text{eff}}^{-1} v_i \geq 0$. □

Part (ii) of Proposition 7.1.1 also shows that the downdate of a surplus release can never make H_{eff} indefinite: in exact arithmetic, the worst case is a singular matrix.

7.2 Detection

Testing σ_r or ω_i before the update would cost an additional solve for every reduction. Instead, the update is performed, and the pivots D_{jj} of the factorization $H_{\text{eff}}^{\text{emb}} = LDL^\top$ are inspected afterwards.

Definition 7.2.1 (Pivot invariant)**.** For $j \in \mathcal{N}_x$, the **pivot scale** of index j is $\|u_j\|^2$ with $u_j = a_j - a_{\text{rep}(j)}$ from (5.4) (all rows, not only those of A_{eff}). The factorization satisfies the **pivot invariant** with tolerance $\varepsilon_{\text{piv}} > 0$ if

$$D_{jj} \;>\; \varepsilon_{\text{piv}} \, \|u_j\|^2 \qquad \text{for all } j \in \mathcal{N}_x.$$

The scale $\|u_j\|^2$ uses all rows on purpose: a column j whose difference u_j has all its nonzeros in rows with free surpluses has $(H_{\text{eff}})_{jj} = 0$, and a scale based on H_{eff} would hide this singularity.
The determinant of H'_{eff} is the product of its pivots. When a column is released (part (i) of Proposition 7.1.1), the row addition creates the pivot of r and downdates the pivots of the later active indices, and their product changes by the factor σ_r (Proposition 6.4.1). When a surplus is released (part (ii)), the pivots change by the factor ω_i in total, since $\det H'_{\text{eff}} = \det H_{\text{eff}} \cdot \omega_i$. All pivots are therefore checked, not only the new one. If all pivots satisfied the invariant before

the update, a vanishing σ_r or ω_i must therefore show up as a pivot that violates it after the update. This detection is a numerical heuristic rather than a theorem about floating-point arithmetic, but it follows directly from the determinant identities of Proposition 7.1.1. When it fires, the update is undone by a refactorization with the previous working set (Definition 6.5.2), which also removes any large entries created by dividing by a tiny pivot. We use $\varepsilon_{\text{piv}} = 10^{-10}$.

For the dense factorization of Chapter 8 the detection is sharper. A released column is appended as the last index, all other pivots stay unchanged, and the new pivot is σ_r itself (Corollary 8.3.2). The test then measures the angle between the new direction and the span of the old ones (Lemma 8.3.3). A released surplus is a rank-one downdate, whose pivots stay positive if and only if $\omega_i > 0$, and whose pivot ratios multiply to ω_i (Proposition 8.4.2).

7.3 Zero-Curvature Directions

Assume now that the iterate y is a subspace minimizer for the working set $\mathcal{W}$, that the index to be released has a negative multiplier, and that its release would make the reduced Hessian singular. We construct a direction along which the objective decreases linearly and the working set (apart from the released index) is respected. Recall from Corollary 5.3.2 what is known at a subspace minimizer:

$$\begin{aligned} g^\top z_j = g_j - g_{\text{rep}(j)} = 0 \;\; (j \in \mathcal{N}_x), \qquad & \pi_i = 0 \;\; (i \in \mathcal{N}_s), \\ g_r - g_{\text{rep}(r)} = (\nu_x)_r \;\; (r \in \mathcal{W}_x), \qquad & \pi_i = (\nu_s)_i \;\; (i \in \mathcal{W}_s). \end{aligned} \tag{7.1}$$

In this section $z_j = e_j - e_{\text{rep}(j)}$ denotes the column part in $\mathbb{R}^n$, and i' is a generic inequality row, as opposed to the specific row i of a released surplus.

7.3.1 Released column

Theorem 7.3.1 (Zero-curvature direction, column)**.** Let $r \in \mathcal{W}_x$ have the multiplier $(\nu_x)_r < 0$, and suppose that the release of r is singular, i.e. $A_{\text{eff}} z_r = \sum_{j \in \mathcal{N}_x} \beta_j A_{\text{eff}} z_j$ for some coefficients β (these are $\beta = H_{\text{eff}}^{-1} \gamma_r$ with γ_r from (6.2)). Define $d = [d_x; d_s]$ by

$$d_x = z_r - \sum_{j \in \mathcal{N}_x} \beta_j z_j, \qquad (d_s)_{i'} = (A_I d_x)_{i'} \;\; (i' \in \mathcal{N}_s), \qquad (d_s)_{i'} = 0 \;\; (i' \in \mathcal{W}_s). \tag{7.2}$$

Then

(i) $C d_x = 0$, and $d_\ell = 0$ for every $\ell \in \mathcal{W} \setminus \{r\}$;

(ii) $\bar{A} d = 0$;

(iii) $\nabla f(y)^\top d = (\nu_x)_r < 0$.

Proof. (i) Every $z_j = e_j - e_{\text{rep}(j)}$ has one entry $+1$ and one entry -1 in the same block, so $C z_j = 0$ and therefore $C d_x = 0$. The vector d_x is supported on $\{r\} \cup \mathcal{N}_x \cup \mathcal{S}$, so it vanishes on $\mathcal{W}_x \setminus \{r\}$, and d_s vanishes on $\mathcal{W}_s$ by definition.

(ii) The rows of $\bar{A} d$ are $A_E d_x$ (equality rows) and $A_I d_x - d_s$ (inequality rows). On the rows of A_{eff}, i.e. the equality rows and the inequality rows in $\mathcal{W}_s$ (where $d_s = 0$), $\bar{A} d$ equals $A_{\text{eff}} d_x = A_{\text{eff}} z_r - \sum_j \beta_j A_{\text{eff}} z_j$, which is zero by the assumption on β. On an inequality row $i' \in \mathcal{N}_s$, $(\bar{A} d)_{i'} = (A_I d_x)_{i'} - (d_s)_{i'} = 0$ by the definition of d_s.

(iii) Splitting $g^\top d = g_x^\top d_x + \sum_{i'} \pi_{i'} (d_s)_{i'}$ (recall $g_s = \pi_I$) and using (7.1),

$$g^\top d = (g_r - g_{\text{rep}(r)}) - \sum_{j \in \mathcal{N}_x} \beta_j \underbrace{(g_j - g_{\text{rep}(j)})}_{=0} + \sum_{i' \in \mathcal{N}_s} \underbrace{\pi_{i'}}_{=0} (d_s)_{i'} = (\nu_x)_r. \qquad \square$$

7.3.2 Released surplus

Theorem 7.3.2 (Zero-curvature direction, surplus)**.** Let the surplus of row $i \in \mathcal{W}_s$ have the multiplier $(\nu_s)_i = \pi_i < 0$, and suppose that its release is singular, $\omega_i = 1 - v_i^\top H_{\text{eff}}^{-1} v_i = 0$. Let $\beta = H_{\text{eff}}^{-1} v_i$, so that $v_i^\top \beta = 1$. Define $d = [d_x; d_s]$ by

$$d_x = \sum_{j \in \mathcal{N}_x} \beta_j \, z_j, \qquad (d_s)_{i'} = (A_I d_x)_{i'} \;\; (i' \in \mathcal{N}_s \cup \{i\}), \qquad (d_s)_{i'} = 0 \;\text{ otherwise.} \tag{7.3}$$

Then

(i) $Cd_x = 0$, d_x vanishes on $\mathcal{W}_x$, d_s vanishes on $\mathcal{W}_s \setminus \{i\}$, and $(d_s)_i = 1$;

(ii) $\bar{A} d = 0$;

(iii) $\nabla f(y)^\top d = \pi_i < 0$.

Proof. (i) As in Theorem 7.3.1, $Cd_x = 0$ and d_x vanishes on $\mathcal{W}_x$. Further, $d_x = Z_x \beta$, so $(d_s)_i = (A_I Z_x \beta)_i = (A_I)_{i,:} Z_x \, \beta = v_i^\top \beta = 1$, using $v_i = Z_x^\top (A_I)_{i,:}^\top$ from (6.1).
(ii) Let A'_{eff} be A_{eff} without row i; its rows are the equality rows and the inequality rows in $\mathcal{W}_s \setminus \{i\}$. By Theorem 6.3.1, $(A'_{\text{eff}} Z_x)^\top (A'_{\text{eff}} Z_x) = H_{\text{eff}} - v_i v_i^\top$, hence

$$\|A'_{\text{eff}} d_x\|^2 = \beta^\top (H_{\text{eff}} - v_i v_i^\top) \, \beta = \beta^\top v_i - (v_i^\top \beta)^2 = 1 - 1 = 0,$$

using $H_{\text{eff}} \beta = v_i$ and $v_i^\top \beta = 1$. So $A'_{\text{eff}} d_x = 0$, which means that $\bar{A} d$ vanishes on the rows of A'_{eff} (where $d_s = 0$). On the remaining inequality rows $i' \in \mathcal{N}_s \cup \{i\}$, $(\bar{A} d)_{i'} = (A_I d_x)_{i'} - (d_s)_{i'} = 0$ by construction.
(iii) Using (7.1), $g_x^\top d_x = \sum_j \beta_j (g_j - g_{\text{rep}(j)}) = 0$ and $\pi_{i'} = 0$ for $i' \in \mathcal{N}_s$, so only the component of surplus i remains: $g^\top d = \pi_i \, (d_s)_i = \pi_i$. □

Remark 7.3.3. For a "$\leq$" row with slack coefficient $\kappa_i = +1$ (Remark 5.3.3), the column part is scaled by $-\kappa_i$, $d_x = -\kappa_i \sum_j \beta_j z_j$, and the surplus part is $(d_s)_{i'} = -\kappa_{i'} (A_I d_x)_{i'}$. Then again $(d_s)_i = 1$ and $g^\top d = -\kappa_i \pi_i = (\nu_s)_i < 0$. The same surplus rule $(d_s)_{i'} = -\kappa_{i'} (A_I d_x)_{i'}$ replaces $(d_s)_{i'} = (A_I d_x)_{i'}$ in (7.2).

Corollary 7.3.4 (Linear decrease)**.** For the directions of Theorems 7.3.1 and 7.3.2,

$$f(y + \alpha d) = f(y) + \alpha \, \nabla f(y)^\top d, \qquad \pi(y + \alpha d) = \pi(y) \qquad \text{for all } \alpha.$$

Proof. Expanding the quadratic f, $f(y + \alpha d) = f(y) + \alpha \nabla f(y)^\top d + \frac{\alpha^2}{2\delta} \|\bar{A} d\|^2$, and $\bar{A} d = 0$. The dual estimate $\pi(y) = \hat{\pi} + \frac{1}{\delta}(b - \bar{A} y)$ depends on y only through $\bar{A} y$, which does not change. □

Remark 7.3.5 (Fully masked columns)**.** If all nonzero coefficients of $u_r = a_r - a_{\text{rep}(r)}$ lie in rows with free surpluses, then $A_{\text{eff}} z_r = 0$, so $\gamma_r = 0$ and $\gamma_{rr} = 0$. Theorem 7.3.1 then applies with $\beta = 0$: the direction is z_r together with the surplus components that absorb $A_I z_r$.

7.4 The Zero-Curvature Step Removes the Singularity

Along d the objective decreases linearly (Corollary 7.3.4), so the method moves as far as the bounds allow. In this section ρ denotes the released index: the column r of Theorem 7.3.1 or the surplus i of Theorem 7.3.2.

Lemma 7.4.1 (Finite step)**.** The directions of Theorems 7.3.1 and 7.3.2 have at least one negative component. Hence the ratio test

$$\alpha^\star = \min_{\ell:\, d_\ell<0} \; -\frac{y_\ell}{d_\ell}$$

is finite, and it is attained at a **blocking index** q in $\mathcal{S} \cup \mathcal{N}_x \cup \mathcal{N}_s$.

Proof. Suppose $d \geq 0$. By part (i) of Theorem 7.3.1 and part (i) of Theorem 7.3.2, every block sum of d_x is zero ($Cd_x = 0$). A nonnegative vector whose block sums are zero is zero, so $d_x = 0$. Then $\bar{A}d = [\, A_E d_x;\; A_I d_x - d_s \,] = [\, 0;\; -d_s \,]$, and $\bar{A}d = 0$ (parts (ii) and (ii), respectively) forces $d_s = 0$. But $d = 0$ contradicts $\nabla f(y)^\top d < 0$ (parts (iii) and (iii)). So some component of d is negative, and the minimum is taken over a nonempty set. The blocking index cannot lie in $\mathcal{W} \setminus \{\rho\}$, where d vanishes, and it is not the released index ρ, whose component of d is positive ($d_r = 1$ or $(d_s)_i = 1$). Hence $q \in \mathcal{S} \cup \mathcal{N}_x \cup \mathcal{N}_s$. □

After the step, the blocking index q is added to the working set by an expansion (Section 6.2), and the release is attempted again. The following theorem shows that this attempt cannot fail for the same reason.

Theorem 7.4.2 (Removal of the dependency)**.** Let ρ be the released index and q the blocking index of Lemma 7.4.1. Then the reduced Hessian of the working set $\mathcal{W}' = (\mathcal{W} \setminus \{\rho\}) \cup \{q\}$ is positive definite. In other words, after the zero-curvature step and the expansion by q, the retried release of ρ succeeds.

Proof. For any working set with nullspace basis Z, the reduced Hessian $Z^\top \bar{A}^\top \bar{A} Z$ is positive definite if and only if $\bar{A}Z\eta \neq 0$ for all $\eta \neq 0$, i.e. if and only if $\bar{A}$ is injective on the nullspace of the constraint matrix of the working set. We use this for three spaces:

$$\begin{aligned}
Y_0 &= \operatorname{Null}(D_{\mathcal{W}}) && \text{(before the release)},\\
Y_1 &= \operatorname{Null}(D_{\mathcal{W}\setminus\{\rho\}}) && \text{(after the release)},\\
Y_2 &= Y_1 \cap \{y : y_q = 0\} = \operatorname{Null}(D_{\mathcal{W}'}) && \text{(after the release and the expansion by } q).
\end{aligned}$$

Step 1: $\ker \bar{A} \cap Y_1 = \operatorname{span}\{d\}$. Before the release the reduced Hessian was positive definite, so $\ker \bar{A} \cap Y_0 = \{0\}$. Removing one constraint from the working set adds one dimension, $Y_0 \subset Y_1$ with $\dim Y_1 = \dim Y_0 + 1$. If $\ker \bar{A} \cap Y_1$ contained two linearly independent vectors, some nonzero combination of them would satisfy the one extra constraint ($y_\rho = 0$) and therefore lie in $Y_0 \cap \ker \bar{A} = \{0\}$, a contradiction. So $\dim(\ker \bar{A} \cap Y_1) \leq 1$. The direction d lies in Y_1 (part (i) of Theorem 7.3.1 and part (i) of Theorem 7.3.2) and in $\ker \bar{A}$ (parts (ii) and (ii), respectively), and $d \neq 0$, so $\ker \bar{A} \cap Y_1 = \operatorname{span}\{d\}$.

Step 2: $\ker \bar{A} \cap Y_2 = \{0\}$. By Step 1, every vector of $\ker \bar{A} \cap Y_2$ is a multiple of d, and its component q is zero. Since $d_q < 0$, the multiple is zero. Hence $\bar{A}$ is injective on $Y_2 = \operatorname{Null}(D_{\mathcal{W}'})$, and the reduced Hessian of $\mathcal{W}'$ is positive definite. □

The retried release starts from the same kind of state as the first attempt: a vanishing reduced gradient and a negative multiplier for ρ.

Proposition 7.4.3 (The reduced gradient stays zero)**.** Unless $\rho = r$ and $q = \operatorname{rep}(r)$ (the case of Remark 7.4.4), after the zero-curvature step and the expansion by q the reduced gradient for the working set $\mathcal{W} \cup \{q\}$ (with ρ still in it) vanishes, and the multiplier of ρ is unchanged. The only index of $\mathcal{W} \cup \{q\}$ whose variable is not at zero is ρ itself ($y_\rho = \alpha^\star$), which is released next.

Proof. By Corollary 7.3.4, π and hence the gradient $g = \hat{c} - \bar{A}^\top \pi$ do not change along the step. The expansion by q changes the nullspace basis as in Theorem 6.2.3: it removes a vector, or, if q is a representative S_k, replaces vectors z_j by $z_j - z_{S'_k}$ for the replacement S'_k. In all cases the new basis vectors are combinations of old ones, whose reduced gradient entries $g^\top z$ were zero; so the new reduced gradient is zero as well. The multiplier of ρ is computed from the unchanged gradient by Corollary 5.3.2; a new representative S'_k does not change μ_k, because $g_{S'_k} - g_{S_k} = g^\top z_{S'_k} = 0$. Finally, $d_\rho = 1$ and d vanishes on $\mathcal{W} \setminus \{\rho\}$, so $y_\rho = \alpha^\star$ and all other working-set variables stay zero. □

Remark 7.4.4 (The representative of the released column blocks)**.** In Theorem 7.3.1, the blocking index may be the representative $q = \mathrm{rep}(r)$ of the block k of r, for example when $\mathcal{N}_x^{(k)}$ is empty. Then r, which now has the weight $\alpha^\star \geq 0$, takes over as the new representative: r leaves $\mathcal{W}_x$, q enters $\mathcal{W}_x$, and for $j \in \mathcal{N}_x^{(k)}$ the vectors become $e_j - e_r = z_j - z_r$, as in case (ii) of Lemma 6.2.1 but without removing a row of $\mathcal{N}_x$. The resulting working set is $\mathcal{W}'$ of Theorem 7.4.2, so H_{eff} stays positive definite, and the release is complete.

Algorithm 7.1 summarizes the procedure. By Theorem 7.4.2, the second pass completes the release in exact arithmetic. With rounding, each further pass adds one index to the working set, so the number of passes is bounded by the number of free variables.

Algorithm 7.1: Release with Singularity Resolution

Input: subspace minimizer y, index ρ (column r or surplus i) with negative multiplier
loop
  Apply the reduction update (`rowadd` for a column, downdate for a surplus)
  if the pivot invariant holds **then**
    return ▷ release completed
  end if
  Undo by refactorization with ρ still in the working set
  Solve $H_{\text{eff}}\beta = \gamma_r$ (column) or $H_{\text{eff}}\beta = v_i$ (surplus)
  Form d by (7.2) or (7.3)
  Ratio test along d; $y \leftarrow y + \alpha^\star d$ ▷ π is unchanged
  if $\rho = r$ and $q = \mathrm{rep}(r)$ **then**
    r becomes the representative, q enters $\mathcal{W}_x$; **return** ▷ Remark 7.4.4
  end if
  Expand the working set by q ▷ Theorem 7.4.2: the next pass succeeds
end loop

If $A_{\text{eff}} z_r = 0$ (a fully masked column, Remark 7.3.5), the release is singular without any test, since $\gamma_{rr} = 0$. Our implementation then skips the update and the refactorization and goes directly to the zero-curvature step.

Remark 7.4.5 (Why not remove the largest coefficient?)**.** A simpler rule moves the index $j = \arg\max_{j' \in \mathcal{N}_x} |\beta_{j'}|$ of the largest coefficient of the linear dependence into the working set and retries. This also removes the dependency, but it ignores the value of x_j. If $x_j > 0$, the working-set constraint $x_j = 0$ is violated by the iterate, and its multiplier no longer certifies optimality: the method can stop with $x_j > 0$ and a positive multiplier. The zero-curvature step avoids this. The iterate stays feasible, every working-set variable other than the released one is exactly at its bound, the objective does not increase (and decreases strictly unless $\alpha^\star = 0$), and the index added to the working set is the one selected by a ratio test rather than by a heuristic. The same argument covers released columns and released surpluses.

Chapter 8

A Dense Factorization of the Effective Reduced Hessian

Chapter 6 expressed every change of the working set as a row addition, a row deletion or a rank-one modification of an $LDL^\top$ factorization, and realized these operations with a sparse factorization of fixed dimension $n_{\max}$. This is the natural choice for a matrix that is embedded in an index range of the size of the column capacity. The matrix that actually has to be factorized, however, is small and dense: H_{eff} never has more rows than there are linking rows, and almost all of its entries are nonzero.

This chapter develops a **dense factorization** that exploits these two properties. It stores only the active indices, in the order in which they entered $\mathcal{N}_x$, so that a new index is always **appended** as the last row of the factor. A removed index is deleted from the factor by a **rank-one update** of the rows behind it, and the factor is compacted at the same time. We derive these operations from the classical rank-one modification algorithm and show four properties that make the dense factor attractive for the active-set method: the new pivot of a released column is exactly its Schur complement, so the singularity test of Chapter 7 reduces to a single number; removals and updates never decrease a pivot; downdates fail exactly when the modified matrix is not positive definite; and the factor never contains a stored entry that does not belong to the current matrix, so that the periodic refactorization of Section 6.5 becomes unnecessary.

The two factorizations are interchangeable: the active-set method only requires the operations of Section 6.4, and both provide them. The chapter closes with a comparison of their costs; the measurements follow in Section 9.8.

8.1 Why a Dense Factor

Three observations from the previous chapters motivate a different representation of the factor.

The factorized matrix is small. By Corollary 5.2.5, as long as the reduced Hessian is positive definite,

$$|\mathcal{N}_x| \;\le\; \operatorname{rank}(A_{\text{eff}}) \;\le\; m. \tag{8.1}$$

The release of a dependent column (Proposition 7.1.1) adds one more index before it is detected and undone, so at most $m+1$ indices are ever active at the same time. The dimension of H_{eff} is therefore bounded by the number of linking rows and does not depend on the column capacity $n_{\max}$ or on the number of generated columns. The fixed-length representation of Section 6.1, in contrast, gives the factor the dimension $n_{\max}$, which is typically one to two orders of magnitude larger than m.

The factorized matrix is dense. The entry $(H_{\text{eff}})_{jj'} = u_j^\top M_{\text{eff}} u_{j'}$ of (5.12) is nonzero as soon as the differences $u_j = a_j - a_{\text{rep}(j)}$ and $u_{j'}$ share a row of A_{eff}. Two columns of the same block share their representative, so

$$(H_{\text{eff}})_{jj'} = a_j^\top M_{\text{eff}} a_{j'} - a_j^\top M_{\text{eff}} a_{S_k} - a_{S_k}^\top M_{\text{eff}} a_{j'} + a_{S_k}^\top M_{\text{eff}} a_{S_k} \qquad (\psi(j) = \psi(j') = k),$$

which vanishes only by cancellation. Columns of different blocks compete for the same linking rows, which is the purpose of these rows, and therefore overlap as well. The measurements of Section 9.7 confirm that the factor of H_{eff} is practically dense, in agreement with Remark 6.5.4. A sparse factorization of a dense matrix stores the same numbers as a dense one, together with row indices and column pointers, and its operations follow these index structures instead of streaming through contiguous memory.

Sparse updates leave structural zeros. As shown in Section 6.5, row deletions and rank-one modifications of a sparse factor keep the positions of the removed entries in its sparsity pattern. Every later operation pays for them until the factor is rebuilt, and the refactorization interval has to balance the two costs.

A dense factor of the active block only has none of these drawbacks, provided that it can be updated as cheaply as the sparse one. The rest of this chapter shows that it can.

8.2 Compact Representation

Throughout this chapter, $t = |\mathcal{N}_x|$ is the number of active indices, and $\mathcal{N}_x = \{j_1, \dots, j_t\}$ lists them in the order in which they entered $\mathcal{N}_x$: j_1 has been active the longest and j_t is the most recent. The **position** of an active index is $\operatorname{pos}(j_\ell) = \ell$.

Definition 8.2.1 (Compact representation)**.** The **compact effective reduced Hessian** is the $t \times t$ matrix

$$\hat{H}_{\ell\ell'} = (H_{\text{eff}})_{j_\ell j_{\ell'}} \qquad (\ell, \ell' = 1, \dots, t),$$

i.e. H_{eff} with its rows and columns arranged in activation order. The **dense factor** is its $LDL^\top$ factorization without pivoting,

$$\hat{H} = LDL^\top, \qquad L \in \mathbb{R}^{t\times t} \text{ unit lower triangular}, \qquad D = \operatorname{diag}(d_1, \dots, d_t),$$

stored as a dense array together with the list $j_1, \dots, j_t$ and the position map pos.

The factor exists and is unique whenever H_{eff} is positive definite, and then all pivots d_ℓ are positive [15]. The only object of size $n_{\max}$ is the integer array pos, with the value 0 for inactive indices (and, in our implementation, the right-hand side and solution buffers, which are indexed by column); the factor occupies at most $(m+1)^2$ floating-point entries by (8.1). The compact representation solves the same systems as the embedded one:

Lemma 8.2.2 (Solves in compact form)**.** Let $\zeta \in \mathbb{R}^{n_{\max}}$ vanish outside $\mathcal{N}_x$, and let $\hat{\zeta}_\ell = \zeta_{j_\ell}$. If $\hat{\eta}$ solves $\hat{H}\hat{\eta} = \hat{\zeta}$, then η with $\eta_{j_\ell} = \hat{\eta}_\ell$ and $\eta_j = 0$ for $j \notin \mathcal{N}_x$ solves $H_{\text{eff}}^{\text{emb}}\eta = \zeta$.

Proof. By Lemma 6.1.2, the solution of $H_{\text{eff}}^{\text{emb}}\eta = \zeta$ vanishes outside $\mathcal{N}_x$, and its restriction to $\mathcal{N}_x$ solves $H_{\text{eff}}\eta_x = \zeta_{\mathcal{N}_x}$. The matrix $\hat{H}$ is H_{eff} with rows and columns permuted by the same permutation as the entries of $\hat{\zeta}$ and $\hat{\eta}$, so $\hat{H}\hat{\eta} = \hat{\zeta}$ is this system written in activation order. □

A solve therefore consists of a gather of ζ into activation order, a forward substitution with L, a division by the pivots, a backward substitution with $L^\top$ and a scatter of the result, at the cost of $2t^2 + O(t)$ arithmetic operations, independent of $n_{\max}$.

The activation order is not a fill-reducing ordering, and it need not be: the matrix is dense, so every ordering of its rows produces a dense factor. It is chosen because it turns a row addition into the cheapest possible operation, an append.

change of the working set	change of $\hat{H}$	dense operation
(i) column leaves $\mathcal{N}_x$	position $\text{pos}(q)$ removed	delete (Section 8.5)
(ii) representative swap	rows of $\mathcal{N}_x^{(k)}$ rewritten	deletes + appends
(iii) surplus becomes active	$+\hat{v}_i\hat{v}_i^\top$	rank-one update (Section 8.4)
(iv) column enters $\mathcal{N}_x$	new last row and column	append (Section 8.3)
(v) surplus becomes free	$-\hat{v}_i\hat{v}_i^\top$	rank-one downdate (Section 8.4)

Here $\hat{v}_i$ is the vector v_i of (6.1) in activation order. A representative swap is carried out as in Section 6.4: the replacement S'_k is deleted, and every row of $\mathcal{N}_x^{(k)} \setminus \{S'_k\}$ is deleted and appended again with its new vector $z'_j = z_j - z_{S'_k}$. These rows thereby move to the end of the activation order.

8.3 Appending an Index

When a column r is released (case (iv) of Theorem 6.3.1), H_{eff} is bordered by the vector γ_r and the scalar γ_{rr} of (6.2). In the compact representation the new index takes position $t+1$, so the new matrix is

$$\hat{H}' = \begin{bmatrix} \hat{H} & \hat{\gamma} \\ \hat{\gamma}^\top & \gamma_{rr} \end{bmatrix}, \qquad \hat{\gamma}_\ell = (\gamma_r)_{j_\ell}.$$

Theorem 8.3.1 (Append)**.** Let $\hat{H} = LDL^\top$ be positive definite. Then

$$\hat{H}' = \begin{bmatrix} L & 0 \\ l^\top & 1 \end{bmatrix} \begin{bmatrix} D & 0 \\ 0 & d_{t+1} \end{bmatrix} \begin{bmatrix} L & 0 \\ l^\top & 1 \end{bmatrix}^\top \qquad \text{with} \qquad Lw = \hat{\gamma}, \quad l = D^{-1}w, \quad d_{t+1} = \gamma_{rr} - w^\top D^{-1} w.$$

The factor of $\hat{H}$ is not changed, and the new row costs one forward substitution, $t^2 + O(t)$ operations.

Proof. Multiplying out the right-hand side gives the blocks

$$LDL^\top = \hat{H}, \qquad LDl = Lw = \hat{\gamma}, \qquad l^\top Dl + d_{t+1} = w^\top D^{-1} w + d_{t+1} = \gamma_{rr},$$

which are the blocks of $\hat{H}'$. The first factor is unit lower triangular and the middle one diagonal, so this is the $LDL^\top$ factorization of $\hat{H}'$ whenever $d_{t+1} \neq 0$, by uniqueness. The vector w is obtained from $Lw = \hat{\gamma}$ by forward substitution, and l and d_{t+1} in $O(t)$ further operations. □

The new pivot has a simple meaning. It is exactly the quantity that decides whether the release is singular.

Corollary 8.3.2 (The new pivot is the Schur complement)**.** The new pivot of Theorem 8.3.1 is the Schur complement of (6.2),

$$d_{t+1} = \sigma_r = \gamma_{rr} - \gamma_r^\top H_{\text{eff}}^{-1} \gamma_r = \left\| (I - \Pi_{\text{eff}})\, A_{\text{eff}} z_r \right\|^2,$$

where Π_{eff} is the orthogonal projector onto the range of $A_{\text{eff}} Z_x$. In particular $d_{t+1} \geq 0$, and $d_{t+1} = 0$ if and only if the release of r is singular.

Proof. From $Lw = \hat{\gamma}$ and $\hat{H} = LDL^\top$,

$$w^\top D^{-1} w = \hat{\gamma}^\top L^{-\top} D^{-1} L^{-1} \hat{\gamma} = \hat{\gamma}^\top \hat{H}^{-1} \hat{\gamma} = \gamma_r^\top H_{\text{eff}}^{-1} \gamma_r,$$

since the permutation to activation order does not change this quadratic form. Hence $d_{t+1} = \sigma_r$. With $U = A_{\text{eff}} Z_x$ and $u = A_{\text{eff}} z_r$ we have $H_{\text{eff}} = U^\top U$, $\gamma_r = U^\top u$ and $\gamma_{rr} = u^\top u$, so

$$\sigma_r = u^\top u - u^\top U (U^\top U)^{-1} U^\top u = u^\top (I - \Pi_{\text{eff}})\, u = \left\| (I - \Pi_{\text{eff}})\, u \right\|^2,$$

because $\Pi_{\text{eff}} = U(U^\top U)^{-1} U^\top$ is symmetric and idempotent. The last statement is part (i) of Proposition 7.1.1. □

8.3.1 What the singularity test measures

Corollary 8.3.2 has a consequence for the singularity detection of Section 7.2. There, a release is accepted if every pivot satisfies the pivot invariant $D_{jj} > \varepsilon_{\text{piv}} \|u_j\|^2$, and this test was justified only heuristically: in the sparse factorization a row addition changes the new pivot and the pivots of all later indices, and only their product is known to equal σ_r (Proposition 6.4.1). In the dense factorization an append changes no existing pivot, so the test decides on the new pivot alone, and this pivot is σ_r exactly. We can therefore say precisely what the test measures. It turns out to be an angle.

For a nonzero vector u and a subspace $\mathcal{S}$, the angle $\theta \in [0, \pi/2]$ between them is the smallest angle between u and a vector of $\mathcal{S}$. It is attained at the orthogonal projection of u onto $\mathcal{S}$, so $\cos\theta = \|\Pi_{\mathcal{S}} u\| / \|u\|$.

Lemma 8.3.3 (The scaled new pivot is an angle)**.** Let $A_{\text{eff}} z_r \neq 0$, let θ_r be the angle between $A_{\text{eff}} z_r$ and the range of $A_{\text{eff}} Z_x$, and let

$$\rho_r = \frac{\|M_{\text{eff}} u_r\|^2}{\|u_r\|^2} \in (0, 1]$$

be the fraction of $\|u_r\|^2$ that lies in the unmasked rows. Then the new pivot of Theorem 8.3.1, scaled by the pivot scale of Definition 7.2.1, is

$$\frac{d_{t+1}}{\|u_r\|^2} \;=\; \rho_r \sin^2\theta_r \;\leq\; \sin^2\theta_r,$$

with equality if and only if $\theta_r = 0$ or u_r has no nonzero entry in a masked row.

Proof. Let $u = A_{\text{eff}} z_r$ and $\Pi = \Pi_{\text{eff}}$. The vectors Πu and $(I - \Pi)u$ are orthogonal, so $\|u\|^2 = \|\Pi u\|^2 + \|(I - \Pi)u\|^2$, and with Corollary 8.3.2

$$\sin^2 \theta_r = 1 - \frac{\|\Pi u\|^2}{\|u\|^2} = \frac{\|(I - \Pi)u\|^2}{\|u\|^2} = \frac{d_{t+1}}{\|u\|^2}.$$

The vector u consists of the unmasked rows of $u_r = a_r - a_{\text{rep}(r)}$, so $\|u\|^2 = \|M_{\text{eff}} u_r\|^2 = \rho_r \|u_r\|^2$. Multiplying by ρ_r gives the identity. Since M_{eff} is diagonal with entries 0 and 1, $\rho_r \le 1$, with equality exactly when every nonzero entry of u_r lies in an unmasked row. □

Remark 8.3.4 (Singularity detection with the dense factor)**.** Lemma 8.3.3 does not introduce a new test. The implementation runs the same pivot check for both factorizations, and with the dense factor it reads

$$\rho_r \sin^2 \theta_r \;>\; \varepsilon_{\text{piv}}.$$

If no row of u_r is masked, a released column is therefore rejected exactly when its effective direction $A_{\text{eff}} z_r$ makes an angle of less than about $\sqrt{\varepsilon_{\text{piv}}} = 10^{-5}$ with the span of the active directions. The tolerance ε_{piv} thus has a geometric meaning that does not depend on the scaling of the columns. Masked rows make the test stricter by the factor ρ_r. In the extreme case $A_{\text{eff}} z_r = 0$ (a fully masked column, Remark 7.3.5) the angle is undefined, but $d_{t+1} = 0$ and the test fails, which is why the pivot scale uses all rows.

In the sparse factorization the same quantity σ_r is spread over the new pivot and the downdated pivots of the later indices, and no single pivot equals $\rho_r \sin^2 \theta_r$. A dependent column can moreover be removed from the dense factor by deleting the last position, which by Proposition 8.5.2 below costs no arithmetic at all. (Our implementation undoes it by a refactorization for both factorizations, which happens only a handful of times per run.)

8.4 Rank-One Modifications

The surplus cases (iii) and (v) change H_{eff} by $\pm v_i v_i^\top$, and we will see in Section 8.5 that the deletion of an index requires a rank-one update as well. All three are instances of the following problem: given the factor $\hat{H} = LDL^\top$, a vector $v \in \mathbb{R}^t$ and a scalar φ, compute the factor of $\hat{H} + \varphi\, vv^\top$. We use the method C1 of Gill, Golub, Murray and Saunders [13], and derive it here in a form that also yields the properties we need.

Theorem 8.4.1 (Rank-one modification)**.** Let $\hat{H} = LDL^\top$ with $d_1, \dots, d_t > 0$, let $v \in \mathbb{R}^t$ and $\varphi \in \mathbb{R}$, and suppose that $\hat{H} + \varphi\, vv^\top$ is positive definite. Define $\varphi_1 = \varphi$, $v^{(0)} = v$, and for $j = 1, \dots, t$

$$\chi_j = v_j^{(j-1)}, \qquad \bar{d}_j = d_j + \varphi_j \chi_j^2, \qquad \beta_j = \frac{\varphi_j \chi_j}{\bar{d}_j}, \qquad \varphi_{j+1} = \frac{d_j}{\bar{d}_j}\, \varphi_j, \qquad v^{(j)} = v^{(j-1)} - \chi_j\, Le_j. \tag{8.2}$$

Then $\hat{H} + \varphi\, vv^\top = \bar{L}\bar{D}\bar{L}^\top$ with $\bar{D} = \operatorname{diag}(\bar{d}_1, \dots, \bar{d}_t)$ and the unit lower triangular matrix $\bar{L}$ whose columns are

$$\bar{L}e_j = Le_j + \beta_j\, v^{(j)} \qquad (j = 1, \dots, t).$$

The vector $v^{(j)}$ vanishes in its first j entries, so column j of $\bar{L}$ differs from column j of L only below the diagonal.

Proof. Step 1: reduction to a diagonal matrix. Let $\tilde{v} = L^{-1} v$. Then

$$\hat{H} + \varphi\, vv^\top = L\,(D + \varphi\, \tilde{v}\tilde{v}^\top)\, L^\top.$$

If $D + \varphi\tilde{v}\tilde{v}^\top = \tilde{L}\bar{D}\tilde{L}^\top$, then $\hat{H} + \varphi vv^\top = (L\tilde{L})\,\bar{D}\,(L\tilde{L})^\top$, and $\bar{L} = L\tilde{L}$ is unit lower triangular as a product of two such matrices. It therefore suffices to factorize the diagonal matrix plus a rank-one term.

Step 2: factor of $D + \varphi\tilde{v}\tilde{v}^\top$. We show by induction over t that

$$\bar{d}_j = d_j + \varphi_j \tilde{v}_j^2, \qquad \tilde{L}_{rj} = \beta_j \tilde{v}_r \;\; (r > j), \qquad \beta_j = \varphi_j \tilde{v}_j / \bar{d}_j, \qquad \varphi_{j+1} = \varphi_j d_j / \bar{d}_j. \tag{8.3}$$

Split off the first row and column:

$$D + \varphi\tilde{v}\tilde{v}^\top = \begin{bmatrix} d_1 + \varphi\tilde{v}_1^2 & \varphi\tilde{v}_1\tilde{v}_2^\top \\ \varphi\tilde{v}_1\tilde{v}_2 & D_2 + \varphi\tilde{v}_2\tilde{v}_2^\top \end{bmatrix},$$

with $\tilde{v} = [\tilde{v}_1; \tilde{v}_2]$ and $D = \operatorname{diag}(d_1, D_2)$. One step of symmetric Gaussian elimination gives the pivot $\bar{d}_1 = d_1 + \varphi\tilde{v}_1^2$, which is positive because the matrix is positive definite, the first column $\varphi\tilde{v}_1\tilde{v}_2/\bar{d}_1 = \beta_1\tilde{v}_2$ of $\tilde{L}$ below the diagonal, and the Schur complement

$$D_2 + \varphi\tilde{v}_2\tilde{v}_2^\top - \frac{\varphi^2\tilde{v}_1^2}{\bar{d}_1}\tilde{v}_2\tilde{v}_2^\top = D_2 + \varphi\Big(1 - \frac{\varphi\tilde{v}_1^2}{\bar{d}_1}\Big)\tilde{v}_2\tilde{v}_2^\top = D_2 + \underbrace{\varphi\,\frac{d_1}{\bar{d}_1}}_{=\varphi_2}\,\tilde{v}_2\tilde{v}_2^\top.$$

The Schur complement of a positive definite matrix is positive definite and has the same form as the original matrix, with D_2, $\tilde{v}_2$ and φ_2, so the induction hypothesis gives (8.3) for the remaining indices.

Step 3: the columns of $\bar{L}$. By Step 2, column j of $\tilde{L}$ is $e_j + \beta_j \sum_{r>j} \tilde{v}_r e_r$, so

$$\bar{L}e_j = L\tilde{L}e_j = Le_j + \beta_j \sum_{r>j} \tilde{v}_r\, Le_r.$$

Since $L\tilde{v} = v$, the sum equals $v - \sum_{i\le j} \tilde{v}_i\, Le_i$. We show that this is $v^{(j)}$ and that $\chi_j = \tilde{v}_j$. Indeed, $v^{(j)} = v - \sum_{i\le j} \chi_i\, Le_i$ by its recursion. If $\chi_i = \tilde{v}_i$ for $i < j$, then $v^{(j-1)} = \sum_{r\ge j} \tilde{v}_r\, Le_r$, whose j-th entry is $\tilde{v}_j$ because L is unit lower triangular; hence $\chi_j = \tilde{v}_j$. By induction, $\chi_j = \tilde{v}_j$ for all j, and $\bar{L}e_j = Le_j + \beta_j v^{(j)}$. The same representation shows that $v^{(j)} = \sum_{r>j} \tilde{v}_r Le_r$ vanishes in its first j entries. Inserting $\chi_j = \tilde{v}_j$ into (8.3) gives (8.2). □

The recursion (8.2) computes $\tilde{v} = L^{-1}v$ by forward substitution and builds the new factor in the same pass, column by column. Algorithm 8.1 states it in the form in which it is implemented: the vector $v^{(j)}$ overwrites v, and the new column overwrites the old one.

Algorithm 8.1: Rank-One Modification $LDL^\top + \varphi\, vv^\top$ (Method C1)

Input: L, d of $\hat{H} = LDL^\top$, vector v, scalar φ
$j_0 \leftarrow$ first index with $v_{j_0} \neq 0$ ▷ columns before j_0 are unchanged
for $j = j_0, \dots, t$ **do**
  $\chi \leftarrow v_j$; **if** $\chi = 0$ **then continue** ▷ column j and φ unchanged
  $\bar{d} \leftarrow d_j + \varphi\chi^2$
  if $\bar{d} \le 0$ **then return** *not positive definite*
  end if
  $\beta \leftarrow \varphi\chi/\bar{d}$; $\varphi \leftarrow \varphi\, d_j/\bar{d}$; $d_j \leftarrow \bar{d}$
  for $r = j+1, \dots, t$ **do**
    $v_r \leftarrow v_r - \chi\, L_{rj}$; $L_{rj} \leftarrow L_{rj} + \beta\, v_r$
  end for
end for

If $\chi_j = 0$, then $\bar{d}_j = d_j$, $\beta_j = 0$ and $\varphi_{j+1} = \varphi_j$, and the column is unchanged; this justifies the skip in Algorithm 8.1. The cost is $2(t - j_0)^2 + O(t)$ operations, where j_0 is the position of the first nonzero of v. The following properties of (8.2) are used below.

Proposition 8.4.2 (Pivots of a rank-one modification)**.** In the situation of Theorem 8.4.1:

(i) If $\varphi > 0$, then $\varphi_j > 0$ and $\bar{d}_j \geq d_j$ for all j: an update never decreases a pivot.

(ii) The pivots change by the factor

$$\prod_{j=1}^{t} \frac{\bar{d}_j}{d_j} = 1 + \varphi\, v^\top \hat{H}^{-1} v.$$

(iii) If $\varphi < 0$ and the recursion (8.2) is run without the assumption that $\hat{H} + \varphi vv^\top$ is positive definite, then all $\bar{d}_j$ are positive if and only if $\hat{H} + \varphi vv^\top$ is positive definite, i.e. if and only if $1 + \varphi\, v^\top \hat{H}^{-1} v > 0$.

Proof. (i) By induction: if $\varphi_j > 0$, then $\bar{d}_j = d_j + \varphi_j \chi_j^2 \geq d_j > 0$ and $\varphi_{j+1} = \varphi_j d_j / \bar{d}_j > 0$.
(ii) The determinant of an $LDL^\top$ factorization is the product of its pivots, so

$$\prod_j \frac{\bar{d}_j}{d_j} = \frac{\det(\hat{H} + \varphi vv^\top)}{\det \hat{H}} = 1 + \varphi\, v^\top \hat{H}^{-1} v$$

by the matrix determinant lemma (Section 6.4).
(iii) The recursion performs symmetric Gaussian elimination without pivoting on $D + \varphi \tilde{v}\tilde{v}^\top$ (Step 2 of the proof of Theorem 8.4.1). Elimination without pivoting produces positive pivots $\bar{d}_1, \ldots, \bar{d}_t$ if and only if all leading principal minors are positive, i.e. if and only if the matrix is positive definite, and $D + \varphi \tilde{v}\tilde{v}^\top$ is positive definite if and only if $\hat{H} + \varphi vv^\top = L(D + \varphi \tilde{v}\tilde{v}^\top)L^\top$ is. Since $\hat{H}$ is positive definite and $\hat{H} + \varphi vv^\top$ differs from it by a rank-one term, the latter is positive definite if and only if its determinant is positive (its smallest eigenvalue is the only one that can become nonpositive, by interlacing), i.e. by (ii) if and only if $1 + \varphi v^\top \hat{H}^{-1} v > 0$. □

For the two surplus cases, $\varphi = \pm 1$ and $v = \hat{v}_i$. An activated surplus ($\varphi = 1$) can never fail and only increases pivots. A released surplus ($\varphi = -1$) produces positive pivots exactly when $\omega_i = 1 - v_i^\top H_{\text{eff}}^{-1} v_i > 0$, i.e. exactly when its release is not singular (part (ii) of Proposition 7.1.1); the product of the pivot ratios is ω_i. Algorithm 8.1 stops as soon as a pivot is not positive, and the method then proceeds as in Algorithm 7.1.

Remark 8.4.3 (Numerical behaviour)**.** For $\varphi > 0$ no cancellation occurs in $\bar{d}_j = d_j + \varphi_j \chi_j^2$, and method C1 is numerically stable [13]. For $\varphi < 0$ the subtraction can cancel, and the error grows when ω_i is small, i.e. when the downdated matrix is nearly singular. This is precisely the situation that the pivot invariant of Definition 7.2.1 detects, and it is then resolved by the zero-curvature step of Chapter 7. Away from it, the factor stays accurate over long runs; Section 9.8 measures its residual.

8.5 Removing an Index

When the index $q \in \mathcal{N}_x$ leaves the nullspace set (case (i) of Theorem 6.2.4), its row and column are removed from $\hat{H}$. Let $\ell = \text{pos}(q)$ and partition the positions into the groups $1 = \{1, \ldots, \ell - 1\}$, $2 = \{\ell\}$ and $3 = \{\ell + 1, \ldots, t\}$:

$$L = \begin{bmatrix} L_{11} & & \\ l_{21}^\top & 1 & \\ L_{31} & l_{32} & L_{33} \end{bmatrix}, \qquad D = \text{diag}(D_1, d_\ell, D_3).$$

Theorem 8.5.1 (Delete)**.** Removing row and column ℓ from $\hat{H} = LDL^\top$ gives the matrix with the factor

$$\begin{bmatrix} L_{11} & 0 \\ L_{31} & \bar{L}_{33} \end{bmatrix}, \qquad \operatorname{diag}(D_1, \bar{D}_3), \qquad \text{where} \quad \bar{L}_{33}\bar{D}_3\bar{L}_{33}^\top = L_{33}D_3L_{33}^\top + d_\ell\, l_{32}l_{32}^\top.$$

The trailing factor is a rank-one update with $\varphi = d_\ell > 0$, computed by Algorithm 8.1. The blocks L_{11}, D_1 and L_{31} are kept.

Proof. Multiplying out $LDL^\top$ block by block gives, for the blocks that remain after the removal,

$$\hat{H}_{11} = L_{11}D_1L_{11}^\top, \qquad \hat{H}_{31} = L_{31}D_1L_{11}^\top, \qquad \hat{H}_{33} = L_{31}D_1L_{31}^\top + d_\ell\, l_{32}l_{32}^\top + L_{33}D_3L_{33}^\top.$$

The proposed factor has the blocks $L_{11}D_1L_{11}^\top = \hat{H}_{11}$, $L_{31}D_1L_{11}^\top = \hat{H}_{31}$ and

$$L_{31}D_1L_{31}^\top + \bar{L}_{33}\bar{D}_3\bar{L}_{33}^\top = L_{31}D_1L_{31}^\top + L_{33}D_3L_{33}^\top + d_\ell\, l_{32}l_{32}^\top = \hat{H}_{33}.$$

It is unit lower triangular with a diagonal middle factor, so by uniqueness it is the $LDL^\top$ factorization of the reduced matrix. The trailing matrix $L_{33}D_3L_{33}^\top + d_\ell l_{32}l_{32}^\top$ is positive definite because $d_\ell > 0$, so Theorem 8.4.1 applies with $\varphi = d_\ell$. □

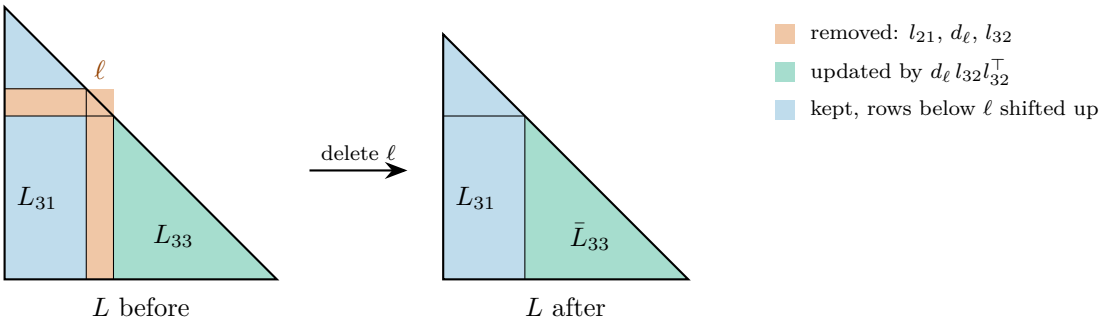


Figure 8.1: Deleting position ℓ from the dense factor (Theorem 8.5.1). The leading columns are kept, the trailing triangle receives a rank-one update with the removed column l_{32} and the positive weight d_ℓ, and all positions behind ℓ move up by one, so that the factor stays contiguous.

In an implementation, the update of the trailing block and the compaction are one pass: Algorithm 8.1 reads column $\ell + s$ of the old trailing block and writes the updated column to position $\ell + s - 1$, which has been read completely in the previous step (Fig. 8.1). The rows $\ell + 1, \ldots, t$ of the leading columns are moved up by one position, and the activation list and the position map are updated.

Proposition 8.5.2 (Cost of a deletion)**.** Deleting position ℓ costs $2(t-\ell)^2 + O(t)$ arithmetic operations for the rank-one update and $O(\ell\,(t-\ell))$ memory moves for the compaction. Deleting the most recent index ($\ell = t$) costs no arithmetic; deleting the oldest one ($\ell = 1$) costs as much as a full rank-one update.

Proof. The trailing block has $t - \ell$ positions, and Algorithm 8.1 on it costs $2(t-\ell)^2 + O(t-\ell)$ operations. The block L_{31} has $t - \ell$ rows and $\ell - 1$ columns, all of which are moved. □

The cost of a deletion therefore depends on how long the removed index has been active. In the active-set method most removed indices have entered $\mathcal{N}_x$ recently: a released column moves away from its bound (Lemma 6.3.2), and the columns that block are typically those with small weights, which have been released last. The measurements of Section 9.8 show that a deletion touches between a quarter and a third of the factor on average.

Corollary 8.5.3 (Expansions preserve the pivot invariant)**.** An expansion of type (i) or (iii) of Lemma 6.2.1 does not decrease any pivot of an index that stays in $\mathcal{N}_x$. In particular, if the pivot invariant of Definition 7.2.1 holds before such an expansion, it holds after it.

Proof. In case (i), the pivots D_1 are kept and $\bar{D}_3$ results from a rank-one update with $\varphi = d_\ell > 0$, so $\bar{D}_3 \geq D_3$ entrywise by part (i) of Proposition 8.4.2; each pivot moves with its index to the position one up. In case (iii), the update $+\hat{v}_i\hat{v}_i^\top$ has $\varphi = 1 > 0$. The pivot scales $\|u_j\|^2$ of the remaining indices do not change, since their representatives do not change. □

In exact arithmetic the same holds for the sparse factorization, whose row deletion is the same rank-one update (Section 6.4). A representative swap, the expansion of type (ii), is not covered: it keeps H_{eff} positive definite (Proposition 6.2.5), but its re-appended rows receive new pivots and new pivot scales $\|u_j - u_{S'_k}\|^2$, and their scaled pivots can be arbitrarily smaller than before. Our implementation therefore checks the pivot invariant after every expansion as well, and refactorizes if it is violated.

8.6 Structural Zeros and Refactorization

The dense factor stores exactly the $t(t-1)/2$ entries below the diagonal of the $t \times t$ array, the t pivots, and the activation list. This is all that it ever stores:

Proposition 8.6.1 (No structural zeros)**.** After every sequence of appends, deletions and rank-one modifications, the leading $t \times t$ block of the stored array is the factor L of the current matrix $\hat{H}$, and no stored entry belongs to a matrix that is no longer current.

Proof. By induction over the operations. Theorem 8.3.1 writes the new last row and pivot and keeps the leading block. Theorem 8.4.1 overwrites the columns from j_0 on and keeps the others, without changing the dimension. Theorem 8.5.1 keeps the leading columns, overwrites the trailing triangle with $\bar{L}_{33}$, and moves everything behind position ℓ up by one, so that the result again occupies exactly the leading $(t-1) \times (t-1)$ block. Entries outside the leading block are never read, and the next append overwrites the entries of its row before using them. □

The cost of every operation therefore depends only on the current t, and the factor never has to be rebuilt to restore its efficiency: there is nothing like the growth of the stored pattern of Section 6.5. A refactorization is only needed for numerical reasons, namely when a pivot violates the invariant of Definition 7.2.1 or a downdate fails (Algorithm 7.1). Such a refactorization appends the active indices one by one as in Definition 6.5.2, at the cost of about $t^3/3$ operations (Proposition 6.5.3); afterwards their activation order is the increasing index order. An optional periodic refactorization could be used to bound the accumulation of rounding errors, but Section 9.8 shows that it is not needed in practice.

Remark 8.6.2 (Reset)**.** Resetting the factor to the identity only clears the position map of the active indices, in $O(t)$. The stored entries need not be zeroed, because every append writes its whole row before it is read.

8.7 Comparison with the Sparse Factorization

Table 8.1 compares the operations of the two factorizations. For the sparse factorization, the costs are those of a factor without structural zeros, i.e. directly after a refactorization; between refactorizations they grow with the stored pattern.

operation	sparse factor of $H_{\text{eff}}^{\text{emb}}$ (Chapter 6)	dense factor of $\hat{H}$ (this chapter)
dimension	$n_{\max}$, identity outside $\mathcal{N}_x$	$t = \lvert\mathcal{N}_x\rvert \le m$
release of a column, (iv)	row addition at index r: solve, new column, downdate of the later indices	append: one forward substitution, t^2
removal of a column, (i)	row deletion: rank-one update of the later indices	delete: rank-one update of the later positions, $2(t-\ell)^2$
surplus, (iii) and (v)	rank-one update or downdate along the elimination tree	Algorithm 8.1 from the first nonzero, at most $2t^2$
solve	sparse solve on $\text{reach}_L(\mathcal{N}_x)$	two triangular solves, $2t^2$
new pivot of a release	$D'_{22} \ge \sigma_r$; later pivots change	$d_{t+1} = \sigma_r$; no other pivot changes
stored entries	grow with every update (Section 6.5)	exactly $t(t-1)/2$ (Proposition 8.6.1)
periodic refactorization	required, every τ updates	not required
factor memory (floating point)	proportional to the stored pattern	at most $(m+1)^2$

Table 8.1: The two factorizations of the effective reduced Hessian. Costs are arithmetic operations; ℓ is the position of the removed index.

For a dense H_{eff}, both factorizations perform the same arithmetic up to constant factors. The dense one does it on contiguous memory with simple loops and triangular solves, without index lookups, without an $n_{\max}$-dimensional embedding and without structural zeros. On the other hand, the dense factor costs $(m+1)^2$ floating-point numbers and $O(t^2)$ operations per update regardless of the sparsity of H_{eff}. A sparse factor remains preferable when H_{eff} is genuinely sparse, for instance when the blocks have disjoint sets of linking rows, and m is large. For the crew scheduling instances of Chapter 9, where H_{eff} is dense, the dense factorization is the better choice, as Section 9.8 shows.

Nothing in Chapters 4, 5 and 7 depends on the choice. The active-set method, the hot start of Theorem 6.6.1 and the singularity treatment of Algorithm 7.1 only use the operations of the table, so in exact arithmetic the two factorizations produce the same sequence of iterates, as long as their pivot tests decide the same way. (The individual pivots depend on the elimination order, so a release whose scaled pivot is close to ε_{piv} may be accepted by one and rejected by the other; an exactly singular release is rejected by both.) Algorithm 6.1 applies unchanged, with the refactorization in its last line triggered only by the pivot invariant.

Chapter 9

Computational Results

This chapter evaluates the hot-started nullspace active-set method on generated instances of the **airline crew scheduling problem**. Every instance is solved in two variants: with equality coverage rows only, and with 30% of the coverage rows relaxed to "$\geq$" inequalities. The active-set method is run with both factorizations of the effective reduced Hessian: the sparse factorization of Chapter 6 and the dense factorization of Chapter 8.

We first check **correctness**: every column generation method must reproduce the optimum of the full linear program, which is still small enough to be solved directly. We then study the **convergence behaviour** of regularized column generation, which consists of two clearly separated kinds of iterations, and the **computational time**, broken down by solver, by iteration and by operation inside the active-set method, up to instances with almost a thousand linking rows.

The last sections look inside the method. We follow the sizes of the nullspace sets over a run with inequality rows, measure the **contents of the sparse factorization** and the value of periodic refactorization, and compare the **two factorizations** operation by operation, in memory and in accuracy. The last section replaces the fixed stabilization parameter by the **adaptive** rule of Section 3.5.

9.1 Test Problem: Airline Crew Scheduling

9.1.1 The model

In the crew scheduling problem, K pilots have to be assigned schedules that together operate a set of m flights (Example 2.1.4). The schedules of a pilot are the source–sink paths in a **time–space network**: a directed acyclic graph $G = (V, E)$ whose nodes represent states of a pilot at given times, and whose edges represent the allowed transitions between them. The edge costs of pilot k form the vector $c^{(k)} \in \mathbb{R}^{|E|}$, and every edge operates a subset of the flights, recorded in the **task matrix** $A_{\text{task}} \in \{0,1\}^{m \times |E|}$. With $\xi_k \in \{0,1\}^{|E|}$ the edge vector of the path of pilot k and F the node–edge incidence matrix of G ($F_{\text{tail}(e),e} = 1$, $F_{\text{head}(e),e} = -1$), the linear relaxation reads

$$\min_{\xi_1,\dots,\xi_K} \quad \sum_{k=1}^{K} c^{(k)\top} \xi_k \tag{9.1a}$$

$$\text{s.t.} \quad \Big(\sum_{k=1}^{K} A_{\text{task}}\, \xi_k\Big)_i = 1 \qquad (i \in \mathcal{R}_E), \tag{9.1b}$$

$$\Big(\sum_{k=1}^{K} A_{\text{task}}\, \xi_k\Big)_i \geq 1 \qquad (i \in \mathcal{R}_I), \tag{9.1c}$$

$$F\xi_k = b^{(k)}, \quad 0 \leq \xi_k \leq 1 \qquad (k = 1, \dots, K), \tag{9.1d}$$

where $b^{(k)}$ has the entry $+1$ at the source, -1 at the sink and 0 elsewhere, and $\mathcal{R}_E$, $\mathcal{R}_I$ are the sets of equality and inequality coverage rows. This is a multi-block linear program (Definition 2.1.1) with identical block polytopes. A coverage row in $\mathcal{R}_E$ requires a flight to be operated exactly once; a row in $\mathcal{R}_I$ requires it at least once, which allows a pilot to travel on a flight as a passenger. The columns of the master problem are complete schedules, and the pricing problem (2.5) of pilot k is a shortest path problem with the edge weights $c^{(k)} - A_{\text{task}}^{\top}\pi$. These weights can be negative, but the graph is acyclic, so one pass of dynamic programming in topological order solves it in $O(|V| + |E|)$.

9.1.2 Instance generator

The instances are generated randomly but reproducibly.

- **Network.** The nodes $1, \dots, |V|$ are ordered in time. A backbone edge connects every node to the next one, which guarantees a source–sink path. A skip edge from a node to any node at least two steps later exists independently with a fixed probability. An edge that spans ℓ steps costs $\ell + U(0,1)$, multiplied by $1 + 0.1\,U(0,1)$ for each pilot separately, where $U(0,1)$ is a uniform random number.
- **Flights.** Each skip edge operates a random set of up to ten flights whose position in time is close to that of the edge. A flight is only attached to edges that overlap in time, so that a single schedule operates every flight at most once. The task matrix therefore has a banded structure.
- **Feasibility.** Only flights that can be covered exactly once in the linear relaxation are kept, so that the equality variant is feasible.
- **Inequality rows.** In the inequality variant, a random 30% of the coverage rows are turned into "≥ 1" rows.

Table 9.1 lists the six instance sizes. The instances I1–I4 are used for the comparison of all methods; S5 and S6, with up to 960 coverage rows, extend the study of scaling. The last column is the number of variables of the full edge-based formulation (9.1), which HiGHS solves directly to provide the reference optimum $z^\star$.

instance	$\lvert V\rvert$	$\lvert E\rvert$	K	m	m_I (inequality variant)	variables of (9.1)
I1	50	156	4	38	11	624
I2	150	1 146	7	225	68	8 022
I3	175	1 546	10	356	107	15 460
I4	225	2 434	13	480	144	31 642
S5	300	3 863	16	668	200	61 808
S6	400	5 932	20	960	288	118 640

Table 9.1: Test instances. K is the number of pilots, m the number of coverage rows, and m_I the number of "$\geq$" rows in the inequality variant.

9.2 Implementation and Setup

All methods are implemented in Julia 1.12 [2] in one framework: the column generation driver, the pricing algorithm, the column pool and the statistics are shared, and only the solver of the restricted master problem differs. We compare six methods:

method	master problem
Exact LP (HiGHS)	none: formulation (9.1) solved directly by the HiGHS simplex method
LP CG (HiGHS)	classical linear master problem, HiGHS simplex, warm-started
Explicit QP CG (HiGHS)	explicit regularized master problem (3.3), HiGHS QP solver
Eliminated QP CG (HiGHS)	eliminated regularized master problem (3.5), HiGHS QP
QPAS CG, sparse factor	eliminated regularized master problem, Algorithm 6.1 with the factorization of Chapter 6
QPAS CG, dense factor	eliminated regularized master problem, Algorithm 6.1 with the factorization of Chapter 8

HiGHS [17] (version 1.15, with its simplex method for the LPs and its active-set QP solver for the QPs) is used through a persistent model, to which new columns are added incrementally, so that its LP solves are warm-started. The LP master starts with artificial columns of cost $M = 10^4$ on both sides of every linking row (Section 2.3), which keep it feasible until enough columns are generated. The eliminated QP master is run on the equality variant only. The sparse factorization of the active-set method uses CHOLMOD [4] from SuiteSparse 7.10 for the factorization and its updates. The dense factorization is our own implementation of Chapter 8; its triangular solves call the BLAS routine `trsv` of OpenBLAS, and its rank-one modifications are plain loops over the columns of the stored array.

All column generation runs use the same settings: the initial columns are the cheapest path of every pilot, pricing visits the pilots in random order and adds the first improving column, the proximal point rule moves the center after every iteration, and, except in Section 9.9, $\delta = 1$, $\varepsilon_{\mathrm{c}} = 10^{-6}$, $\varepsilon_{\mathrm{rc}} = 10^{-6}$. The active-set method uses the column capacity $n_{\max} = 20\,000$ and the pivot tolerance $\varepsilon_{\mathrm{piv}} = 10^{-10}$ with both factorizations. The sparse factorization is refactorized every $\tau = 150$ updates unless stated otherwise; the dense factorization is only refactorized when the pivot invariant is violated (Section 8.6).

The HiGHS QP masters are slow, so they are run to completion on I1 only; on the equality variant of I2 they are stopped after 120 seconds, which is enough to compare their path and their time per master problem with the active-set method. The LP master is run on I1–I3 with a limit of 900 seconds, and the direct LP solve of S5 and S6 is stopped after 300 seconds. All experiments ran single-threaded on an AMD Ryzen 7 7700 processor with 32 GB of memory under Windows 11. Times of the active-set method are the faster of two runs, except for runs longer than one minute on S5 and S6, which were run once; all other times are from a single run.

9.3 Correctness

Tables 9.2 and 9.3 compare all methods on both variants. Besides the status, the number of column generation iterations, the number of generated columns and the times, they report three accuracy measures relative to the reference optimum $z^\star$:

- the relative error of the cost, $(c^\top \lambda - z^\star)/|z^\star|$;
- the relative gap of the best Lagrangian bound, $(z^\star - \theta)/|z^\star|$, which is nonnegative for every valid bound (Proposition 2.3.4);
- the infeasibility $\|b - A\lambda\|_\infty$ of the final weights in the linking rows (for inequality rows only the violated part).

	method	status	CG iter.	columns	time [s]	master [ms/iter.]	cost error	bound gap	infeasibility
I1	Exact LP (HiGHS)	opt.	–	–	0.01	–	0	0	0
	LP CG (HiGHS)	opt.	204	207	0.07	0.344	0	0	$4.4 \cdot 10^{-16}$
	Explicit QP CG (HiGHS)	opt.	3 960	130	7.14	1.793	$5.4 \cdot 10^{-10}$	$3.5 \cdot 10^{-10}$	$2.9 \cdot 10^{-9}$
	Eliminated QP CG (HiGHS)	opt.	3 960	130	5.31	1.334	$5.4 \cdot 10^{-10}$	$3.5 \cdot 10^{-10}$	$2.9 \cdot 10^{-9}$
	QPAS CG, sparse factor	opt.	3 928	129	0.05	0.009	0	0	$2.4 \cdot 10^{-14}$
	QPAS CG, dense factor	opt.	3 927	133	0.02	0.003	0	0	$2.7 \cdot 10^{-14}$
I2	Exact LP (HiGHS)	opt.	–	–	1.50	–	0	0	0
	LP CG (HiGHS)	opt.	2 410	2 416	70.91	29.371	$-6.2 \cdot 10^{-13}$	$9.1 \cdot 10^{-11}$	$1.3 \cdot 10^{-12}$
	Explicit QP CG (HiGHS)	time	2 042	985	120.05	58.712	$-4.2 \cdot 10^{-3}$	$7.5 \cdot 10^{-4}$	$1.2 \cdot 10^{-2}$
	Eliminated QP CG (HiGHS)	time	1 886	976	120.06	63.445	$-4.3 \cdot 10^{-3}$	$9.0 \cdot 10^{-4}$	$1.3 \cdot 10^{-2}$
	QPAS CG, sparse factor	opt.	11 974	1 268	2.54	0.193	$-2.2 \cdot 10^{-5}$	$1.7 \cdot 10^{-7}$	$5.8 \cdot 10^{-5}$
	QPAS CG, dense factor	opt.	11 974	1 264	1.18	0.081	$-2.3 \cdot 10^{-5}$	$1.7 \cdot 10^{-7}$	$6.0 \cdot 10^{-5}$
I3	Exact LP (HiGHS)	opt.	–	–	6.61	–	0	0	0
	LP CG (HiGHS)	opt.	4 679	4 688	662.53	141.495	0	$1.4 \cdot 10^{-12}$	$7.7 \cdot 10^{-13}$
	QPAS CG, sparse factor	opt.	8 961	1 651	7.79	0.835	$8.7 \cdot 10^{-9}$	$5.4 \cdot 10^{-9}$	$1.4 \cdot 10^{-5}$
	QPAS CG, dense factor	opt.	8 879	1 680	2.96	0.301	$-1.1 \cdot 10^{-8}$	$5.4 \cdot 10^{-9}$	$1.3 \cdot 10^{-5}$
I4	Exact LP (HiGHS)	opt.	–	–	24.16	–	0	0	0
	QPAS CG, sparse factor	opt.	3 216	2 408	22.55	6.939	$-2.7 \cdot 10^{-8}$	$1.6 \cdot 10^{-10}$	$7.8 \cdot 10^{-6}$
	QPAS CG, dense factor	opt.	3 252	2 430	6.98	2.093	$-6.9 \cdot 10^{-8}$	$1.2 \cdot 10^{-10}$	$6.7 \cdot 10^{-6}$

Table 9.2: Equality variant: all methods. “time” denotes a method stopped by its time limit (120 s for the HiGHS QP masters on I2). Master times are averages per column generation iteration.

All methods reach the optimum. Every column generation run that finishes within its time limit stops with an optimal status, and its Lagrangian bound agrees with $z^\star$ to within $2 \cdot 10^{-7}$ relative or better. This confirms the exact termination criterion of Theorem 3.4.1 in practice, including the sign conditions of the inequality rows: the bound is evaluated at the projected duals π^+, since at duals that violate them it would not be valid and could exceed $z^\star$ (Section 3.4.3 and Remark 2.3.5).

	method	status	CG iter.	columns	time [s]	master [ms/iter.]	cost error	bound gap	infeasibility
I1	Exact LP (HiGHS)	opt.	–	–	0.01	–	0	0	0
	LP CG (HiGHS)	opt.	138	141	0.03	0.206	0	0	$1.8 \cdot 10^{-15}$
	Explicit QP CG (HiGHS)	opt.	370	89	0.53	1.409	$5.1 \cdot 10^{-11}$	$2.1 \cdot 10^{-10}$	$1.2 \cdot 10^{-9}$
	QPAS CG, sparse factor	opt.	369	94	0.01	0.021	0	0	$8.4 \cdot 10^{-15}$
	QPAS CG, dense factor	opt.	370	91	0.00	0.005	0	0	$1.1 \cdot 10^{-14}$
I2	Exact LP (HiGHS)	opt.	–	–	1.20	–	0	0	0
	LP CG (HiGHS)	opt.	1 860	1 866	31.28	16.756	$1.1 \cdot 10^{-12}$	$1.1 \cdot 10^{-10}$	$1.0 \cdot 10^{-11}$
	QPAS CG, sparse factor	opt.	1 630	716	0.79	0.468	$-1.9 \cdot 10^{-7}$	0	$5.6 \cdot 10^{-6}$
	QPAS CG, dense factor	opt.	1 669	718	0.36	0.200	$-4.4 \cdot 10^{-7}$	0	$6.9 \cdot 10^{-6}$
I3	Exact LP (HiGHS)	opt.	–	–	5.51	–	0	0	0
	LP CG (HiGHS)	opt.	3 761	3 770	305.14	81.036	0	$3.2 \cdot 10^{-13}$	$3.6 \cdot 10^{-13}$
	QPAS CG, sparse factor	opt.	5 460	1 377	5.09	0.899	$-4.7 \cdot 10^{-9}$	$2.7 \cdot 10^{-10}$	$4.0 \cdot 10^{-6}$
	QPAS CG, dense factor	opt.	5 193	1 392	2.11	0.376	$-1.6 \cdot 10^{-8}$	$2.6 \cdot 10^{-10}$	$5.8 \cdot 10^{-6}$
I4	Exact LP (HiGHS)	opt.	–	–	22.29	–	0	0	0
	QPAS CG, sparse factor	opt.	2 856	2 099	17.06	5.910	$5.9 \cdot 10^{-10}$	$4.8 \cdot 10^{-11}$	$4.3 \cdot 10^{-6}$
	QPAS CG, dense factor	opt.	2 960	2 102	6.22	2.047	$5.9 \cdot 10^{-10}$	$6.7 \cdot 10^{-11}$	$4.3 \cdot 10^{-6}$

Table 9.3: Inequality variant (30% "$\geq$" rows). The eliminated HiGHS QP master is not run on this variant, and the explicit HiGHS QP master is run on I1 only.

The accuracy of the stabilized methods. The stabilized methods stop when $\|\pi - \hat{\pi}\|_\infty \leq \varepsilon_c \max(1, \|\pi\|_\infty)$, which by Proposition 3.4.2 allows an infeasibility of up to $\delta\varepsilon_c \max(1, \|\pi\|_\infty)$ in the linking rows. The infeasibility column shows exactly this order of magnitude, about 10^{-5} for the larger instances, where the duals are of order 10 to 100. The cost $c^\top\lambda$ of such a slightly infeasible solution can be off by more than the infeasibility itself: on I2 it is $2 \cdot 10^{-5}$ below $z^\star$. The Lagrangian bound, which certifies the optimal value, is unaffected and agrees with $z^\star$ to about $2 \cdot 10^{-7}$. A tighter ε_c removes the cost error at the price of a few more center-only iterations. In a test on a smaller instance with $\delta = 5$, the cost error dropped from 10^{-6} for $\varepsilon_c = 10^{-6}$ to 10^{-15} for $\varepsilon_c = 10^{-9}$, with one additional iteration.

The active-set method and HiGHS solve the same problem. On instance I1, where all stabilized methods finish, the explicit and eliminated HiGHS QPs and the active-set method perform almost the same number of column generation iterations and generate almost the same number of columns: they follow the same sequence of master problems, as they should. The small differences come from the accuracy of the master solves, which affects the duals passed to pricing and hence the order in which columns are found.

The two factorizations solve the same problem. The two variants of the active-set method differ only in the linear algebra, so in exact arithmetic they produce the same iterates (Section 8.7). In floating point they agree closely: their final objectives differ by at most $6 \cdot 10^{-7}$ relative on all twelve runs (Table 9.8 below), with the same accuracy against $z^\star$, and on I2 they perform exactly the same 11 974 iterations. Where the iteration counts differ, rounding differences have broken a tie in a ratio test or a pricing round differently, after which the runs continue on slightly different paths. The differences are below 7% on all runs but one: on the inequality variant of S6 the dense variant needs 9 574 iterations against 6 926, with the same final objective.

9.4 Convergence and the Two Phases of Regularized Column Generation

Figure 9.1 shows the convergence of the master objective on instance I2 for all column generation methods, together with the change of the duals from one iteration to the next. Here and in the rest of this section, "QPAS" denotes the active-set method with the dense factorization; the sparse one follows the same path (it performs the same iterations on I2).

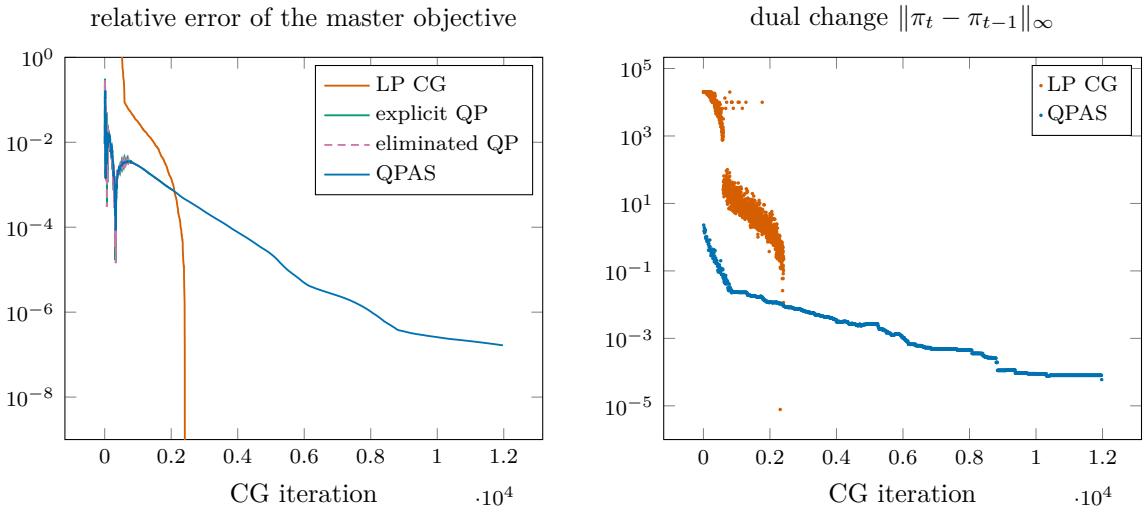


Figure 9.1: Instance I2 (equality variant). Left: relative distance of the master objective to $z^\star$. Right: change of the duals between consecutive iterations. The two HiGHS QP masters stop at their time limit of 120 seconds after about 2 000 iterations.

Stabilization. The duals of the linear master problem jump by up to $2 \cdot 10^4$ in the first iterations, when the artificial columns of cost $M = 10^4$ are still active, and by about 2 in a typical iteration of the second half of the run. The regularized duals change by about 10^{-4} in a typical iteration of the second half. The linear master problem, however, reaches the optimum in fewer iterations on this instance. Its jumps explore the dual space quickly, while the proximal term keeps the regularized duals close to the center, so that they need many small steps. Regularization therefore does not reduce the number of iterations here. What it buys is iterations whose master problems differ only slightly from each other, which is exactly what the hot start exploits: an iteration of the active-set method costs 0.08 ms on average with the dense factorization and 0.19 ms with the sparse one, against 29 ms for the warm-started LP master (Table 9.2).

The QP masters follow the same path. The curves of the explicit QP, the eliminated QP and the active-set method are almost indistinguishable over the first 2 000 iterations, after which the HiGHS QP masters run out of time.

The right panel of Fig. 9.1 shows a second feature of the regularized methods: from some point on, the duals change by very small amounts in most iterations. These are the center-only iterations of Section 3.4. Figure 9.2 shows how the two kinds of iterations are distributed over a run.

Two phases. A run starts with a **column phase**, in which (almost) every iteration adds a column. It is followed by a **proximal phase**, in which most iterations are center-only iterations

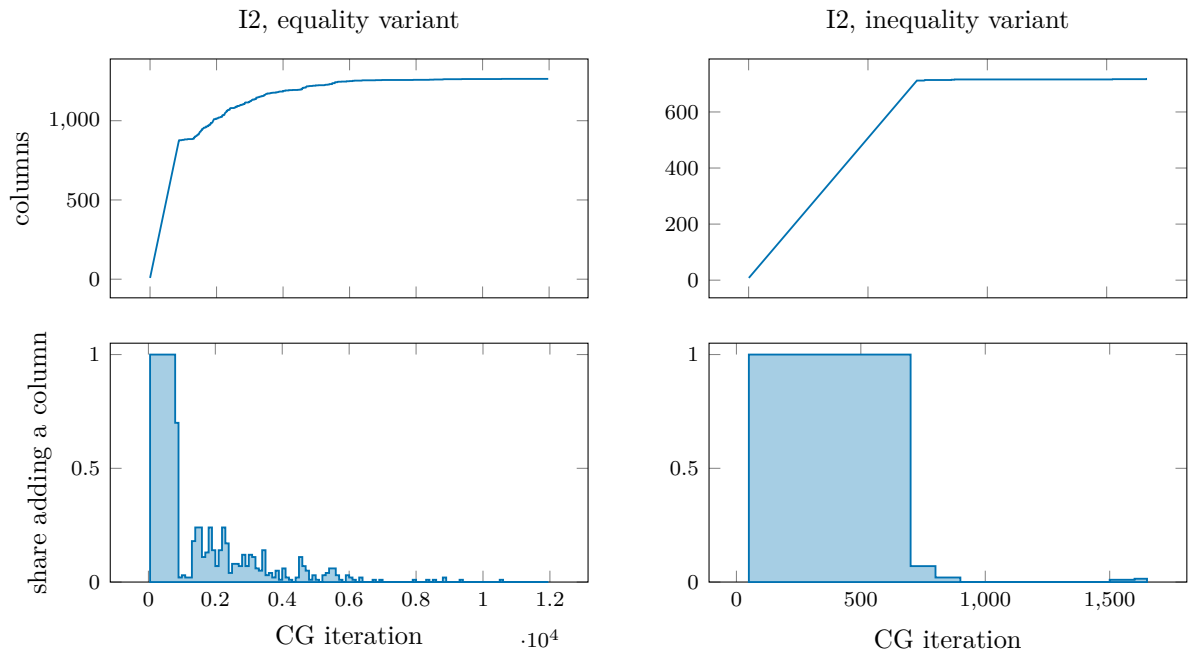


Figure 9.2: The two phases of regularized column generation (active-set method, instance I2). Top: number of columns. Bottom: share of the iterations in each window of 100 iterations that add a column; the remaining iterations are center-only iterations.

that move the center towards the restricted dual optimum, interrupted by occasional iterations that find a new column because the duals have moved. The phases are not strictly separated: columns keep appearing far into the proximal phase. Table 9.4 quantifies this for all instances.

instance	ineq.	CG iter.	with column	center-only	first center-only	last column	mean inner iter. column	mean inner iter. center-only	master time center-only
I1	–	3 927	129	3 798	111	3 925	10.6	1.0	87%
I1	30%	370	87	283	80	368	12.8	1.4	39%
I2	–	11 974	1 257	10 717	870	10 576	34.0	1.8	55%
I2	30%	1 669	711	958	706	1 667	45.5	1.6	10%
I3	–	8 879	1 670	7 209	1 639	5 969	65.7	1.7	22%
I3	30%	5 193	1 382	3 811	1 360	5 192	65.8	1.9	15%
I4	–	3 252	2 417	835	2 408	2 894	87.4	2.0	2%
I4	30%	2 960	2 089	871	2 084	2 769	93.0	2.0	2%

Table 9.4: Column-adding and center-only iterations of the active-set method (dense factorization). “inner iter.” counts the iterations of the active-set method per master problem.

The two kinds of iterations differ sharply in cost. A column-adding iteration needs on average tens of active-set iterations, since the new column is released and the working set adapts to it. A center-only iteration needs at most two on average: as shown in Proposition 6.6.2, once the working set has settled, the direction is zero, and the master problem is solved by a single multiplier test. The length of the proximal phase varies a lot between instances. On I2 and I3 (equality variant) center-only iterations are the large majority, on I4 they are a minority. On all instances except the tiny I1 their share of the master time is much smaller than their share of the iterations.

Figure 9.3 shows how the optimality measures evolve over the two phases for the active-set method on I2. In the column phase, the cost error, the error of the master objective and the

infeasibility decrease together. In the proximal phase, the Lagrangian bound becomes available, since all pilots are priced in every center-only iteration. The bound approaches $z^\star$ from below at the same rate at which the master objective approaches it, so the two errors are indistinguishable in the plot. The infeasibility, and with it the cost error, decrease more slowly and in steps: each step is a change of the working set that ends one of the dual walks of Proposition 6.6.2.

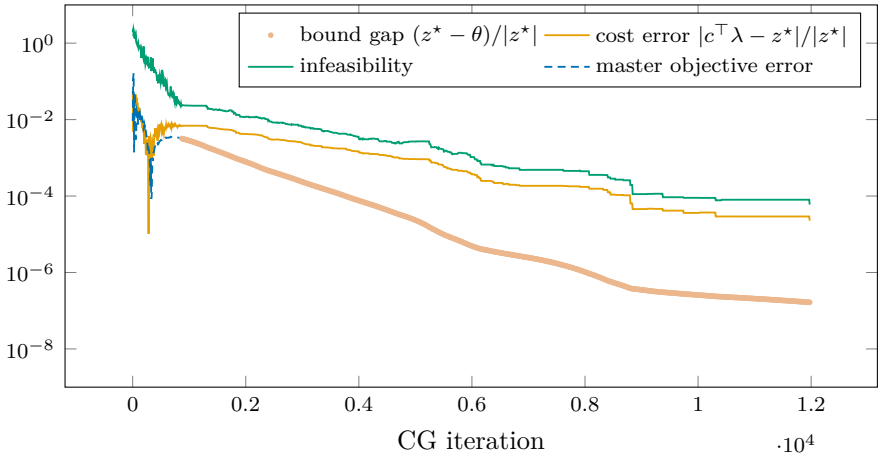


Figure 9.3: Optimality measures of the active-set method over a run (I2, equality variant). The Lagrangian bound is only available in iterations that price all pilots, which are the iterations that find no column.

9.5 Computational Time

Master time per iteration. Figure 9.4 shows the time of the master solve per column generation iteration on I2. The time of the LP master and of the two HiGHS QP masters grows with the number of columns. The HiGHS QP solvers take 80–95 ms per iteration in the second half of their run, since they rebuild their internal factorization for every master problem. The active-set method stays between 0.003 and 1.6 ms per iteration with the dense factorization and between 0.006 and 6.5 ms with the sparse one, with medians of 0.04 and 0.07 ms in the second half of the run. That is about three orders of magnitude below the HiGHS QP masters and almost three orders of magnitude below the median of 30 ms of the warm-started LP master.

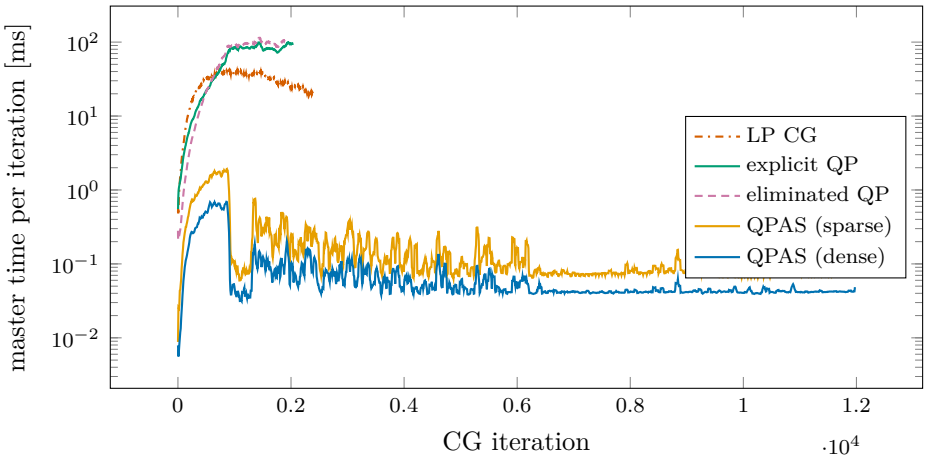


Figure 9.4: Master time per column generation iteration on I2 (moving average over 50 iterations).

Where the time goes. Figure 9.5 splits the time of the active-set method into its operations, for both factorizations, as shares of the total time of the run with the sparse factorization. With the sparse factorization, the largest share, 32–48%, is the direction computation (the right-hand side and the sparse solve with the factor), followed by the updates of the factorization and the periodic refactorization, together 26–55%; the refactorization share grows with the instance size. The dense factorization cuts the direction computation by a factor of 2.5 to 4, the updates by a factor of 3 to 4.5, and the refactorizations almost entirely. What remains is dominated by operations that do not touch the factor: the multiplier test, the only operation whose cost grows with the number of columns, is now about as expensive as the direction computation on the equality variants and takes between a fifth and two fifths of the time of the dense runs. Pricing takes less than 7% of the total time on all instances shown.

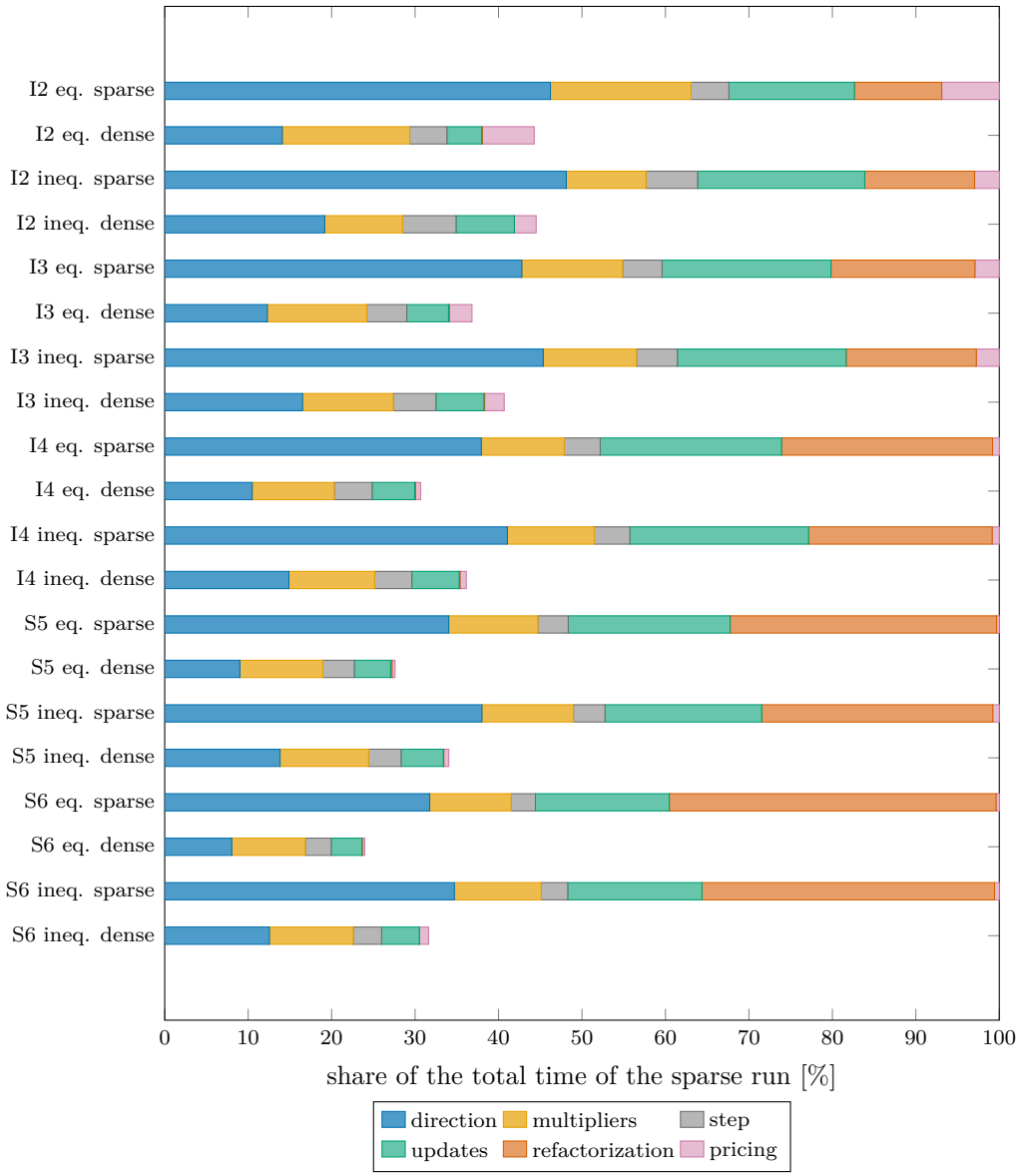


Figure 9.5: Time breakdown of the active-set method and of pricing with the sparse and the dense factorization, as shares of the total time of the sparse run on the same instance. “updates” are the expansions and reductions of the working set without the refactorizations.

Table 9.5 lists the events inside the active-set method for every run. Over a full run, the working set changes tens to hundreds of thousands of times, and every change is a single update

of the factorization. Representative swaps are rare, because representatives carry the largest weights. Singular reduced Hessians occur, but only a handful of times per run, and each is resolved by one zero-curvature step. The final dimension $|\mathcal{N}_x|$ of H_{eff} stays below the number of coverage rows, as guaranteed by Corollary 5.2.5. The sparse factorization is rebuilt hundreds to thousands of times per run, almost all of them periodic refactorizations; the dense one is rebuilt at most 16 times, each time because a singular release had to be undone or a pivot had decayed.

		AS iter.	steps (full)	expans.	swaps	reduct.	surplus exp./red.	sing.	updates	refact.	$\lvert\mathcal{N}_x\rvert$	$\lvert\mathcal{N}_s\rvert$
I1	–	5 354	985 (577)	409	31	442	–	1	1 020	6 / 1	33	–
I1	30%	1 501	773 (445)	328	29	358	54 / 61	0	811	5 / 0	23	7
I2	–	62 208	34 852 (19 690)	15 164	214	15 382	–	2	33 500	221 / 2	218	–
I2	30%	33 849	21 687 (11 414)	10 273	172	10 493	1 953 / 1 987	0	22 497	149 / 0	186	34
I3	–	122 257	77 409 (41 799)	35 615	463	35 969	–	5	78 623	509 / 5	354	–
I3	30%	98 069	62 993 (33 467)	29 529	360	29 883	4 069 / 4 117	3	64 748	424 / 3	306	48
I4	–	212 855	140 409 (71 698)	68 717	669	69 194	–	6	149 798	966 / 6	477	–
I4	30%	196 102	129 336 (66 015)	63 327	463	63 806	11 199 / 11 261	6	134 830	878 / 6	417	62
S5	–	440 459	291 882 (148 030)	143 868	921	144 518	–	16	310 444	2 012 / 16	650	–
S5	30%	382 858	253 540 (128 835)	124 709	621	125 370	20 955 / 21 038	4	259 667	1 714 / 4	578	83
S6	–	949 298	629 823 (318 670)	311 157	1 543	312 110	–	4	651 627	4 337 / 4	953	–
S6	30%	839 901	556 127 (282 885)	273 243	1 189	274 200	43 098 / 43 209	1	564 924	3 727 / 1	846	111

Table 9.5: Events inside the active-set method (dense factorization) over a whole column generation run: active-set iterations, primal steps (of which full steps), expansions, representative swaps, reductions, expansions/reductions of surpluses, singular steps, factorization updates, refactorizations (sparse / dense factorization), and the final sizes of $\mathcal{N}_x$ and $\mathcal{N}_s$.

Scaling. Figure 9.6 shows the total time of the active-set method with both factorizations and of the direct LP solve as the instances grow, up to S6 with 960 coverage rows, and Table 9.6 gives the details of the two largest instances. From I2 to S6 the time of the dense variant grows almost exactly with the third power of m. Of this, the number of active-set iterations accounts for a factor of about m^2 (from 62 000 on I2 to 950 000 on S6, Table 9.5) and the time per iteration for a factor of about m. The latter is less than the $O(|\mathcal{N}_x|^2)$ of the factor operations because, at these sizes, the parts of an iteration that grow only linearly, such as the right-hand side, the step and the multiplier test, still take a large share of its time. The sparse variant grows faster, about with $m^{3.5}$, because its refactorizations take an increasing share of the time, and the direct LP solve grows faster still. With the dense factorization, the complete column generation run is faster than the direct solution of the full LP from I2 on in the equality variant and on all instances in the inequality variant, by a factor of three to four on I4 and S5. On S6, and on the inequality variant of S5, the direct LP solve did not finish within 300 seconds, while the active-set method with the dense factorization solved both variants of S6 in 84–95 seconds. The inequality variant is consistently faster than the equality variant, since it needs fewer active-set iterations (Section 9.6).

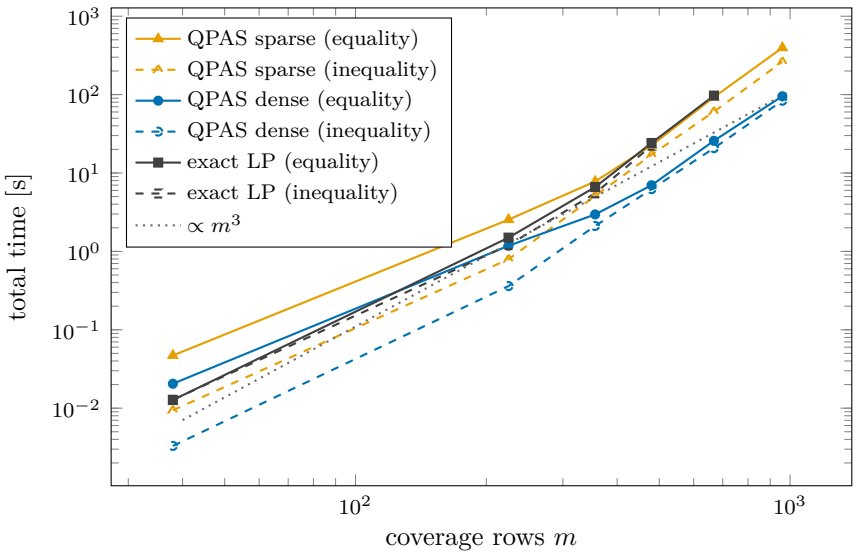


Figure 9.6: Total time against the number of coverage rows for instances I1–I4, S5 and S6. The exact LP of S6 and of the inequality variant of S5 did not finish within 300 seconds.

	ineq.	exact LP [s]	factor	status	CG iter.	columns	time [s]	master [ms/iter.]	refact.	$\lvert\mathcal{N}_x\rvert$	bound gap	infeasibility
S5	–	97.04	sparse	opt.	4 042	3 853	92.67	22.83	2 012	652	$8.6 \cdot 10^{-9}$	$7.2 \cdot 10^{-5}$
			dense	opt.	4 059	3 854	25.76	6.25	16	650	$6.3 \cdot 10^{-9}$	$5.6 \cdot 10^{-5}$
S5	30%	> 300	sparse	opt.	4 203	3 229	60.61	14.29	1 714	577	–	$3.3 \cdot 10^{-6}$
			dense	opt.	3 948	3 228	20.75	5.15	4	578	–	$3.6 \cdot 10^{-6}$
S6	–	> 300	sparse	opt.	7 837	6 026	396	50.32	4 337	954	–	$1.7 \cdot 10^{-5}$
			dense	opt.	7 365	6 022	95.34	12.75	4	953	–	$1.5 \cdot 10^{-5}$
S6	30%	> 300	sparse	opt.	6 926	5 008	262	37.60	3 727	846	–	$1.1 \cdot 10^{-5}$
			dense	opt.	9 574	5 024	83.55	8.40	1	846	–	$9.9 \cdot 10^{-6}$

Table 9.6: The large instances S5 and S6. The bound gap is only available where the direct LP solve finished within its limit of 300 seconds.

9.6 Inequality Rows

With 30% inequality rows, column generation needs fewer active-set iterations on all instances, and fewer column generation iterations on I1–I4, considerably fewer on I1–I3 (Tables 9.2, 9.3 and 9.5): over-covering a flight is allowed, so the optimal schedules are found with fewer columns, and the proximal phase is shorter.

Figure 9.7 follows the sizes of the nullspace sets over a run with inequality rows. Within the first iterations, about 35 surpluses are released: the initial schedules over-cover their rows, so the duals of these rows become negative. These rows drop out of A_{eff}, and by Corollary 5.2.5 they also reduce the room for nullspace columns, $|\mathcal{N}_x| \leq m - |\mathcal{N}_s|$. Over the rest of the run $|\mathcal{N}_s|$ only changes slowly, between 37 and 51, ending at 48, while individual surpluses are released and fixed again thousands of times (Table 9.5). The number of nullspace columns grows during the column phase and ends at $|\mathcal{N}_x| = 306$, just below its bound $m - |\mathcal{N}_s| = 308$: at the end of the run, the effective reduced Hessian is almost as large as it can be.

Surplus operations are frequent (Table 9.5): with inequality rows, 14–19% of all expansions and reductions of the working set on I2–S6 are expansions and reductions of surpluses, each of which is a rank-one update or downdate of the factorization. The average time per master problem of the inequality variant is of the same order as that of the equality variant (Tables 9.2

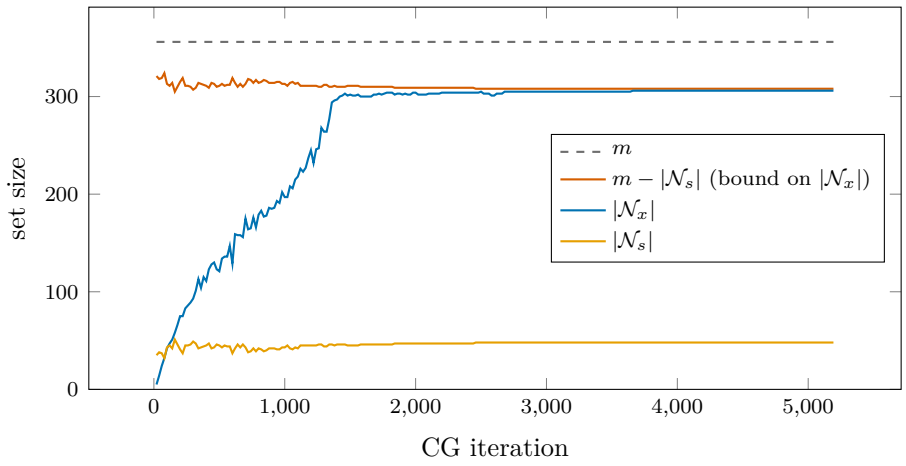


Figure 9.7: Nullspace columns and free surpluses over a run (I3, inequality variant, dense factorization), sampled every 20 iterations.

and 9.3). The remaining differences come from the mix of iterations. On I2, for instance, the inequality variant has relatively few of the cheap center-only iterations (Table 9.4), which raises its average. Singular reduced Hessians, of columns or surpluses, occurred at most six times per run on I1–I4. The singular surplus release of Theorem 7.3.2 is exercised by the test suite of the implementation on constructed instances.

9.7 Fill-In and Refactorization of the Sparse Factor

The factorization of H_{eff} is updated tens of thousands of times per run (Table 9.5). Figure 9.8 shows what this does to the contents of the sparse factor, using the quantities of Definition 6.5.1, without refactorization, with refactorization every 150 updates, and, for comparison, for the dense factor of Chapter 8.

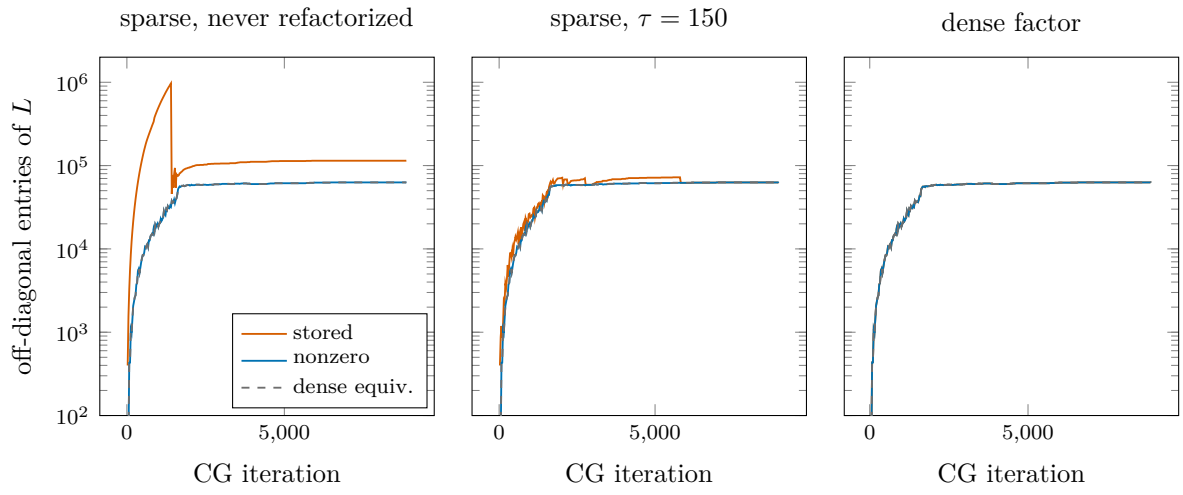


Figure 9.8: Contents of the factor L of H_{eff} over a run (I3, equality variant), sampled every 25 iterations: stored off-diagonal entries, the nonzero ones among them, and the off-diagonal entries of a dense factor of the same dimension. In the right panel the three curves coincide.

The true factor is dense. In all panels the nonzero entries coincide with the dense equivalent: the factor of H_{eff} has essentially no exploitable sparsity. This confirms Remark 6.5.4 and the starting point of Chapter 8: a fill-reducing ordering cannot help, and the cost of a solve grows quadratically with $|\mathcal{N}_x|$.

Structural zeros accumulate. Without the periodic refactorization, the stored pattern of the sparse factor only grows. On I3 it reaches almost a million entries in the column phase, about thirty times the size of the actual factor, and almost all of them are structural zeros. Every solve and every update pays for them, and they connect active indices with inactive ones, which enlarges the reach of the sparse solves. The drop around iteration 1 400 is a refactorization triggered by a degraded pivot (Section 7.2). It resets the pattern, after which it grows again and stays at 1.2 to 2.6 times the size of the factor. With refactorization every 150 updates, the stored pattern stays within a small margin of the effective one for most of the run. The dense factor stores exactly the dense equivalent at every moment (Proposition 8.6.1).

The refactorization interval. Table 9.7 and Fig. 9.9 vary the refactorization interval of the sparse factorization. Without refactorization, the method is three to five times slower, and the sparse factor needs six to eleven times more memory. Refactorizing too often is also expensive, since every rebuild costs $O(|\mathcal{N}_x|^3)$ (Proposition 6.5.3): with an interval of 25, I3 needs almost 3 000 rebuilds and is 70% slower than with the default. Between these extremes the optimum is flat: any interval between 100 and 600 is within about 10% of the best time, and the default of 150 is within 3% of the best on both instances. For the dense factorization, which has no structural zeros, a periodic refactorization only costs time: with $\tau = 150$ it is 5–10% slower than without. Both dense variants are faster than the best sparse one by a factor of 2.1 to 2.6.

	factor	interval τ	time [s]	refact.	max. stored	max. zeros	mean zero share	reach/$\lvert\mathcal{N}_x\rvert$	peak memory [MB]
I2	sparse	never	7.84	2	361 143	343 768	62%	1.85	44.2
	sparse	25	3.78	1 299	25 626	2 420	4%	1.02	6.5
	sparse	50	3.20	662	27 945	4 179	7%	1.04	6.5
	sparse	100	2.75	326	31 572	8 288	17%	1.10	7.2
	sparse	150	2.72	221	33 872	10 226	15%	1.09	7.2
	sparse	300	2.77	110	40 987	17 636	32%	1.22	7.9
	sparse	600	3.00	56	48 151	25 544	42%	1.33	8.8
	dense	never	1.24	2	24 310	48	0%	1.00	0.9
	dense	150	1.30	222	24 310	52	0%	1.00	0.9
I3	sparse	never	37.18	5	971 176	939 055	52%	1.79	125.6
	sparse	25	13.16	2 989	63 500	3 465	2%	1.01	11.3
	sparse	50	9.56	1 502	67 105	5 965	6%	1.03	11.3
	sparse	100	8.09	764	71 179	9 840	8%	1.06	11.3
	sparse	150	7.80	509	72 309	13 305	10%	1.07	11.3
	sparse	300	7.60	261	82 932	21 358	23%	1.16	12.8
	sparse	600	7.65	130	94 279	34 994	21%	1.17	12.9
	dense	never	2.97	5	62 835	58	0%	1.00	1.4
	dense	150	3.25	518	62 835	60	0%	1.00	1.4

Table 9.7: Effect of the refactorization interval (equality variants). “max. zeros” is the largest number of stored entries that are zero; for the dense factor these are entries of L that happen to vanish, not structural zeros. “reach/$|\mathcal{N}_x|$” is the average number of rows touched by a solve per nullspace column. “never” still refactorizes when a pivot degrades. These runs form a separate series (times are the faster of two runs), so the times differ slightly from Table 9.2.

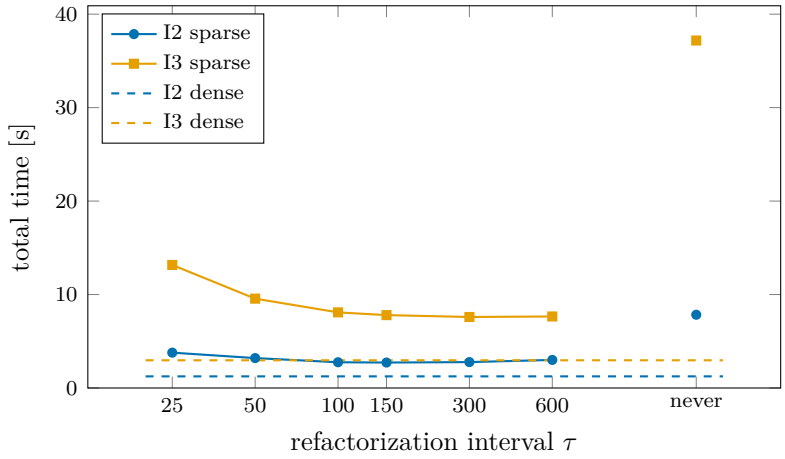


Figure 9.9: Total time of the active-set method against the refactorization interval of the sparse factorization. The dashed lines show the dense factorization without periodic refactorization.

9.8 The Sparse and the Dense Factorization

Table 9.8 compares the two factorizations on all instances and variants. Both variants of the active-set method reach the same objective and the same accuracy (Section 9.3), and their iteration counts are close, so the time difference is caused by the linear algebra.

			CG iter.	master time [s]			total time [s]		refact.	factor memory [MB]		objective
instance	ineq.	m	sparse / dense	sparse	dense	speedup	sparse	dense	sparse / dense	sparse	dense	rel. difference
I1	–	38	3 928 / 3 927	0.037	0.011	3.2	0.047	0.021	6 / 1	5.5	0.5	0
I1	30%	38	369 / 370	0.008	0.002	3.8	0.009	0.003	5 / 0	5.5	0.5	0
I2	–	225	11 974 / 11 974	2.31	0.970	2.4	2.54	1.18	221 / 2	7.2	0.8	$5.2 \cdot 10^{-7}$
I2	30%	225	1 630 / 1 669	0.763	0.334	2.3	0.794	0.361	149 / 0	6.0	0.8	$2.5 \cdot 10^{-7}$
I3	–	356	8 961 / 8 879	7.49	2.68	2.8	7.79	2.96	509 / 5	11.3	1.4	$2.0 \cdot 10^{-8}$
I3	30%	356	5 460 / 5 193	4.91	1.95	2.5	5.09	2.11	424 / 3	10.0	1.4	$1.1 \cdot 10^{-8}$
I4	–	480	3 216 / 3 252	22.32	6.80	3.3	22.55	6.98	966 / 6	17.0	2.2	$4.2 \cdot 10^{-8}$
I4	30%	480	2 856 / 2 960	16.88	6.06	2.8	17.06	6.22	878 / 6	14.8	2.2	0
S5	–	668	4 042 / 4 059	92.28	25.35	3.6	92.67	25.76	2 012 / 16	31.7	3.9	$2.2 \cdot 10^{-7}$
S5	30%	668	4 203 / 3 948	60.06	20.32	3.0	60.61	20.75	1 714 / 4	23.1	3.9	$1.3 \cdot 10^{-9}$
S6	–	960	7 837 / 7 365	394	93.93	4.2	396	95.34	4 337 / 4	62.2	7.5	$5.8 \cdot 10^{-8}$
S6	30%	960	6 926 / 9 574	260	80.38	3.2	262	83.55	3 727 / 1	44.2	7.5	$1.2 \cdot 10^{-8}$

Table 9.8: The two factorizations of the effective reduced Hessian. Master times are summed over the run, the speedup is their ratio, and the factor memory is the peak of the memory held by the factorization.

Speed. The dense factorization makes the master problems 2.3 to 4.2 times faster, and the speedup grows with the number of linking rows: 2.4 to 2.8 at $m = 225$ and 356, 3.3 at $m = 480$, 3.6 at $m = 668$ and 4.2 at $m = 960$ (equality variants). Two effects contribute. First, every operation on the factor is cheaper, as shown below. Second, the sparse factorization has to be rebuilt periodically, and since a rebuild costs $O(|\mathcal{N}_x|^3)$ while the interval stays fixed, these rebuilds take a growing share of its time (Fig. 9.5): almost 40% on S6, with more than 4 000 rebuilds. The dense factorization was rebuilt at most 16 times per run.

Cost per operation. Table 9.9 lists the average time of the individual factor operations as measured inside the runs. On the instances of the study, a solve and a row addition are about five times faster with the dense factorization, and a row deletion five to seven times. Assembling

a new row of H_{eff} from the column pool, which is the same code for both factorizations, now costs more than adding it to the factor.

	solve		row addition		row deletion		assembly of a row	
instance	sparse	dense	sparse	dense	sparse	dense	sparse	dense
I2	22.6	4.8	7.7	1.6	5.0	0.7	2.0	2.6
I3	36.4	7.2	14.8	2.9	8.3	1.3	2.9	4.2
I4	54.7	10.2	25.1	4.9	13.4	2.3	4.2	6.4
S5	96.6	17.5	43.9	8.8	22.7	4.2	7.1	10.9
S6	180.0	32.3	70.5	15.6	39.1	7.7	10.7	18.0

Table 9.9: Average time per factor operation in μs over a column generation run (equality variants). Row additions include those of the refactorizations.

To separate the cost of the operations from the behaviour of the active-set method, Fig. 9.10 and Table 9.10 measure them in isolation. The matrix is the Gram matrix of $k+1$ random sparse columns with entries in $\{-1, 0, 1\}$, which is dense like H_{eff}, both factorizations hold its first k indices, and every operation is repeated and the median time is reported.

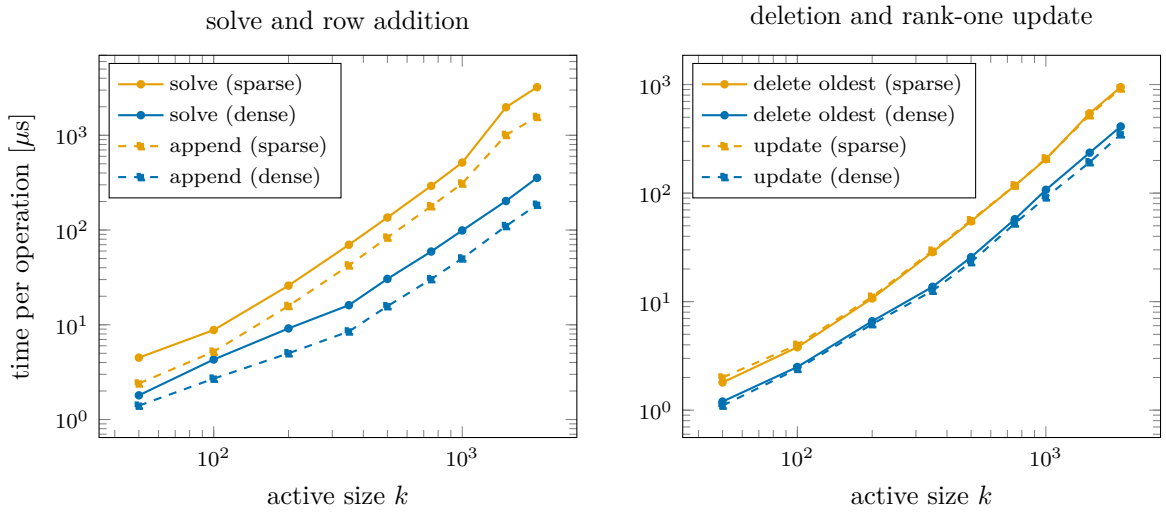


Figure 9.10: Median time of the factor operations against the number k of active indices, for a dense Gram matrix. The sparse factor has the dimension $n_{\max} = 20\,000$.

The dense factorization is faster for every operation and every size. The largest gains are for the operations that the sparse factorization performs through index structures: solves and appends are 2 to 10 times faster, refactorizations 6 to 12 times. Deletions and rank-one modifications, which both factorizations perform by the same column-oriented rank-one recursion, are 1.2 to 2.6 times faster. For the sparse factorization the time per operation grows faster than k^2 beyond $k \approx 1000$, the dense one stays close to k^2. The memory of the dense factor is seven to nine times smaller, since it neither stores indices nor embeds the factor in $n_{\max}$ dimensions.

Deletions. By Proposition 8.5.2, the cost of a deletion depends on the position of the removed index. Over a run, the trailing part $(t-\ell)/t$ that a deletion updates was on average 0.22 to 0.36 of the factor on all instances: as anticipated in Section 8.5, most removed indices had been active only for a short time. The dense factor never needed more than $m+1$ positions, as guaranteed by (8.1) and the one index of an undone singular release (Section 8.1).

	$k = 100$		$k = 350$		$k = 1000$		$k = 2000$	
operation	sparse	dense	sparse	dense	sparse	dense	sparse	dense
solve	8.8	4.3	69.9	16.1	515.6	99.0	3 221	355.3
append / row addition at the end	5.2	2.7	42.3	8.5	309.7	50.1	1 562	184.9
delete the oldest index	3.8	2.5	28.6	13.7	205.5	107.2	947.3	411.4
delete a middle index	2.3	1.9	11.0	7.0	67.8	55.4	313.1	199.2
rank-one update	4.0	2.4	29.5	12.6	208.4	91.4	919.9	348.2
rank-one downdate	4.0	2.4	29.1	12.5	211.2	91.1	966.8	348.2
refactorization (k appends)	2 770	241.0	9 554	1 548	133 505	22 275	1 180 155	137 412
memory [MB]	5.6	0.6	9.9	1.5	62.3	8.3	256.9	31.4

Table 9.10: Median time per operation in μs, and the memory of the factor, against the number k of active indices.

Accuracy. Figure 9.11 shows the residual $\max |(LDL^\top)_{\mathcal{N}_x\mathcal{N}_x} - H_{\text{eff}}| / \max(1, \max |H_{\text{eff}}|)$ of the factor over a run, with H_{eff} recomputed from the columns at every sample. For both factorizations it stays at the level of the rounding error, below $3 \cdot 10^{-15}$ on all runs of I1–I4, and it does not grow over the run. The dense factor is rebuilt only a handful of times per run and yet does not drift, even over almost 150 000 updates on I4: the rank-one modifications of Algorithm 8.1 are stable in practice, as Remark 8.4.3 suggests.

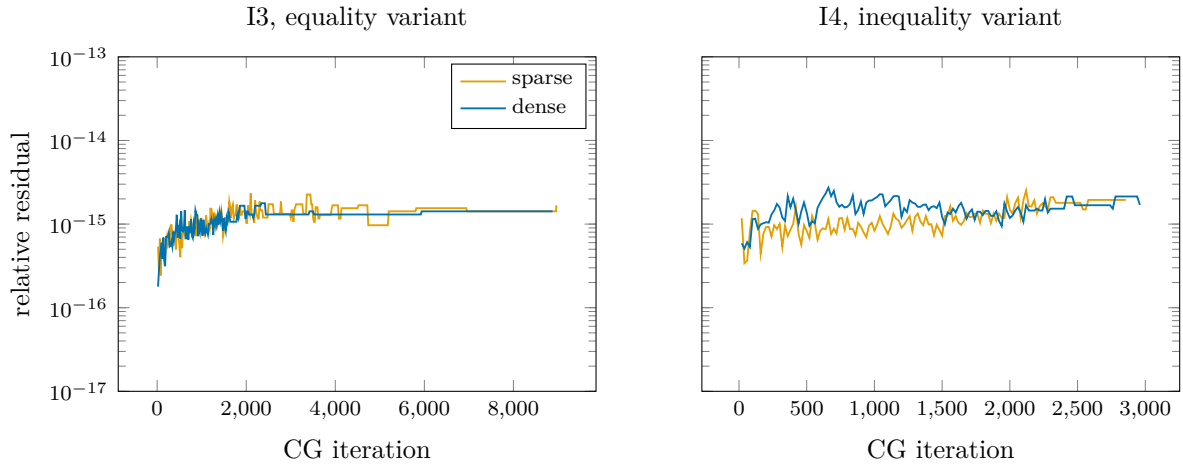


Figure 9.11: Relative residual of the factorization over a run, sampled every 20 iterations.

9.9 An Adaptive Stabilization Parameter

All runs so far used the fixed parameter $\delta = 1$. This section replaces it by the geometric rule of Definition 3.5.3 with $\gamma = 1/2$, $\beta = 2$ and $\delta_{\min} = 10^{-6}$, for the base values $\delta_0 = 1$ and $\delta_0 = 3$. As controls, the fixed parameters $\delta = 0.1$ and $\delta = 3$ are run as well. Every variant is run with two pricing rules: the first-improving pricing of Section 9.2, and **all-blocks pricing**, which prices every pilot in every iteration and adds one column for each pilot with negative reduced cost. All runs use the dense factorization; times are the faster of two runs.

The proximal phase disappears. Table 9.11 shows the effect on the number of iterations. With first-improving pricing and $\delta_0 = 1$, the column-adding iterations are almost the same as with the fixed parameter, but the center-only iterations drop from 10 717 to 233 on I2 and from

		first-improving pricing			all-blocks pricing		
instance	ineq.	fixed $\delta = 1$	geom. $\delta_0 = 1$	geom. $\delta_0 = 3$	fixed $\delta = 1$	geom. $\delta_0 = 1$	geom. $\delta_0 = 3$
I1	–	3 927 (129)	163 (134)	193 (136)	4 043 (81)	108 (71)	142 (87)
I1	30%	370 (87)	105 (91)	124 (91)	388 (48)	62 (42)	118 (76)
I2	–	11 974 (1 257)	1 490 (1 257)	1 543 (1 201)	12 123 (656)	812 (510)	993 (608)
I2	30%	1 669 (711)	725 (710)	752 (674)	1 976 (239)	272 (198)	401 (265)
I3	–	8 879 (1 670)	1 726 (1 676)	1 475 (1 402)	5 885 (399)	448 (343)	833 (600)
I3	30%	5 193 (1 382)	1 416 (1 385)	1 163 (1 119)	2 090 (201)	230 (191)	511 (378)
I4	–	3 252 (2 417)	2 436 (2 416)	1 948 (1 907)	1 640 (304)	337 (267)	603 (458)
I4	30%	2 960 (2 089)	2 104 (2 088)	1 672 (1 643)	1 721 (246)	279 (226)	306 (227)

Table 9.11: Column generation iterations with a fixed and with a geometric stabilization parameter (in parentheses: iterations that add a column).

7 209 to 50 on I3. With all-blocks pricing the reduction is similar. Figure 9.12 shows why on I2. With the fixed parameter, the error of the master objective decreases in a long tail of small steps; with the geometric rule, δ falls by several orders of magnitude in the runs of center-only iterations, the duals reach the restricted optimum in a few steps, and the parameter grows back towards δ_0 when new columns arrive.

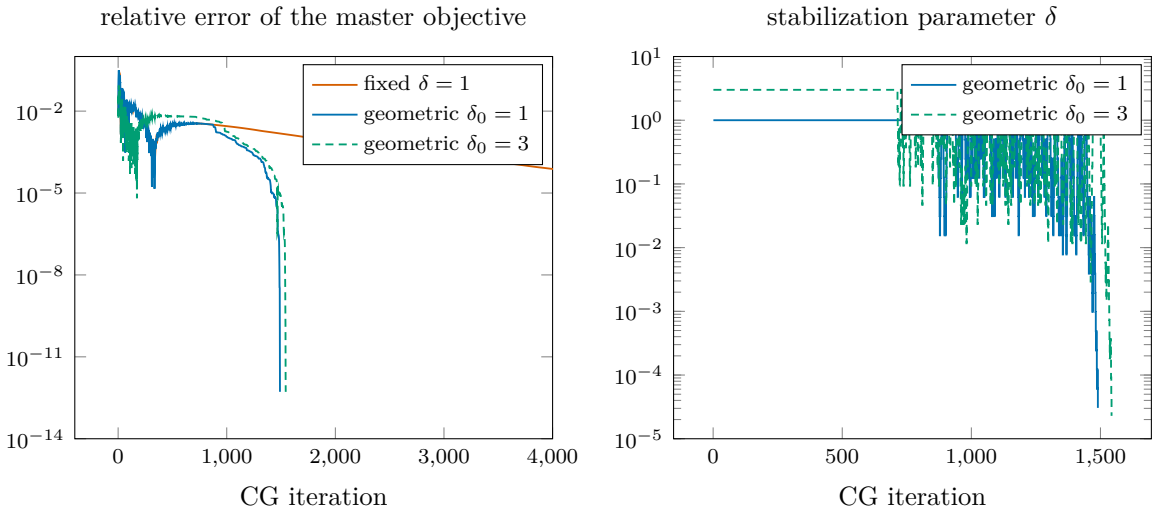


Figure 9.12: Instance I2 (equality variant, first-improving pricing). Left: relative distance of the master objective to $z^\star$; the run with the fixed parameter continues until iteration 11 974. Right: the parameter δ of every master problem under the geometric rule.

Time. Table 9.12 gives the total times. With the best base value, the geometric rule is faster than the fixed parameter $\delta = 1$ on every instance. On I2–I4 the factor is 1.3 (I2, inequality variant) to 1.9 (I3) with first-improving pricing and $\delta_0 = 3$, and 1.2 (I4) to 2.6 (I2) with all-blocks pricing and $\delta_0 = 1$. The speedup is smaller than the reduction of the iteration count, because the center-only iterations were already cheap (Table 9.4): what remains is the column phase, whose iterations cost tens of active-set iterations each. With the geometric rule, all-blocks pricing is faster than first-improving pricing on every instance, and this combination is the fastest configuration overall: I4 takes 1.0 s, against 6.9 s with the setting of the previous sections and 24 s for the direct solution of the full linear program.

		first-improving pricing					all-blocks pricing				
		fixed δ			geometric δ_0		fixed δ			geometric δ_0	
instance	ineq.	0.1	1	3	1	3	0.1	1	3	1	3
I1	–	0.003	0.021	0.074	**0.003**	0.004	**0.002**	0.022	0.064	0.002	0.003
I1	30%	0.002	0.003	0.007	0.002	**0.002**	0.002	0.003	0.007	**0.001**	0.002
I2	–	0.885	1.24	2.01	0.725	**0.723**	0.684	1.15	2.20	**0.438**	0.483
I2	30%	0.670	0.367	0.444	0.310	**0.282**	0.398	0.221	0.365	**0.135**	0.156
I3	–	4.61	2.98	2.46	2.22	**1.58**	3.87	1.41	2.59	**0.744**	1.34
I3	30%	3.77	2.13	1.82	1.76	**1.15**	1.90	0.565	1.04	**0.393**	0.670
I4	–	14.15	6.92	4.44	6.70	**4.14**	5.53	1.19	1.95	**1.00**	1.42
I4	30%	11.39	6.26	4.32	6.11	**3.70**	3.65	1.21	1.35	0.939	**0.773**

Table 9.12: Total time of the column generation run in seconds. The fastest variant of each pricing rule is in bold.

Why no fixed parameter can do this. The controls show that δ acts in opposite directions in the two phases. In the column phase, more stabilization means fewer column-adding iterations: with first-improving pricing on I4 they decrease from 3 631 ($\delta = 0.1$) to 2 417 ($\delta = 1$) and 1 903 ($\delta = 3$), and $\delta = 0.1$ is twice as slow as $\delta = 1$. In the proximal phase, more stabilization means a longer tail: with $\delta = 3$, I2 needs 26 789 iterations. A fixed parameter has to compromise between the two, and the best compromise differs from instance to instance (from 0.1 to 3 in Table 9.12). The geometric rule removes the compromise: δ_0 is chosen for the column phase, and the tail that a large δ_0 would cause is removed by the halvings. With the best base value of the pricing rule, it is at least as fast as the best fixed parameter on every instance except I1, where all variants take a few milliseconds.

The base value depends on the pricing rule. With first-improving pricing, $\delta_0 = 3$ is better than $\delta_0 = 1$ on all instances except the equality variant of I1; with all-blocks pricing it is the other way around, except on the inequality variant of I4. A plausible explanation is the number of columns per iteration. With first-improving pricing a single column enters, and a strong stabilization makes each of these columns count. With all-blocks pricing up to one column per pilot enters, the model improves much faster per iteration, and less stabilization is needed.

Accuracy. Every run ends optimal. Table 9.13 shows that the geometric rule also makes the final solution more accurate: the cost error drops from up to $2.6 \cdot 10^{-5}$ to at most $5.3 \cdot 10^{-13}$, and the infeasibility from up to $6.7 \cdot 10^{-5}$ to the level of rounding errors. This is Proposition 3.5.4 (ii): the stopping test bounds the residual by $\delta\, \varepsilon_c \max(1, \|\pi\|_\infty)$ with the parameter of the last master problem, and in a run that ends in the proximal phase this parameter is several orders of magnitude smaller than δ_0.

		first-improving pricing				all-blocks pricing			
		fixed $\delta = 1$		geom. $\delta_0 = 3$		fixed $\delta = 1$		geom. $\delta_0 = 1$	
instance	ineq.	cost error	infeas.	cost error	infeas.	cost error	infeas.	cost error	infeas.
I1	–	$< 10^{-15}$	$2.7 \cdot 10^{-14}$	$< 10^{-15}$	$< 10^{-15}$	$< 10^{-15}$	$3.9 \cdot 10^{-14}$	$< 10^{-15}$	$1.8 \cdot 10^{-15}$
I1	30%	$< 10^{-15}$	$1.1 \cdot 10^{-14}$	$< 10^{-15}$	$< 10^{-15}$	$< 10^{-15}$	$1.6 \cdot 10^{-14}$	$< 10^{-15}$	$< 10^{-15}$
I2	–	$2.3 \cdot 10^{-5}$	$6.0 \cdot 10^{-5}$	$5.3 \cdot 10^{-13}$	$< 10^{-15}$	$2.6 \cdot 10^{-5}$	$6.7 \cdot 10^{-5}$	$5.3 \cdot 10^{-13}$	$< 10^{-15}$
I2	30%	$4.4 \cdot 10^{-7}$	$6.9 \cdot 10^{-6}$	$< 10^{-15}$	$1.1 \cdot 10^{-15}$	$5.1 \cdot 10^{-7}$	$8.0 \cdot 10^{-6}$	$< 10^{-15}$	$1.6 \cdot 10^{-15}$
I3	–	$1.1 \cdot 10^{-8}$	$1.3 \cdot 10^{-5}$	$< 10^{-15}$	$< 10^{-15}$	$6.0 \cdot 10^{-7}$	$1.3 \cdot 10^{-5}$	$< 10^{-15}$	$< 10^{-15}$
I3	30%	$1.6 \cdot 10^{-8}$	$5.8 \cdot 10^{-6}$	$< 10^{-15}$	$< 10^{-15}$	$6.8 \cdot 10^{-9}$	$6.0 \cdot 10^{-6}$	$< 10^{-15}$	$< 10^{-15}$
I4	–	$6.9 \cdot 10^{-8}$	$6.7 \cdot 10^{-6}$	$< 10^{-15}$	$1.2 \cdot 10^{-15}$	$4.4 \cdot 10^{-8}$	$6.3 \cdot 10^{-6}$	$< 10^{-15}$	$< 10^{-15}$
I4	30%	$5.9 \cdot 10^{-10}$	$4.3 \cdot 10^{-6}$	$3.3 \cdot 10^{-14}$	$< 10^{-15}$	$3.3 \cdot 10^{-14}$	$3.2 \cdot 10^{-13}$	$3.3 \cdot 10^{-14}$	$< 10^{-15}$

Table 9.13: Accuracy of the final solution: relative cost error $|c^\top \lambda - z^\star|/|z^\star|$ and infeasibility of the linking rows.

9.10 Discussion

The experiments support the main claims of this work.

- **Correctness.** The active-set method solves the same sequence of master problems as a general-purpose QP solver, reproduces the optimum of the full linear program, and certifies it by the Lagrangian bound, for equality and inequality rows and with both factorizations.
- **Speed.** With the dense factorization, a master problem of the active-set method costs between 0.003 and 13 ms on average on our instances, and 0.08 ms on I2. That is almost three orders of magnitude less than a HiGHS QP master and two to three orders of magnitude less than a warm-started LP master. This makes regularized column generation much faster than classical column generation, although it often needs more iterations: on I3, 3.0 s against 663 s. The complete run is faster than the direct solution of the full LP from I2 on, and on the largest instances the direct solve does not finish within the time in which the active-set method is done.
- **Inequality rows** add no new kind of linear algebra: the surplus operations are rank-one modifications of the same factorization, free surpluses lower the bound $m - |\mathcal{N}_s|$ on the size of H_{eff}, the time per master problem stays of the same order, and the runs are shorter.
- **The factorization.** The factor of H_{eff} is dense. A sparse factorization needs periodic refactorization to control its structural zeros, and a fixed interval of a few hundred updates works well. A dense factorization in activation order needs none, is two to four times faster with a speedup that grows with m, needs an order of magnitude less memory, and is just as accurate.
- **Adaptive stabilization.** Halving δ in every center-only iteration and doubling it in every iteration that adds a column removes almost all center-only iterations, at no cost to the linear algebra. With the right base value it is 1.2 to 2.6 times faster than a fixed δ on I2–I4, and the final solution is exact up to rounding errors.

The experiments also show the limits of the approach. Since H_{eff} is dense, the cost of the method grows with the cube of the number of linking rows. With a fixed δ the **proximal phase** can be long, since the duals approach the restricted optimum in many small steps (Proposition 6.6.2); the adaptive parameter removes this tail, and what remains is the column phase, whose length

is set by the base value δ_0. With the dense factorization the factor is no longer the dominant cost: the multiplier test, which touches every column of the working set, now costs as much as the direction computation. A natural next step is therefore an incremental multiplier test that avoids rescanning all columns.

Chapter 10

Conclusion

This work developed a hot-started nullspace active-set method for the quadratic master problems of regularized column generation, for multi-block linear programs whose linking constraints may be equalities or inequalities of either direction.

10.1 Summary

The starting point was a **unified formulation** (Chapter 2): surplus variables bring every linking row into equality form, and the sign conditions of the inequality rows become ordinary dual constraints of the surplus columns. Proximal regularization of the dual then leads to a convex quadratic master problem (Chapter 3). Because all linking rows are equalities, its auxiliary variable can be eliminated for all rows, and the result has only the convexity constraints and nonnegativity bounds. Two identities underlie everything that follows: the duals of the linking rows are an explicit affine function of the primal iterate, and the gradient of the quadratic objective consists of the reduced costs of the columns and the duals of the inequality rows.

For the active-set method (Chapter 4), the constraint matrix of a working set admits a **reordering into five sets**: representatives, working-set columns, nullspace columns, active surpluses and free surpluses. The surpluses violate the column generation property of the convexity matrix, but this only adds a group with identity structure (active surpluses) and a group of empty columns (free surpluses). The reordering yields an explicit nullspace basis of vectors $e_j - e_{\mathrm{rep}(j)}$ and unit vectors, which is never stored (Chapter 5). The reduced KKT system **decouples**: the surplus part of the direction is explicit, and only the **effective reduced Hessian** $H_{\mathrm{eff}} = (A_{\mathrm{eff}} Z_x)^\top (A_{\mathrm{eff}} Z_x)$ has to be factorized. Here A_{eff} holds the equality rows and the inequality rows with active surplus. The column part of the method is thus the method for equality rows, applied to a linking matrix whose rows change with the surplus working set. The Lagrange multipliers are the reduced costs and the dual signs, so the optimality test of the master problem and the pricing test are the same test.

Every change of the working set is a row addition, a row deletion or a rank-one modification of an $LDL^\top$ factorization of H_{eff} (Chapter 6). A new column or a new stability center leaves the factorization unchanged, which gives a hot start of every master problem. Singular reduced Hessians are resolved by an exact zero-curvature step, which removes the dependency and keeps every working-set variable except the released one at its bound, for released columns and released surpluses alike (Chapter 7).

The factorization was realized in two ways. A sparse factorization of fixed dimension $n_{\max}$, updated with CHOLMOD, accumulates structural zeros, which periodic refactorization in increasing index order controls (Chapter 6). Since H_{eff} is small and dense, it can instead be

factorized densely in the order in which its indices became active (Chapter 8). A released column is then appended, a removed index is deleted by a rank-one update of the positions behind it, and the surplus operations are rank-one updates and downdates by the method of Gill, Golub, Murray and Saunders [13]. This factor never contains a stale entry and needs no periodic refactorization; its new pivot is exactly the Schur complement of a released column, its downdates fail exactly when the downdated matrix is singular, and removals never decrease a pivot.

On generated crew scheduling instances (Chapter 9), the method reproduces the optimum of the full linear program and certifies it by the Lagrangian bound, with both factorizations. With the dense factorization, its master problems cost between 0.003 and 13 ms on average, almost three orders of magnitude less than a general-purpose QP solver, and fast enough to make regularized column generation much faster than classical column generation with a warm-started LP master: 3.0 s instead of 663 s on the largest instance on which both were run. The dense factorization is two to four times faster than the sparse one, with a speedup that grows with the number of linking rows, needs an order of magnitude less memory, and is just as accurate. The complete column generation run is faster than solving the full linear program directly from the second instance on, by a factor of three to four on the larger instances, and on the largest ones the direct solve did not finish in the time in which the active-set method was done. Inequality rows need no new kind of linear algebra, keep the time per master problem of the same order, and lead to shorter runs.

The stabilization parameter is the step size of a proximal point method on the restricted dual (Section 3.5), and for this method a change of it is free. A geometric rule that halves δ in every center-only iteration and doubles it, up to a base value, in every iteration that adds a column removes the long proximal phase of a fixed parameter. It makes the runs 1.2 to 2.6 times faster on I2–I4, with all-blocks pricing 6.9 times faster on I4 than the fixed parameter with first-improving pricing, and it makes the final solution exact up to rounding errors (Section 9.9).

10.2 Outlook

The experiments point to three directions for further work.

- **A cheaper multiplier test.** With the dense factorization, the linear algebra is no longer the dominant cost. The multiplier test, which computes the reduced cost of every working-set column at every subspace minimizer, now costs as much as the direction computation. Since a step changes π only by a multiple of $\bar{A}p$, the reduced costs could be updated incrementally, or the test could be restricted to a candidate list as in partial pricing.

- **Adaptive stabilization in the column phase.** The geometric rule leaves δ at its base value while columns are added, and the best base value depends on the instance and on the pricing rule (Section 9.9). Proximity control as in bundle methods [19], which compares the actual increase of the Lagrangian bound with the increase predicted by the restricted model, would adapt δ in this phase too, at the price of pricing all blocks in every iteration.

- **Inexact master solves.** Since the multipliers are the reduced costs, pricing could be called at any subspace minimizer instead of only at the optimum of the master problem. This would turn column generation into a single active-set method over all columns.

Bibliography

[1] H. Ben Amor, J. Desrosiers, and A. Frangioni. On the choice of explicit stabilizing terms in column generation. *Discrete Applied Mathematics*, 157(6):1167–1184, 2009.

[2] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017.

[3] O. Briant, C. Lemaréchal, Ph. Meurdesoif, S. Michel, N. Perrot, and F. Vanderbeck. Comparison of bundle and classical column generation. *Mathematical Programming*, 113(2):299–344, 2008.

[4] Y. Chen, T. A. Davis, W. W. Hager, and S. Rajamanickam. Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate. *ACM Transactions on Mathematical Software*, 35(3):22:1–22:14, 2008.

[5] G. B. Dantzig and P. Wolfe. Decomposition principle for linear programs. *Operations Research*, 8(1):101–111, 1960.

[6] T. A. Davis and W. W. Hager. Modifying a sparse Cholesky factorization. *SIAM Journal on Matrix Analysis and Applications*, 20(3):606–627, 1999.

[7] T. A. Davis and W. W. Hager. Row modifications of a sparse Cholesky factorization. *SIAM Journal on Matrix Analysis and Applications*, 26(3):621–639, 2005.

[8] G. Desaulniers, J. Desrosiers, and M. M. Solomon, editors. *Column Generation*. Springer, New York, 2005.

[9] O. du Merle, D. Villeneuve, J. Desrosiers, and P. Hansen. Stabilized column generation. *Discrete Mathematics*, 194(1–3):229–237, 1999.

[10] M. C. Ferris. Finite termination of the proximal point algorithm. *Mathematical Programming*, 50(1–3):359–366, 1991.

[11] R. Fletcher. A general quadratic programming algorithm. *Journal of the Institute of Mathematics and its Applications*, 7(1):76–91, 1971.

[12] M. P. Friedlander and P. Tseng. Exact regularization of convex programs. *SIAM Journal on Optimization*, 18(4):1326–1350, 2007.

[13] P. E. Gill, G. H. Golub, W. Murray, and M. A. Saunders. Methods for modifying matrix factorizations. *Mathematics of Computation*, 28(126):505–535, 1974.

[14] P. E. Gill, W. Murray, and M. H. Wright. *Practical Optimization*. Academic Press, London, 1981.

[15] G. H. Golub and C. F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, 4th edition, 2013.

[16] O. Güler. On the convergence of the proximal point algorithm for convex minimization. *SIAM Journal on Control and Optimization*, 29(2):403–419, 1991.

[17] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

[18] J. E. Kelley. The cutting-plane method for solving convex programs. *Journal of the Society for Industrial and Applied Mathematics*, 8(4):703–712, 1960.

[19] K. C. Kiwiel. Proximity control in bundle methods for convex nondifferentiable minimization. *Mathematical Programming*, 46(1–3):105–122, 1990.

[20] M. E. Lübbecke and J. Desrosiers. Selected topics in column generation. *Operations Research*, 53(6):1007–1023, 2005.

[21] O. L. Mangasarian and R. R. Meyer. Nonlinear perturbation of linear programs. *SIAM Journal on Control and Optimization*, 17(6):745–752, 1979.

[22] J. Nocedal and S. J. Wright. *Numerical Optimization*. Springer, New York, 2nd edition, 2006.

[23] A. Pessoa, R. Sadykov, E. Uchoa, and F. Vanderbeck. Automation and combination of linear-programming based stabilization techniques in column generation. *INFORMS Journal on Computing*, 30(2):339–360, 2018.

[24] R. T. Rockafellar. Monotone operators and the proximal point algorithm. *SIAM Journal on Control and Optimization*, 14(5):877–898, 1976.

[25] A. Schrijver. *Theory of Linear and Integer Programming*. Wiley, Chichester, 1986.